George Grätzer

# Python Companion

## Vibe Coding for Legal Professionals

Complete reference programs and working inputs

October 24, 2026
Release `legal-copyedit-2026-10-24.2`

# Contents

Introduction v

1 Build a Matter Timeline 1

2 Check the Matter Records 5

3 From Eighteen Records to 12,000 Events 9

4 Check Consultation Completion 13

5 Catalogue Matter Documents 19

6 Check a Document Register 23

7 Prepare a Limited Management Extract 29

8 Match Requests and Responses 33

9 Define the Matters in a Report 41

10 Report Monthly Turnaround 47

11 Monitor Weekly Requests 53

12 Build a Production Index 57

13 Prepare Administrative Letters 63

14 Track Professionally Supplied Dates 69

15 Search Names and Parties 75

16 Organize Legal Authorities 79

17 Compare Document Versions 85

18 Check Time Entries and Totals 89

**19 Review Files Recorded as Closed** **95**

**20 Test the Applications** **101**

**21 Package a Reproducible Demonstration** **103**

**22 Check Application-Review Records** **107**

**23 Build an Integrated Matter Worklist** **111**

**A The Shared Opening Implementation** **117**

**B The Shared Application Infrastructure** **123**

**C The Later Reference Implementations** **131**

**D Large-File Conversion** **163**

**E Regression Tests and Expectations** **167**

**F Reproducible Experiments and Baseline Runs** **187**

**G Export Preparation and Additional Regression Cases** **197**

# Introduction

The main book contains prompts, examples, results, and checks rather than full programs. This companion prints the canonical source files included in the release. Run those files directly; do not reconstruct a long program from the PDF unless you specifically need an extracted copy. The printed listings are generated with LaTeX's file-listing command from the actual program files.

Every application has literal input files and saved reference output. The schemas and small inputs below let you inspect the complete record structure. Large CSVs are supplied as actual files; only their headers and selected facts are printed here. The baseline chapter map is `CHAPTER_FILES.json`; `SUPPLEMENTARY_CASES.json` maps the longer export, repair, recorded-build, irregular, and integration cases. Exact CSV headers are also recorded in `data/SCHEMAS.json`.

## Chapter Folders

The folder `Chapters` contains one folder per chapter, `chapter01` through `chapter23`. Each ordinary application folder holds the programs and inputs for its baseline run, and a `RUN.txt` that lists them, gives the local command, and records the printed result of a test run. Supplementary cases may use files elsewhere in the complete release; follow their entries in `SUPPLEMENTARY_CASES.json`. Upload those files to an assistant, or open a terminal in the chapter folder and run the command. In a chapter folder the programs find their inputs beside them; in the release folder they use `data/chNN`. The script `tools/make_chapter_folders.py` rebuilds the chapter folders from the canonical files and test-runs each one. Chapters 20 and 21 work in the release folder itself.

## Run Locally on a Personal Computer

The applications use the Python standard library. They do not require pandas, an AI account, a web server, or a network connection. The earlier release was tested with Python 3.12 and 3.13 on Linux. The current release records its interpreter and fresh results in `verification`; historical checks are kept separately and are not evidence of a new run. The code uses features available in Python 3.10 and later; rerun the included tests in your own environment. Windows and macOS have not been separately validated.

On macOS or Linux, open Terminal in the extracted release folder and check:

```
python3 --version
```

On Windows, open a terminal in the release folder and check `python --version` or `py --version`. In the commands below, replace the initial `python3` with the interpreter command that works on your computer. Do not change a filename merely because the interpreter command differs.

For installation, use Python's official instructions for your operating system.[1] The interpreter is separate from a text editor. Installing an editor alone does not establish that a Python command is available.

## Reproduce the Opening Result

First open `reference_outputs/ch01/report.html` in a browser. Then, from the root of the extracted release, run:

```
python3 programs/matter_view.py --input data/matter_events_demo.csv
--matters data/matters_demo.csv --out my_first_run
```

Open `my_first_run/report.html`. It should contain eighteen accepted records and no held records. Select M01, then E02, and inspect SRC02. The other output files are `accepted.csv`, `review.csv`, and `run.json`.

The destination must be new. A second run to the same path should refuse rather than replace your first evidence. Use `my_second_run` for another run. Do not delete an earlier result merely to make a command succeed.

## Run a Later Application

For Chapter 4, the universal runner is:

```
python3 programs/run_chapter.py 4 --out my_consultations
```

Its equivalent named entry point is:

```
python3 programs/consultation_report.py --out my_consultations_named
```

The matching `data/ch04` folder is selected by default. For an experiment using a copied input folder, add `--data` followed by that folder's path.

Every named entry point for Chapters 4–19, 22, and 23 is a short wrapper that delegates to its chapter's function in `programs/legal_apps.py`; the command-line and output infrastructure is in `run_chapter.py` and `legal_common.py`. Keep these files together with the wrapper. Each chapter below prints its own function, and the complete shared listings are in the appendices.

[1] Python documentation, *Using Python on Windows*, https://docs.python.org/3/using/windows.html; and *Using Python on a Mac*, https://docs.python.org/3/using/mac.html; checked September 24, 2026.

### Command-Line Options

The universal runner and the named entry points share one parser. Run any of them with `--help` to print the options. Only the options a chapter's function accepts are passed on and recorded in `run.json`; the others are ignored for that chapter.

| Option | Chapters | Meaning and default |
|---|---|---|
| `--out` | all | New output folder; required, never overwritten. |
| `--data` | 4–19, 22, 23 | Folder holding the chapter's complete inputs; default `data/chNN`, or the folder beside the program in a chapter folder. |
| `--as-of` | 4, 8, 9, 10, 14, 19, 22, 23 | Explicit historical reporting date, `YYYY-MM-DD`; default 2026-09-24, except 2026-09-15 for Chapter 9 and 2026-12-31 for Chapter 10. |
| `--window` | 8, 23 | Inclusive response window in days; default 14. |
| `--query` | 15 | Name to search for; default `Alex Morgan`. |
| `--exact-only` | 15 | Switch off the one-edit spelling rule. |
| `--whitespace` | 17 | Ignore leading/trailing whitespace and collapse internal whitespace runs for status; keep the original texts and literal difference marks. |
| `--reverse` | 17 | Compare the two files in the other direction. |
| `--artifact` | 22 | Declared artifact version; default 2. |
| `--rule` | 22 | Declared rule version; default 1. |
| `--matter` | 3, 23 | Select one matter for the delivered tables after whole-file checking; the three opening tools take it too. |

The opening tools of Chapters 1–3 take `--input`, `--matters`, `--out`, and `--matter` instead of `--data`; the converter of Chapter 23 takes `--events`, `--matters`, `--assignments`, and `--out`; and the export preparation of Chapter 2 takes `--input`, `--matters`, `--out`, and an optional `--corrections` file.

## Run the Tests and the Controlled Experiments

From the release folder, run:

```
python3 -m unittest discover -s programs -p "test_*.py" -v
```

The current recorded suite gives its test count and pass/fail result in `verification/regression_tests.txt`. Check the count as well as the result of your own fresh run: discovering no tests establishes nothing. The suite covers the opening application, the later applications, and all twenty-three chapter

experiments. Expected small results were established separately and recorded in `expected/baselines.json`; the tests do not generate their own expected answers from the program under test.

To reproduce Chapter 4's controlled correction, run:

```
python3 tools/run_experiments.py 4 --out my_experiment_04
```

The helper copies the original input, makes the declared change, runs the reference, and saves an experiment record. Other chapters use their corresponding number. Chapters 20 and 21 intentionally test a faulty copy and a package boundary; read their descriptions before interpreting a failure or an excluded file.

## When the First Run Fails

If the interpreter is not found, establish which Python command is installed before changing the program. If a script cannot be found, check the current directory: it should contain `programs`, `data`, and `START_HERE.txt`. If an imported module is missing, retain the shared modules beside the entry point rather than copying only the small wrapper.

A missing-column error usually means the wrong input file, an altered header, or a file saved in the wrong format. Compare the literal schema below. A permission error concerns the destination or operating system; choose an authorized location without bypassing access controls. An existing-output error is intentional protection. If writing is interrupted, do not describe a partial folder as a completed run.

An assistant without file execution can discuss the code but cannot establish that it ran. Use the local route or another suitable execution environment, then inspect the actual files. The browser reports are ordinary saved HTML; no server needs to be started.

## Additional Teaching Cases

Chapter 2's export preparation uses `prepare_event_export.py` with `data/export_example`. Chapter 13's constructed repair lesson uses `teaching_cases/letter_repair`; Version 1 intentionally contains the demonstrated defect, and Version 2 is the corrected application. Chapter 23's mixed and reassigned fixtures are `data/ch23_mixed` and `data/ch23_reassigned`. They are separate from the baseline chapter folders. The appendix *A Recorded Build Session* of the main book is supported by `teaching_cases/recorded_build`: three programs an AI assistant actually produced from the Chapter 8 files, their outputs, and the abridged transcript in `SESSION.txt`. Those programs are the assistant's, kept as evidence; they are not reference implementations, and unlike the reference applications they overwrite their own output folders.

Reproduce all these additional cases into a new folder from the release root:

```
python3 tools/reproduce_learning_cases.py --out my_learning_cases
```

The saved reference is `reference_outputs/learning`. Its `recorded_build_probes` folder adds four one-change comparisons of the historical third matcher with the reference: a blank owner, a blank source, an unknown matter, and a future request. Each saves the changed input, both results, and `comparison.json`; `checks.json` at the probe root summarizes the expected differences. These probes preserve the historical program unchanged and demonstrate why agreement on the original fixture is not general equivalence. The regression suite includes additional cases in `programs/test_revision_cases.py`; their expectations name outputs and identities independently of the implementation.

## Rebuild the Books

The main source, both companion sources, shared style files, program files, and figures belong together. Use pdfLaTeX and makeindex with the listed standard packages, including Latin Modern and microtype; Python alone cannot typeset LaTeX, and no shell-escape execution or private font file is required. Font substitution is not part of the intended build. Run `build.py` from the release folder. Before typesetting, the build synchronizes repeated exercise questions and the Python Companion's source-listing ranges. The range helper, `tools/sync_source_listings.py`, derives all eighteen application blocks from the current Python syntax tree rather than relying on old line numbers. The source enables Unicode mappings for searchable text; after any font change, extract and copy representative prompts, filenames, and words containing ligatures before distributing the PDFs. Building the books requires LaTeX; running the applications does not.

Two author-side tools complete a release. `tools/check_browser.py` opens every saved report in Chromium through Playwright, checks it, and regenerates every figure in `figures` from the saved reports; it is a tool for producing the book, not a dependency of the applications. `tools/make_manifest.py` regenerates `SOURCE_MANIFEST.json`, the SHA-256 fingerprint of every packaged file, and is run last, after the chapter folders, reference outputs, figures, and PDFs are in place. After extracting a release into a new folder, run the complete tests and reproduce at least one application, and check that the source listings come from the canonical files named in the manifest: a fresh run can reveal dependencies accidentally available in an author's old working folder. It does not substitute for a human novice trial or professional legal review.

This edition preserves the small teaching examples and records additional boundary checks. Chapter 18 now states a bounded numeric range and precision, with explicit review reasons for unsupported amounts. The release identifier on the title page ties this companion to the current programs, data, generated results, and verification

records; it does not certify suitability for office use.

## Literal Values in the Teaching Files

A whole-number count, delay, or version must use ordinary decimal digits, with no sign, decimal point, or surrounding spaces; a version must also be at least one. Thus `8` is a whole-number cell, while `8.0` and `+8` are not. A blank is missing rather than zero. Chapter 18 adds bounds for minutes and decimal rates. Identifier cells are compared as supplied; the programs do not silently trim, change case, or repair them. Required text must contain something other than whitespace, and dates must have the stated year-month-day form.

Chapter 1

# Build a Matter Timeline

**Row unit.** One recorded event; a separate register identifies each matter.

**Reference entry point.** `programs/matter_view.py`

The opening entry points share the complete `matter_tools.py` implementation, printed in Appendix A. They expose the same checking rules. The main book explores those rules progressively.

```
python3 programs/matter_view.py --input data/matter_events_demo.csv
--matters data/matters_demo.csv --out my_ch01
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch01/report.html`

**Outputs.** `report.html`, `accepted.csv`, `review.csv`, `run.json`.

## Complete Small Inputs and Their Headers

### matter_events_demo.csv

Exact columns, in order: `event_id`, `client_id`, `matter_id`, `event_date`, `event_type`, `request_id`, `source_id`, `summary`.

`event_id`
Unique recorded-event identifier; when a value appears twice, both rows are held.

`client_id`
Designated synthetic client identifier; it must agree with the client the register names for the matter.

`matter_id`
Supplied matter key, which must exist in the matter register.

`event_date`
Supplied calendar date of the event in year-month-day form; a blank or impossible date holds the row.

`event_type`
One of `request`, `response`, or `note`; any other value holds the row rather than being corrected.

`request_id`
Request label that, with matter_id, links a response to its request; required on requests and responses and absent on notes.

`source_id`
Supplied source reference, required to be nonblank; it is a traceable label, not authentication.

`summary`
Short supplied description of the event, required to be nonblank and otherwise not interpreted.

data/matter_events_demo.csv

```
event_id,client_id,matter_id,event_date,event_type,request_id,source_id,summary
E01,C01,M01,2026-09-01,request,R01,SRC01,Draft document requested
E02,C01,M01,2026-09-04,response,R01,SRC02,A response is recorded
E03,C01,M01,2026-09-05,note,,SRC03,Administrative review note
E04,C01,M02,2026-09-03,request,R02,SRC04,Draft document requested
E05,C01,M02,2026-09-06,response,R02,SRC05,A response is recorded
E06,C01,M02,2026-09-07,note,,SRC06,Administrative review note
E07,C02,M03,2026-09-05,request,R03,SRC07,Draft document requested
E08,C02,M03,2026-09-08,response,R03,SRC08,A response is recorded
E09,C02,M03,2026-09-09,note,,SRC09,Administrative review note
E10,C02,M04,2026-09-07,request,R04,SRC10,Draft document requested
E11,C02,M04,2026-09-10,response,R04,SRC11,A response is recorded
E12,C02,M04,2026-09-11,note,,SRC12,Administrative review note
E13,C03,M05,2026-09-09,request,R05,SRC13,Draft document requested
E14,C03,M05,2026-09-12,response,R05,SRC14,A response is recorded
E15,C03,M05,2026-09-13,note,,SRC15,Administrative review note
E16,C03,M06,2026-09-11,request,R06,SRC16,Draft document requested
E17,C03,M06,2026-09-14,response,R06,SRC17,A response is recorded
E18,C03,M06,2026-09-15,note,,SRC18,Administrative review note
```

### matters_demo.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/matters_demo.csv

```
matter_id,client_id,description
M01,C01,Agreement review
M02,C01,Employment documents
M03,C02,Estate administration
M04,C02,Commercial lease
M05,C03,Tenancy records
M06,C03,Service agreement
```

## One Controlled Experiment

In a working copy, give E03 the date already carried by E02. Keep both identifiers and all other values. Predict what the viewer should preserve: eighteen accepted events in the whole input, three events for M01, and separate selectable records for E02 and E03. Their horizontal positions coincide, but their rows do not.

Open the rerun and select both events. This checks more than the total. An implementation that draws one marker over the other might retain both CSV rows yet make one inaccessible on the page. The reference places events on distinct rows, so the second record remains selectable. Save both reports: the old page shows the date change, and the new page shows that the identities survived it.

```
python3 tools/run_experiments.py 1 --out my_experiment_01
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/matter_view.py

```
"""Entry point for the shared, checked synthetic matter application."""

from matter_tools import main

if __name__ == "__main__":
    main()
```

The checking rules this entry point runs are implemented in `programs/matter_tools.py`, printed in full in Appendix A.

Chapter 2

# Check the Matter Records

**Row unit.** One recorded event, retained as accepted or held.

**Reference entry point.** `programs/matter_check.py`

```
python3 programs/matter_check.py --input data/matter_events_problem.csv
--matters data/matters_demo.csv --out my_ch02
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch02/report.html`

**Outputs.** `report.html`, `accepted.csv`, `review.csv`, `run.json`.

## Complete Small Inputs and Their Headers

### matter_events_problem.csv

Exact columns, in order: `event_id`, `client_id`, `matter_id`, `event_date`, `event_type`, `request_id`, `source_id`, `summary`.

`event_id`
Unique recorded-event identifier; when a value appears twice, both rows are held.

`client_id`
Designated synthetic client identifier; it must agree with the client the register names for the matter.

`matter_id`
Supplied matter key, which must exist in the matter register.

`event_date`
Supplied calendar date of the event in year-month-day form; a blank or impossible date holds the row.

`event_type`
One of `request`, `response`, or `note`; any other value holds the row rather than being corrected.

`request_id`
Request label that, with matter_id, links a response to its request; required on requests and responses and absent on notes.

`source_id`
Supplied source reference, required to be nonblank; it is a traceable label, not authentication.

`summary`
Short supplied description of the event, required to be nonblank and otherwise not interpreted.

data/matter_events_problem.csv

```
event_id,client_id,matter_id,event_date,event_type,request_id,source_id,summary
E01,C01,M01,2026-09-01,request,R01,SRC01,Draft document requested
E02,C01,M01,2026-08-31,response,R01,SRC02,A response is recorded
E03,C01,M01,2026-09-05,note,,SRC03,Administrative review note
E04,C01,M02,2026-09-03,request,R02,SRC04,Draft document requested
E05,C01,M02,,response,R02,SRC05,A response is recorded
E06,C01,M02,2026-09-07,note,,SRC06,Administrative review note
E07,C02,M03,2026-09-05,request,R03,SRC07,Draft document requested
E08,C02,M03,2026-02-30,response,R03,SRC08,A response is recorded
E09,C02,M03,2026-09-09,note,,SRC09,Administrative review note
E10,C02,M04,2026-09-07,request,R04,SRC10,Draft document requested
E10,C02,M04,2026-09-10,response,R04,SRC11,A response is recorded
E12,C02,M04,2026-09-11,note,,SRC12,Administrative review note
E13,C03,M05,2026-09-09,request,R05,SRC13,Draft document requested
E14,C03,M99,2026-09-12,response,R05,SRC14,A response is recorded
E15,C03,M05,2026-09-13,note,,SRC15,Administrative review note
E16,C03,M06,2026-09-11,request,R06,SRC16,Draft document requested
E17,C99,M06,2026-09-14,response,R06,SRC17,A response is recorded
E18,C03,M06,2026-09-15,note,,SRC18,Administrative review note
```

### matters_demo.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/matters_demo.csv

```
matter_id,client_id,description
M01,C01,Agreement review
M02,C01,Employment documents
M03,C02,Estate administration
M04,C02,Commercial lease
M05,C03,Tenancy records
M06,C03,Service agreement
```

## One Controlled Experiment

Restore E05's date using its literal value in the supplied clean file. The controlled-run helper reads that value from the clean fixture rather than asking for a plausible replacement. The new output has twelve accepted and six held rows. Inspect the identity leaving the held table: it must be E05, not just any row.

The two duplicated E10 positions remain held, as do the other unresolved cases. This is the appropriate size of a repair experiment. You know the changed field, its source, the one expected reclassification, and the classifications that should remain unchanged. A general "clean the data" request would make that comparison much harder.

```
python3 tools/run_experiments.py 2 --out my_experiment_02
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/matter_check.py

```
"""Entry point for the shared, checked synthetic matter application."""

from matter_tools import main

if __name__ == "__main__":
    main()
```

The checking rules this entry point runs are implemented in `programs/matter_tools.py`, printed in full in Appendix A.

Chapter 3

# From Eighteen Records to 12,000 Events

**Row unit.** One event in the same schema as the small input.

**Reference entry point.** `programs/matter_run.py`

```
python3 programs/matter_run.py --input data/matter_events_large.csv
--matters data/matters_large.csv --out my_ch03 --matter M0001
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch03/report.html`

**Outputs.** `report.html`, `accepted.csv`, `review.csv`, `run.json`.

## Complete Small Inputs and Their Headers

### matter_events_large.csv

Exact columns, in order: `event_id`, `client_id`, `matter_id`, `event_date`, `event_type`, `request_id`, `source_id`, `summary`.

`event_id`
Unique recorded-event identifier; when a value appears twice, both rows are held.

`client_id`
Designated synthetic client identifier; it must agree with the client the register names for the matter.

`matter_id`
Supplied matter key, which must exist in the matter register.

`event_date`
Supplied calendar date of the event in year-month-day form; a blank or impossible date holds the row.

`event_type`
One of `request`, `response`, or `note`; any other value holds the row rather than being corrected.

`request_id`
Request label that, with matter_id, links a response to its request; required on requests and responses and absent on notes.

`source_id`
Supplied source reference, required to be nonblank; it is a traceable label, not authentication.

`summary`
Short supplied description of the event, required to be nonblank and otherwise not interpreted.

This actual supplied input has 12,000 data rows. Do not reconstruct it from this PDF. Its exact header and the first three rows are shown below; the file and release manifest identify the complete bytes.

data/matter_events_large.csv

```
event_id,client_id,matter_id,event_date,event_type,request_id,source_id,summary
E00001,C0001,M0001,2026-01-01,note,,SRC00001,Synthetic note for matter 1
E00002,C0001,M0001,2026-01-02,request,R0001,SRC00002,Synthetic request for matter 1
E00003,C0001,M0001,2026-01-04,response,R0001,SRC00003,Synthetic response for matter 1
```

### matters_large.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

This actual supplied input has 3,000 data rows. Do not reconstruct it from this PDF. Its exact header and the first three rows are shown below; the file and release manifest identify the complete bytes.

data/matters_large.csv

```
matter_id,client_id,description
M0001,C0001,Synthetic matter 1
M0002,C0001,Synthetic matter 2
M0003,C0001,Synthetic matter 3
```

## One Controlled Experiment

The supplied large-file experiment changes the client on E00007, a response outside M0001, to C9999. The selected M0001 report still contains four events. The whole-file record changes to 11,999 accepted and one held event.

This is a test of the order of operations. An implementation that selects M0001 first and checks only those four records would miss the changed event. Inspect both the selected output and the whole-file checking counts. A selected report and an internal run record serve different audiences; the existence of the latter does not make it an appropriate external attachment.

```
python3 tools/run_experiments.py 3 --out my_experiment_03
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/matter_run.py

```
"""Entry point for the shared, checked synthetic matter application."""

from matter_tools import main

if __name__ == "__main__":
    main()
```

The checking rules this entry point runs are implemented in `programs/matter_tools.py`, printed in full in Appendix A.

Chapter 4

# Check Consultation Completion

**Row unit.** One consultation appointment.
**Reference entry point.** `programs/consultation_report.py`

```
python3 programs/consultation_report.py --out my_ch04
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch04/report.html`
**Outputs.** `ledger.csv`, `report.html`, `run.json`, `summary.json`.

## Complete Small Inputs and Their Headers

### appointments.csv

Exact columns, in order: `appointment_id`, `matter_id`, `scheduled_date`, `status`, `completion_date`, `name`, `note`.

`appointment_id`
Unique nonblank appointment identifier; a blank or repeated value suppresses the headline rate.

`matter_id`
Supplied matter key; a matter absent from the register suppresses the headline rate.

`scheduled_date`
Supplied ISO date on which the appointment was scheduled; decides eligibility relative to the reporting date, and an invalid date suppresses the headline.

`status`
One of `completed`, `cancelled`, `missed`, or `scheduled`; cancelled rows leave the denominator, and an unknown status suppresses the headline.

`completion_date`
Supplied date of a documented completion; a completed row enters the numerator

only when this date lies between the scheduled date and the reporting date. A non-completed row with a nonblank completion date keeps a warning; the date does not turn it into a completion.

`name`
Invented name, not used by this chapter's rule; it matters in Chapter 7.

`note`
Invented free-text note, not used by this chapter's rule; Chapter 7 shows why it must not leave the source.

data/ch04/appointments.csv

```
appointment_id,matter_id,scheduled_date,status,completion_date,name,note
A01,M01,2026-09-01,completed,2026-09-01,Example recipient 1,Synthetic internal note
A02,M02,2026-09-02,completed,2026-09-02,Example recipient 2,Synthetic internal note
A03,M03,2026-09-03,cancelled,,Example recipient 3,Synthetic internal note
A04,M01,2026-09-04,missed,,Example recipient 4,Synthetic internal note
A05,M02,2026-09-05,completed,,Example recipient 5,PRIVATE-NOTE-X83
A06,M03,2026-09-06,completed,2026-09-06,Example recipient 6,Synthetic internal note
A07,M01,2026-09-07,scheduled,,Example recipient 7,Synthetic internal note
A08,M02,2026-09-08,cancelled,,Example recipient 8,Synthetic internal note
```

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch04/matters.csv

```
matter_id,client_id,description
M01,C01,Agreement documents
M02,C01,Employment records
M03,C02,Correspondence review
```

## One Controlled Experiment

Supply the supported teaching date September 5 for A05 and run to a new destination. Its numerator flag changes from zero to one; its denominator flag stays one. The total therefore becomes four of six, approximately 66.7%. The input still has eight rows, including two cancellations.

Compare the ledger, not just the headline. A05 is the only appointment whose documented-completion contribution changed. A07 is still unresolved. This comparison teaches the reader to ask for the smallest explanation consistent with the observed difference, rather than treating any improved percentage as evidence that the whole file has been repaired.

```
python3 tools/run_experiments.py 4 --out my_experiment_04
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/consultation_report.py

```
"""Chapter 4: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(4)
```

The function `consultation` computes documented consultation completion as of a reporting date: the denominator holds every appointment scheduled on or before that date except cancelled ones, and the numerator holds the denominator rows marked completed whose completion date lies between the scheduled date and the reporting date.

programs/legal_apps.py, lines 39–141

```
# Chapter 4: documented completions, not just completed status labels.
def consultation(folder: Path, as_of: str = "2026-09-24", **_) -> Result:
    """Documented consultation completion as of a reporting date.

    Rule (Section 4.1 of the book). The denominator holds every appointment
    scheduled on or before the reporting date except cancelled ones. The
    numerator holds the denominator rows marked completed that also have a
    completion date between the scheduled date and the reporting date.
    An appointment scheduled after the reporting date is outside the report:
    it is labelled not_yet_scheduled and takes part in no count. A structural
    defect (duplicate ID, unknown status or matter, invalid scheduled date)
    suppresses the headline percentage but keeps every row in the ledger.
    """
    cutoff = required_day(as_of)
```

```
    known = registry(folder)
    rows = load(folder, "appointments.csv", APPOINTMENT_FIELDS)
    issues = identity_issues(rows, "appointment_id")
    ledger = []
    structural = False
    for i, r in enumerate(rows):
        e = issues[i]
        scheduled = day(r["scheduled_date"])
        completed = day(r["completion_date"])
        # Structural checks first: these suppress the headline.
        if r["status"] not in {"completed", "cancelled", "missed", "scheduled"}:
            e.append("unknown_status")
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if scheduled is None:
            e.append("invalid_scheduled_date")
        structural |= bool(e)
        # Then the population: is the row in the report at all, and in the denominator?
        in_report = not e and scheduled <= cutoff
        eligible = in_report and r["status"] != "cancelled"
        documented = (
            completed is not None and scheduled is not None and scheduled <= completed <=
cutoff
        )
        numerator = eligible and r["status"] == "completed" and documented
        # Review labels apply only to rows the report covers.
        if in_report and r["status"] == "completed" and not documented:
            e.append("completion_date_needs_review")
        if in_report and r["status"] != "completed" and r["completion_date"]:
            e.append("date_on_noncompleted_status")
        if e:
            explanation = ";".join(e)
        elif not in_report:
            explanation = "not_yet_scheduled"
        elif r["status"] == "cancelled":
            explanation = "cancelled"
        elif numerator:
            explanation = "documented_completion"
        elif r["status"] == "scheduled":
            explanation = "unresolved"
        else:
            explanation = "denominator_only"
        ledger.append(
            {
                "source_row": i + 2,
                "appointment_id": r["appointment_id"],
                "matter_id": r["matter_id"],
                "status": r["status"],
                "scheduled_date": r["scheduled_date"],
                "completion_date": r["completion_date"],
                "eligible": int(eligible),
                "numerator": int(numerator),
                "explanation": explanation,
            }
        )
    den = sum(r["eligible"] for r in ledger)
    num = sum(r["numerator"] for r in ledger)
    covered = [r for r in ledger if r["explanation"] != "not_yet_scheduled"]
    result = Result(
        "Documented consultation completion",
```

```
        {
            "input_rows": len(rows),
            "reporting_date": as_of,
            "numerator": num,
            "denominator": den,
            "percentage": None if structural or den == 0 else num / den * 100,
            "headline_available": not structural and den > 0,
            "cancelled": sum(r["status"] == "cancelled" for r in covered),
            "unresolved": sum(
                r["explanation"] == "unresolved" or "review" in r["explanation"]
                for r in covered
            ),
        },
    )
    result.add(
        "ledger.csv",
        ledger,
        [
            "source_row",
            "appointment_id",
            "matter_id",
            "status",
            "scheduled_date",
            "completion_date",
            "eligible",
            "numerator",
            "explanation",
        ],
    )
    return result
```

Chapter 5

# Catalogue Matter Documents

**Row unit.** One file record, with supplied document and version labels.
**Reference entry point.** `programs/document_catalogue.py`

```
python3 programs/document_catalogue.py --out my_ch05
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch05/report.html`
**Outputs.** `catalogue.csv`, `membership.csv`, `report.html`, `review.csv`, `run.json`, `summary.json`.

## Complete Small Inputs and Their Headers

### documents.csv

Exact columns, in order: `file_id`, `matter_id`, `document_id`, `version`, `type`, `source_id`, `title`.

`file_id`
Unique identifier of one file record; a blank or repeated value holds every row concerned.

`matter_id`
Supplied matter key, checked against `matters.csv`; document labels are local to a matter.

`document_id`
Supplied document label, required to be nonblank; with matter_id it identifies a document.

`version`
Supplied version label, which must be a whole number of at least 1 written in digits; with matter and document it identifies a labelled version.

`type`
Supplied type label, required to be nonblank; it does not define document or version identity.

`source_id`
Supplied source reference, required to be nonblank; a nonblank value is not authentication.

`title`
Supplied title, carried through as metadata and not checked.

data/ch05/documents.csv

```
file_id,matter_id,document_id,version,type,source_id,title
F01,M01,D01,1,draft,S01,Agreement
F02,M01,D01,2,draft,S02,Agreement
F03,M01,D01,2,draft copy,S03,Agreement
F04,M01,D02,1,recorded as executed,S04,Letter
F05,M02,D03,1,correspondence,S05,Employment request
F06,M02,D04,1,draft,S06,Schedule
F07,M02,D04,2,draft,S07,Schedule
F08,M03,D05,1,correspondence,S08,Response
```

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch05/matters.csv

```
matter_id,client_id,description
M01,C01,Agreement documents
M02,C01,Employment records
M03,C02,Correspondence review
```

## One Controlled Experiment

Add F09 as a new file record for M01, document D01, version 2, using source S09. The complete register now has nine file records. It still has five document groups and seven labelled-version groups. M01 contributes five files, two documents, and three versions.

Inspect the membership table for D01 version 2. It now contains F02, F03, and F09. The new file was not ignored: it changed one count but not the other two. This is why a summary should name its counting unit. A colleague can answer a question about copies without confusing it with a question about distinct labelled versions.

```
python3 tools/run_experiments.py 5 --out my_experiment_05
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/document_catalogue.py

```
"""Chapter 5: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(5)
```

The shared loader `document_rows` validates the document register row by row and returns the accepted and held rows; `catalogue` then counts file records, documents, and labelled versions. Chapter 6 uses the same loader.

programs/legal_apps.py, lines 144–208

```
# Chapters 5 and 6: common register checks, different deliverables.
def document_rows(folder: Path) -> tuple[list[dict], list[dict]]:
    """Shared loader for Chapters 5 and 6: validate the document register row by row.

    Returns (accepted, held). A row is held, with every reason, when its file
    ID is blank or duplicated, its document ID, type or source reference is
    blank, its matter is not in matters.csv, or its version is not a whole
    number of at least 1 written in digits. Nothing is corrected.
    """
    rows = load(folder, "documents.csv", DOCUMENT_FIELDS)
    known = registry(folder)
    issues = identity_issues(rows, "file_id")
    accepted = []
    held = []
    for i, r in enumerate(rows):
        e = issues[i]
        require_cells(r, ["document_id", "source_id", "type"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
```

```
        v = natural(r["version"])
        if v is None or v < 1:
            e.append("invalid_version")
        enriched = dict(r, source_row=i + 2, issues=";".join(e))
        (held if e else accepted).append(enriched)
    return accepted, held


def catalogue(folder: Path, **_) -> Result:
    """Chapter 5: count file records, documents, and labelled versions.

    A document is the pair (matter, document ID); a labelled version is the
    triple (matter, document ID, version). Two rows with the same triple are
    two copies of one version and both stay in the membership table.
    """
    accepted, held = document_rows(folder)
    groups = defaultdict(list)
    for r in accepted:
        groups[r["matter_id"]].append(r)
    summaries = []
    for m, rr in sorted(groups.items()):
        summaries.append(
            {
                "matter_id": m,
                "file_records": len(rr),
                "documents": len({r["document_id"] for r in rr}),
                "versions": len({(r["document_id"], r["version"]) for r in rr}),
            }
        )
    result = Result(
        "Matter-document catalogue",
        {
            "file_records": len(accepted),
            "held": len(held),
            "documents": len({(r["matter_id"], r["document_id"]) for r in accepted}),
            "versions": len(
                {(r["matter_id"], r["document_id"], r["version"]) for r in accepted}
            ),
        },
    )
    result.add(
        "catalogue.csv", summaries, ["matter_id", "file_records", "documents", "versions"]
    )
    result.add("membership.csv", accepted, DOCUMENT_FIELDS + ["source_row", "issues"])
    result.add("review.csv", held, DOCUMENT_FIELDS + ["source_row", "issues"])
    return result
```

Chapter 6

# Check a Document Register

**Row unit.** One file-register row, including rows needing review.
**Reference entry point.** `programs/document_register_check.py`

```
python3 programs/document_register_check.py --out my_ch06
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch06/report.html`
**Outputs.** `accepted.csv`, `consistency.csv`, `report.html`, `review.csv`, `run.json`, `summary.json`.

## Complete Small Inputs and Their Headers

### documents.csv

Exact columns, in order: `file_id`, `matter_id`, `document_id`, `version`, `type`, `source_id`, `title`.

`file_id`
Unique identifier of one file record; a blank or repeated value holds every row concerned.

`matter_id`
Supplied matter key, checked against `matters.csv`; document labels are local to a matter.

`document_id`
Supplied document label, required to be nonblank; with matter_id it identifies a document.

`version`
Supplied version label, which must be a whole number of at least 1 written in digits; with matter and document it identifies a labelled version.

`type`
Supplied type label, required to be nonblank; a labelled-version group that mixes a `draft` type with an `executed` type is listed in `consistency.csv` without being held.

`source_id`
Supplied source reference, required to be nonblank; a nonblank value is not authentication.

`title`
Supplied title, carried through as metadata and not checked.

data/ch06/documents.csv

```
file_id,matter_id,document_id,version,type,source_id,title
F01,M01,D01,1,draft,S01,Agreement
F02,M01,D01,2,draft,S02,Agreement
F03,M01,D01,2,draft copy,,Agreement
F04,M99,D02,1,recorded as executed,S04,Letter
F05,M02,D03,1,correspondence,S05,Employment request
F06,M02,D04,two,draft,S06,Schedule
F07,M02,D04,2,draft,S07,Schedule
F05,M03,D05,1,correspondence,S08,Response
```

### documents_clean.csv

Exact columns, in order: `file_id`, `matter_id`, `document_id`, `version`, `type`, `source_id`, `title`.

`file_id`
Unique identifier of one file record; a blank or repeated value holds every row concerned.

`matter_id`
Supplied matter key, checked against `matters.csv`; document labels are local to a matter.

`document_id`
Supplied document label, required to be nonblank; with matter_id it identifies a document.

`version`
Supplied version label, which must be a whole number of at least 1 written in digits; with matter and document it identifies a labelled version.

`type`
Supplied type label, required to be nonblank; a labelled-version group that mixes a `draft` type with an `executed` type is listed in `consistency.csv` without being held.

`source_id`
Supplied source reference, required to be nonblank; a nonblank value is not authentication.

`title`
Supplied title, carried through as metadata and not checked.

data/ch06/documents_clean.csv

```
file_id,matter_id,document_id,version,type,source_id,title
F01,M01,D01,1,draft,S01,Agreement
F02,M01,D01,2,draft,S02,Agreement
F03,M01,D01,2,draft copy,S03,Agreement
F04,M01,D02,1,recorded as executed,S04,Letter
F05,M02,D03,1,correspondence,S05,Employment request
F06,M02,D04,1,draft,S06,Schedule
F07,M02,D04,2,draft,S07,Schedule
F08,M03,D05,1,correspondence,S08,Response
```

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch06/matters.csv

```
matter_id,client_id,description
M01,C01,Agreement documents
M02,C01,Employment records
M03,C02,Correspondence review
```

## One Controlled Experiment

Repair only the missing source on F03. The accepted count changes from three to four and the held count from five to four. The report must still show the unknown matter, invalid version, and both duplicate-ID positions.

The experiment folder records the input hashes before and after the change and saves the modified input beside the rerun. For an office correction log, add the field name, old and new values, source reference, and authorized person as well. The hash identifies which file changed; the field-level account explains why. Neither replaces the other.

```
python3 tools/run_experiments.py 6 --out my_experiment_06
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/document_register_check.py

```
"""Chapter 6: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(6)
```

The function `document_check` returns the accepted and held lists for the register, plus a consistency list of the labelled-version groups that mix a draft type with an executed type. It uses `document_rows`, printed in Chapter 5.

programs/legal_apps.py, lines 211–250

```
def document_check(folder: Path, **_) -> Result:
    """Chapter 6: accepted and held lists for the register, plus a consistency list.

    The consistency check groups every row, held or not, by (matter, document,
    version) and lists groups that mix a draft type with an executed type. It
    reports; it never holds or repairs a row.
    """
    accepted, held = document_rows(folder)
    allrows = sorted(accepted + held, key=lambda r: r["source_row"])
    consistency = []
```

```
    grouped = defaultdict(list)
    for r in allrows:
        grouped[(r["matter_id"], r["document_id"], r["version"])].append(r)
    for key, rr in grouped.items():
        labels = {r["type"] for r in rr}
        if any("draft" in s for s in labels) and any("executed" in s for s in labels):
            for r in rr:
                consistency.append(
                    {
                        "source_row": r["source_row"],
                        "file_id": r["file_id"],
                        "issue": "conflicting_supplied_types",
                        "original_type": r["type"],
                    }
                )
    result = Result(
        "Document-register review",
        {
            "input_rows": len(allrows),
            "accepted": len(accepted),
            "held": len(held),
            "consistency_rows": len(consistency),
        },
    )
    result.add("accepted.csv", accepted, DOCUMENT_FIELDS + ["source_row", "issues"])
    result.add("review.csv", held, DOCUMENT_FIELDS + ["source_row", "issues"])
    result.add(
        "consistency.csv", consistency, ["source_row", "file_id", "issue", "original_type"]
    )
    return result
```

Chapter 7

# Prepare a Limited Management Extract

**Row unit.** One appointment in the source; one week–status count in the export.
**Reference entry point.** `programs/management_extract.py`

```
python3 programs/management_extract.py --out my_ch07
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch07/report.html`
**Outputs.** `extract.csv`, `report.html`, `run.json`, `summary.json`.

## Complete Small Inputs and Their Headers

### appointments.csv

Exact columns, in order: `appointment_id`, `matter_id`, `scheduled_date`, `status`, `completion_date`, `name`, `note`.

`appointment_id`
Unique nonblank appointment identifier; a blank or repeated value stops the run.

`matter_id`
Supplied matter key; it is never copied into the extract.

`scheduled_date`
Supplied ISO date that places the appointment in the Monday-beginning week containing it; an invalid date stops the run.

`status`
One of `completed`, `cancelled`, `missed`, or `scheduled`, counted exactly as supplied; an unknown status stops the run.

`completion_date`
Supplied completion date; it does not affect the counts and is not exported.

`name`
Invented name; it must not appear in any delivered file.

`note`
Invented note; A05 carries the canary `PRIVATE-NOTE-X83`, which must not appear in any delivered file.

data/ch07/appointments.csv

```
appointment_id,matter_id,scheduled_date,status,completion_date,name,note
A01,M01,2026-09-01,completed,2026-09-01,Example recipient 1,Synthetic internal note
A02,M02,2026-09-02,completed,2026-09-02,Example recipient 2,Synthetic internal note
A03,M03,2026-09-03,cancelled,,Example recipient 3,Synthetic internal note
A04,M01,2026-09-04,missed,,Example recipient 4,Synthetic internal note
A05,M02,2026-09-05,completed,,Example recipient 5,PRIVATE-NOTE-X83
A06,M03,2026-09-06,completed,2026-09-06,Example recipient 6,Synthetic internal note
A07,M01,2026-09-07,scheduled,,Example recipient 7,Synthetic internal note
A08,M02,2026-09-08,cancelled,,Example recipient 8,Synthetic internal note
```

## One Controlled Experiment

Change A05's supplied status from completed to missed. Do not change its missing completion date or its note. The first week's completed count falls from four to three and its missed count rises from one to two. The exported counts still sum to eight appointments; the output still has eight week–status rows.

Search the actual exported CSV, HTML, summary, and run record for the distinctive note. Then inspect the exported CSV header. A clean-looking page with a copied note hidden elsewhere in the delivered package would fail the content requirement even though its displayed counts were correct. The reference constructs the three-column export directly.

```
python3 tools/run_experiments.py 7 --out my_experiment_07
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/management_extract.py

```
"""Chapter 7: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(7)
```

The function `management_extract` builds weekly status counts from three permitted fields only; an unknown status, invalid date, or duplicate identifier stops the run with a message that quotes no source value.

programs/legal_apps.py, lines 253–288

```
# Chapter 7: allowlist output fields; do not copy private rows to diagnostics.
def management_extract(folder: Path, **_) -> Result:
    """Chapter 7: weekly status counts built from three permitted fields only.

    Weeks begin on Monday. Every week present in the input gets one row for
    each of the four statuses, zeros included. An unknown status, invalid date
    or duplicate ID stops the run with a message that quotes no source value.
    """
    rows = load(folder, "appointments.csv", APPOINTMENT_FIELDS)
    categories = ["completed", "cancelled", "missed", "scheduled"]
    counts = Counter()
    weeks = set()
    if any(identity_issues(rows, "appointment_id")):
        raise ValueError("Appointment identity error; inspect the source locally")
    for r in rows:
        d = day(r["scheduled_date"])
        if d is None or r["status"] not in categories:
            raise ValueError("Invalid reporting date or status; inspect the source locally")
        week = str(d - timedelta(days=d.weekday()))
        weeks.add(week)
        counts[(week, r["status"])] += 1
    output = [
        {"week_beginning": w, "status": s, "count": counts[(w, s)]}
        for w in sorted(weeks)
        for s in categories
    ]
    result = Result(
        "Limited weekly status extract",
        {
            "input_rows": len(rows),
            "output_rows": len(output),
            "total_count": sum(r["count"] for r in output),
        },
    )
    result.add("extract.csv", output, ["week_beginning", "status", "count"])
    return result
```

Chapter 8

# Match Requests and Responses

**Row unit.** One request in one input and one response in the other.

**Reference entry point.** `programs/request_response_match.py`

```
python3 programs/request_response_match.py --out my_ch08
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch08/report.html`

**Outputs.** `candidates.csv`, `matches.csv`, `report.html`, `run.json`, `summary.json`, `unmatched.csv`.

Use `--window 21` to change the inclusive window, `--as-of 2026-09-25` to change the historical date. (The shared entry point accepts `--matter` for every chapter, but the matching application does not use it; selection by matter belongs to the Chapter 23 worklist.) Future requests and responses remain explicitly distinguished from observed records.

## Complete Small Inputs and Their Headers

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch08/matters.csv

```
matter_id,client_id,description
M01,C01,Agreement documents
M02,C01,Employment records
M03,C02,Correspondence review
```

### requests.csv

Exact columns, in order: `request_id`, `matter_id`, `owner`, `request_date`, `source_id`.

`request_id`
Request label; with matter_id it forms the key a response must match, and a repeated key is flagged on every occurrence.

`matter_id`
Supplied matter key, checked against the register; a response must agree on it as well as on the request label.

`owner`
Supplied assignment label, carried through and never inferred by the application.

`request_date`
Supplied ISO date of the request, from which elapsed days are counted.

`source_id`
Supplied source reference, required to be nonblank; not authentication.

data/ch08/requests.csv

```
request_id,matter_id,owner,request_date,source_id
R01,M01,A,2026-09-01,REQ01
R02,M01,B,2026-09-03,REQ02
R03,M02,A,2026-09-02,REQ03
R04,M03,B,2026-09-10,REQ04
```

### responses.csv

Exact columns, in order: `response_id`, `matter_id`, `request_id`, `response_date`, `source_id`.

`response_id`
Unique identifier of one response; a repeated value is flagged on every occurrence.

`matter_id`
Matter the response names; it must agree with the request's matter for the response to link.

`request_id`
Request label the response names; with matter_id it must match a request, or the response is unmatched.

`response_date`
Supplied ISO date of the response; it decides the elapsed days, the reporting-date cutoff, and the window test.

`source_id`
Supplied source reference; a nonblank value is not authentication.

data/ch08/responses.csv

```
response_id,matter_id,request_id,response_date,source_id
S01,M01,R01,2026-09-01,RES01
S02,M01,R01,2026-09-15,RES02
S03,M01,R02,2026-09-18,RES03
S04,M02,R03,2026-09-08,RES04
S05,M01,R03,2026-09-08,RES05
S06,M03,R04,2026-09-09,RES06
S07,M03,R99,2026-09-12,RES07
```

## Input Validity and Evidence Fields

The matching rule includes validation, not just selection. Apply the following checks before a response can count as evidence.

1. The matter register must have the exact header shown above, unique nonblank matter identifiers, and nonblank client identifiers; an unusable register stops the run.

2. Each request needs a nonblank request identifier, owner, and source reference; a matter present in that register; and a real date in `YYYY-MM-DD` form. The pair of matter and request identifiers must be unique. Every occurrence of a repeated pair needs review; file order does not choose a winner.

3. Each response needs a unique nonblank response identifier, a nonblank request identifier and source reference, and a real ISO date. A repeated response identifier invalidates every occurrence. The response's matter and request identifiers must name exactly one request that passes its own checks. The response date must not precede that request's date.

4. A candidate's `valid` flag means that all those response and linked-request checks pass. Its `observed` flag additionally requires both dates to be on or before the reporting date. Its `eligible` flag additionally requires elapsed days from zero through the chosen window, inclusive. A future, invalid, or unmatched response remains in the candidate ledger with its explanation; it is not silently discarded.

A defective request has status `needs_review` and unavailable evidence fields. A valid future request has unavailable evidence fields and is `not_yet_requested`, unless a defective linked response requires review. An otherwise valid request also needs review if one of its linked responses is invalid. In that last case, usable evidence from another response is retained: a review status need not erase an eligible response.

For a valid nonfuture request, `any_response` asks whether an observed valid response exists; `in_window` asks whether an eligible one exists. Among eligible responses the earliest date is selected only if unique; a tie gives `ambiguous` and no selected response. Without an observed response and without a review issue, the status is `window_open` through the inclusive endpoint and `window_passed` afterwards. An observed valid response gives `response_recorded`, even if it is outside the chosen window.

For a small validation probe, blank S01's source reference in a copy and rerun. S01 must be invalid, R01 must need review, and S02 must remain its unique eligible response on day fourteen. This checks the complete contract rather than only the baseline headline.

## One Controlled Experiment

Add S08 with the same matter, request, and date as S04, but with its own response identity and source reference. The R03 result now records that an in-window response exists, while its earliest-selection field is ambiguous. S04 and S08 both carry the tied-earliest label in the candidate ledger.

The unique-match count falls from two to one; the number of requests with at least one eligible response remains two. These are not contradictory results. They count different properties. Keep both questions visible before using the matcher inside a worklist or a percentage. A tie does not justify suppressing the evidence that responses were recorded.

```
python3 tools/run_experiments.py 8 --out my_experiment_08
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/request_response_match.py

```
"""Chapter 8: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(8)
```

The function `match_records` links responses to requests by matter and request identifier and selects the earliest in-window response; `matching` produces one result row per request and a candidate ledger with one row per response. The two field lists between them name the output columns. Chapter 23 reuses `match_records` unchanged.

`programs/legal_apps.py`, lines 291–470

```
# Chapter 8 and Chapter 23 share this relationship and historical-cutoff logic.
def match_records(
    requests: list[dict], responses: list[dict], known: dict, cutoff: date, window: int = 14
) -> tuple[list[dict], list[dict]]:
    """Chapters 8 and 23: link responses to requests and select the earliest in-window one.

    A response belongs to a request only when matter_id and request_id both
    agree. Elapsed days are calendar days; a negative value is invalid and
    sends the request to review. A response after the reporting date is kept
    but not observed. An observed response is eligible when 0 <= elapsed <=
    window, both ends included. Among eligible responses the earliest date is
    selected; a tie makes the request ambiguous and selects nothing.
    """
    if not isinstance(window, int) or window < 0:
        raise ValueError("Window must be a nonnegative integer")
    keys = Counter((r["matter_id"], r["request_id"]) for r in requests)
    request_issues = []
    request_index = {}
    for i, r in enumerate(requests):
        e = []
        require_cells(r, ["request_id", "owner", "source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if day(r["request_date"]) is None:
            e.append("invalid_request_date")
        key = (r["matter_id"], r["request_id"])
        if keys[key] > 1:
            e.append("duplicate_request_key")
        request_issues.append(e)
        if keys[key] == 1:
            request_index[key] = i
    issues = identity_issues(responses, "response_id")
    candidates = []
    by_request = defaultdict(list)
    for i, r in enumerate(responses):
        e = issues[i]
        require_cells(r, ["source_id", "request_id"], e)
        d = day(r["response_date"])
        if d is None:
            e.append("invalid_response_date")
        key = (r["matter_id"], r["request_id"])
        j = request_index.get(key)
        if keys[key] > 1:
            e.append("ambiguous_request_key")
        elif j is None:
            e.append("unknown_request_key")
        elif request_issues[j]:
            e.append("request_needs_review")
        elapsed = None
        if j is not None and not request_issues[j] and d is not None:
```

```
            elapsed = (d - required_day(requests[j]["request_date"])).days
            if elapsed < 0:
                e.append("response_before_request")
        linked_valid = not e
        observed = (
            linked_valid and d <= cutoff and required_day(requests[j]["request_date"]) <=
cutoff
        )
        eligible = bool(observed and elapsed is not None and 0 <= elapsed <= window)
        reason = (
            ";".join(e)
            if e
            else "after_as_of"
            if not observed
            else "in_window"
            if eligible
            else "outside_window"
        )
        row = dict(
            r,
            source_row=i + 2,
            elapsed_days="" if elapsed is None else elapsed,
            valid=int(linked_valid),
            observed=int(observed),
            eligible=int(eligible),
            selection="not_selected",
            explanation=reason,
        )
        candidates.append(row)
        if j is not None:
            by_request[j].append(row)
    results = []
    for i, r in enumerate(requests):
        e = request_issues[i]
        d = day(r["request_date"])
        rr = by_request[i]
        observed = [x for x in rr if x["observed"]]
        eligible = [x for x in rr if x["eligible"]]
        linked_problems = any(not x["valid"] for x in rr)
        future = not e and d > cutoff
        evidence_unavailable = bool(e) or future
        any_response = "unavailable" if evidence_unavailable else "yes" if observed else "no"
        in_window = "unavailable" if evidence_unavailable else "yes" if eligible else "no"
        selected = ""
        earliest = "unavailable" if evidence_unavailable else "none"
        if eligible:
            earliest_date = min(x["response_date"] for x in eligible)
            tied = [x for x in eligible if x["response_date"] == earliest_date]
            earliest = "unique" if len(tied) == 1 else "ambiguous"
            for x in tied:
                x["selection"] = "selected" if len(tied) == 1 else "tied_earliest"
            if len(tied) == 1:
                selected = tied[0]["response_id"]
        if e or linked_problems:
            status = "needs_review"
        elif future:
            status = "not_yet_requested"
        elif observed:
            status = "response_recorded"
        elif (cutoff - d).days <= window:
```

```
            status = "window_open"
        else:
            status = "window_passed"
        results.append(
            dict(
                r,
                source_row=i + 2,
                status=status,
                any_response=any_response,
                in_window=in_window,
                earliest=earliest,
                selected_response=selected,
                issues=";".join(
                    e + (["linked_response_needs_review"] if linked_problems else [])
                ),
            )
        )
    return results, candidates


MATCH_FIELDS = REQUEST_FIELDS + [
    "source_row",
    "status",
    "any_response",
    "in_window",
    "earliest",
    "selected_response",
    "issues",
]
CANDIDATE_FIELDS = RESPONSE_FIELDS + [
    "source_row",
    "elapsed_days",
    "valid",
    "observed",
    "eligible",
    "selection",
    "explanation",
]


def matching(folder: Path, as_of: str = "2026-09-24", window: int = 14, **_) -> Result:
    """Chapter 8: one result row per request and a candidate ledger with one row per response.
  """
    req = load(folder, "requests.csv", REQUEST_FIELDS)
    resp = load(folder, "responses.csv", RESPONSE_FIELDS)
    matched, candidates = match_records(
        req, resp, registry(folder), required_day(as_of), window
    )
    result = Result(
        "Document request-response matching",
        {
            "requests": len(req),
            "responses": len(resp),
            "matched": sum(r["earliest"] == "unique" for r in matched),
            "ambiguous": sum(r["earliest"] == "ambiguous" for r in matched),
            "reporting_date": as_of,
            "window_days": window,
        },
    )
    result.add("matches.csv", matched, MATCH_FIELDS)
```

```
    result.add("candidates.csv", candidates, CANDIDATE_FIELDS)
    result.add(
        "unmatched.csv",
        [
            r
            for r in candidates
            if "unknown_request_key" in r["explanation"]
            or "ambiguous_request_key" in r["explanation"]
        ],
        CANDIDATE_FIELDS,
    )
    return result
```

Chapter 9

# Define the Matters in a Report

**Row unit.** One candidate matter, with optional observed response days.
**Reference entry point.** `programs/matter_population.py`

```
python3 programs/matter_population.py --out my_ch09
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch09/report.html`
**Outputs.** `population.csv`, `report.html`, `run.json`, `summary.json`.
The default as-of date is September 15, 2026.

## Complete Small Inputs and Their Headers

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch09/matters.csv

```
matter_id,client_id,description
M01,C01,Synthetic matter
M02,C02,Synthetic matter
M03,C03,Synthetic matter
M04,C04,Synthetic matter
M05,C05,Synthetic matter
M06,C06,Synthetic matter
M07,C07,Synthetic matter
M08,C08,Synthetic matter
```

### population.csv

Exact columns, in order: `matter_id`, `opening_date`, `closure_date`, `response_days`, `source_id`.

`matter_id`
Candidate matter key, which must be unique and present in `matters.csv`.

`opening_date`
Supplied ISO opening date; it must be a valid date no later than the reporting date for the matter to be included.

`closure_date`
Supplied ISO closure date, or blank for an open matter; a closure on or before the reporting date excludes the matter, and a closure before the opening sends it to review.

`response_days`
Supplied whole number of days used for the delay statistics of included matters; a blank is missing, any other non-integer is invalid, and neither is zero.

`source_id`
Supplied source reference; a blank sends the row to review.

data/ch09/population.csv

```
matter_id,opening_date,closure_date,response_days,source_id
M01,2026-09-01,,4,POP01
M02,2026-08-20,2026-09-05,,POP02
M03,2026-09-03,,,POP03
M04,2026-10-01,,,POP04
M05,2026-08-01,2026-08-31,,POP05
M06,2026-09-10,,8,POP06
M07,not-a-date,,,POP07
M08,2026-09-02,2026-09-01,,POP08
```

## One Controlled Experiment

Advance only the reporting date to October 2. M04 now joins the active-register population. M01, M03, and M06 remain included; M07 and M08 remain unresolved. The included count becomes four, but the observed response-delay count remains two because no new delay was supplied.

The mean therefore stays six while the missing-or-invalid observation count rises to two. A population expansion need not change a statistic based on the same observed values. The ledger makes that distinction visible, and the explicit reporting-date parameter lets the earlier three-matter population be regenerated.

```
python3 tools/run_experiments.py 9 --out my_experiment_09
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/matter_population.py

```
"""Chapter 9: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(9)
```

The function `population` decides which matters are active on the reporting date and reports their observed response delays.

programs/legal_apps.py, lines 473–565

```
# Chapter 9: population membership and observation eligibility are separate.
def population(folder: Path, as_of: str = "2026-09-15", **_) -> Result:
    """Chapter 9: which matters are active on the reporting date, and their observed delays.

    Included: opened on or before the reporting date and closed after it (or
    not closed). Excluded: valid dates that fail that test. Review: invalid or
    inconsistent dates, duplicate or unregistered identifiers, blank source.
    Delay statistics use included matters with a whole-number delay; a blank
    is missing and a non-integer is invalid. Neither is zero.
    """
    rows = load(
        folder,
        "population.csv",
        ["matter_id", "opening_date", "closure_date", "response_days", "source_id"],
    )
    known = registry(folder)
    cutoff = required_day(as_of)
    issues = identity_issues(rows, "matter_id")
    ledger = []
    values = []
```

```
    included = excluded = review = 0
    for i, r in enumerate(rows):
        e = issues[i]
        o = day(r["opening_date"])
        c = day(r["closure_date"])
        require_cells(r, ["source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if o is None:
            e.append("invalid_opening")
        if r["closure_date"] and (c is None or o is not None and c < o):
            e.append("invalid_closure")
        status = (
            "review"
            if e
            else "included"
            if o <= cutoff and (c is None or c > cutoff)
            else "excluded"
        )
        included += status == "included"
        excluded += status == "excluded"
        review += status == "review"
        value = natural(r["response_days"])
        obs = status == "included" and value is not None
        if obs:
            values.append(value)
        observation = (
            "observed"
            if obs
            else "not_in_population"
            if status != "included"
            else "missing"
            if not r["response_days"]
            else "invalid_value"
        )
        ledger.append(
            dict(
                r,
                source_row=i + 2,
                classification=status,
                observation=observation,
                issues=";".join(e),
            )
        )
    result = Result(
        "Active-register population",
        {
            "included": included,
            "excluded": excluded,
            "review": review,
            "observations": len(values),
            "missing_or_invalid_observations": included - len(values),
            "mean": mean(values) if values else None,
            "median": median(values) if values else None,
            "reporting_date": as_of,
        },
    )
    result.add(
        "population.csv",
        ledger,
```

```
        [
            "matter_id",
            "opening_date",
            "closure_date",
            "response_days",
            "source_id",
            "source_row",
            "classification",
            "observation",
            "issues",
        ],
    )
    return result
```

Chapter 10

# Report Monthly Turnaround

**Row unit.** One selected request–response pair with two actual dates.

**Reference entry point.** `programs/turnaround_report.py`

```
python3 programs/turnaround_report.py --out my_ch10
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch10/report.html`

**Outputs.** `monthly.csv`, `pairs_checked.csv`, `report.html`, `run.json`, `summary.json`, `turnaround.svg`.

The default as-of date for this full-year hypothetical series is December 31, 2026. Supply a different `--as-of` deliberately, not implicitly from the computer clock.

## Complete Small Inputs and Their Headers

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch10/matters.csv

```
matter_id,client_id,description
M01,C01,Agreement documents
M02,C01,Employment records
M03,C02,Correspondence review
```

### pairs.csv

Exact columns, in order: `pair_id`, `matter_id`, `request_id`, `request_date`, `response_date`, `elapsed_days`, `source_id`.

`pair_id`
Unique nonblank pair identifier; a blank or repeated value is retained for review.

`matter_id`
Supplied matter key, which must be in `matters.csv`.

`request_id`
Supplied request identifier, required; no other pair may name the same matter and request.

`request_date`
Supplied ISO request date; it must fall in 2026 and decides the month under which the pair is grouped.

`response_date`
Supplied ISO response date, not earlier than the request and on or before the cutoff for the pair to be observed.

`elapsed_days`
Supplied whole-day difference, which must equal the calendar days from the request date to the response date.

`source_id`
Supplied source reference, required to be nonblank.

data/ch10/pairs.csv

```
pair_id,matter_id,request_id,request_date,response_date,elapsed_days,source_id
PAIR01,M01,REQ01,2026-01-01,2026-01-05,4,PAIR-SOURCE-01
PAIR02,M01,REQ02,2026-01-02,2026-01-10,8,PAIR-SOURCE-02
PAIR03,M01,REQ03,2026-01-03,2026-01-15,12,PAIR-SOURCE-03
PAIR04,M01,REQ04,2026-02-01,2026-02-03,2,PAIR-SOURCE-04
PAIR05,M01,REQ05,2026-02-02,2026-02-04,2,PAIR-SOURCE-05
PAIR06,M01,REQ06,2026-02-03,2026-02-23,20,PAIR-SOURCE-06
PAIR07,M01,REQ07,2026-03-01,2026-03-06,5,PAIR-SOURCE-07
PAIR08,M01,REQ08,2026-03-02,2026-03-09,7,PAIR-SOURCE-08
PAIR09,M01,REQ09,2026-05-01,2026-05-05,4,PAIR-SOURCE-09
```

```
PAIR10,M01,REQ10,2026-06-01,2026-06-07,6,PAIR-SOURCE-10
PAIR11,M01,REQ11,2026-07-01,2026-07-09,8,PAIR-SOURCE-11
PAIR12,M01,REQ12,2026-08-01,2026-08-11,10,PAIR-SOURCE-12
PAIR13,M01,REQ13,2026-09-01,2026-09-13,12,PAIR-SOURCE-13
PAIR14,M01,REQ14,2026-10-01,2026-10-15,14,PAIR-SOURCE-14
PAIR15,M01,REQ15,2026-11-01,2026-11-17,16,PAIR-SOURCE-15
PAIR16,M01,REQ16,2026-12-01,2026-12-19,18,PAIR-SOURCE-16
```

## One Controlled Experiment

Add PAIR17: request REQ17 in M01 on April 1, response April 10, source PAIR-SOURCE-17. The elapsed field is nine. The checker recomputes the date difference before including the pair. April now has one observation, mean nine, and median nine; the total observed-pair count becomes seventeen.

The new chart acquires an April point. All other monthly values should remain unchanged. Also try changing only the elapsed field to ninety. That row must need review because its supplied value disagrees with its dates. A chart based directly on unvalidated supplied delays could look polished while concealing that inconsistency.

```
python3 tools/run_experiments.py 10 --out my_experiment_10
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/turnaround_report.py

```
"""Chapter 10: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(10)
```

The function `turnaround` reports the monthly count, mean, and median of elapsed days, grouped by request month; `turnaround_svg` derives the chart, its scale, and its missing-month notes from the saved monthly table alone.

programs/legal_apps.py, lines 568–717

```
# Chapter 10: chart is derived only from the saved monthly table.
def turnaround_svg(summary: list[dict]) -> str:
    """Derive scale, missing-month notes and accessible text from the table."""
    values = [float(r[f]) for r in summary for f in ("mean", "median") if r[f] != ""]
    maximum = max(values, default=0)
    # Leave room above the largest marker, including when every value is zero.
    raw = max(maximum * 1.1, 1) / 4
    unit = 10 ** math.floor(math.log10(raw))
```

```
  step = next(n * unit for n in (1, 2, 5, 10) if n * unit >= raw)
  top = 4 * step
  missing = [r["month"] for r in summary if r["count"] == 0]
  note = (
      "No observed pairs: " + ", ".join(missing) + ". Gaps are not zero days."
      if missing
      else "Every displayed month has at least one observed pair."
  )
  description = (
      "Monthly mean and median recorded turnaround, in days, grouped by request month. "
      + note
  )
  pieces = [
      '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 900 450" role="img" aria-label
="'
      + esc(description)
      + '"><title>'
      + esc(description)
      + '</title><rect width="900" height="450" fill="white"/>'
  ]
  for tick in range(5):
      value = tick * step
      yp = 320 - value / top * 260
      pieces.append(
          f'<line x1="70" y1="{yp}" x2="870" y2="{yp}" stroke="#ddd"/><text x="8" y="{yp +
5}" font-size="18">{value:g}</text>'
      )
  spacing = 780 / max(1, len(summary) - 1)
  for i, r in enumerate(summary):
      pieces.append(
          f'<text x="{78 + i * spacing}" y="350" font-size="18">{esc(r["month"][5:])}</text
>'
      )
  for field, marker in [("mean", "circle"), ("median", "rect")]:
      previous = None
      for i, r in enumerate(summary):
          if r[field] == "":
              previous = None
              continue
          x = 85 + i * spacing
          y = 320 - float(r[field]) / top * 260
          if previous:
              dash = ' stroke-dasharray="5 4"' if field == "median" else ""
              pieces.append(
                  f'<line x1="{previous[0]}" y1="{previous[1]}" x2="{x}" y2="{y}" stroke="
black"{dash}/>'
              )
          pieces.append(
              f'<circle cx="{x}" cy="{y}" r="5" fill="white" stroke="black"/>'
              if marker == "circle"
              else f'<rect x="{x - 4}" y="{y - 4}" width="8" height="8" fill="black"/>'
          )
          previous = (x, y)
  pieces.append(
      '<text x="70" y="25" font-size="21">Days: circle / solid = mean; square / dashed =
median</text>'
  )
  pieces.append(
      '<text x="70" y="380" font-size="18">Request month in 2026; the scale follows the
displayed values.</text>'
```

```
    )
    import textwrap

    for line_number, line in enumerate(textwrap.wrap(note, width=87)):
        pieces.append(
            f'<text x="70" y="{406 + line_number * 23}" font-size="17">{esc(line)}</text>'
        )
    pieces.append("</svg>")
    return "".join(pieces)


def turnaround(folder: Path, as_of: str = "2026-12-31", **_) -> Result:
    """Chapter 10: monthly count, mean and median of elapsed days, grouped by request month.

    A pair is observed when its matter is registered, its identifiers are
    present and unique, both dates are valid and ordered, the supplied
    elapsed_days equals the calendar difference, the request is in 2026, and
    both dates are on or before the observation cutoff. A month with no
    observation keeps count 0 and blank statistics; nothing is plotted there.
    """
    fields = [
        "pair_id",
        "matter_id",
        "request_id",
        "request_date",
        "response_date",
        "elapsed_days",
        "source_id",
    ]
    rows = load(folder, "pairs.csv", fields)
    known = registry(folder)
    cutoff = required_day(as_of)
    issues = identity_issues(rows, "pair_id")
    keys = Counter((r["matter_id"], r["request_id"]) for r in rows)
    groups = defaultdict(list)
    ledger = []
    for i, r in enumerate(rows):
        e = issues[i]
        d = day(r["request_date"])
        v = day(r["response_date"])
        n = natural(r["elapsed_days"])
        require_cells(r, ["request_id", "source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if keys[(r["matter_id"], r["request_id"])] > 1:
            e.append("duplicate_selected_pair")
        if d is None or v is None or v < d:
            e.append("invalid_dates")
        elif n is None or n != (v - d).days:
            e.append("elapsed_disagrees_with_dates")
        if not e and d.year != 2026:
            e.append("outside_teaching_year")
        status = "review" if e else "after_as_of" if v > cutoff or d > cutoff else "observed"
        if status == "observed":
            groups[r["request_date"][:7]].append(n)
        ledger.append(dict(r, classification=status, issues=";".join(e)))
    summary = []
    for month in range(1, 13):
        key = f"2026-{month:02}"
        values = groups[key]
```

```
        summary.append(
            {
                "month": key,
                "count": len(values),
                "mean": mean(values) if values else "",
                "median": median(values) if values else "",
            }
        )
    result = Result(
        "Monthly recorded turnaround",
        {
            "observations": sum(r["count"] for r in summary),
            "months": 12,
            "reporting_date": as_of,
            "grouping": "request month",
            "outstanding_requests": "not derivable from observed-pair input",
        },
    )
    result.add("monthly.csv", summary, ["month", "count", "mean", "median"])
    result.add("pairs_checked.csv", ledger, fields + ["classification", "issues"])
    result.extra["turnaround.svg"] = turnaround_svg(summary)
    result.body = (
        "<p>Observation cutoff: "
        + esc(as_of)
        + ". Missing pairs do not supply an outstanding-request denominator.</p>"
        + result.extra["turnaround.svg"]
    )
    return result
```

Chapter 11

# Monitor Weekly Requests

**Row unit.** One reporting week with counts and a final/provisional flag.
**Reference entry point.** `programs/weekly_request_monitor.py`

```
python3 programs/weekly_request_monitor.py --out my_ch11
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch11/report.html`
**Outputs.** `report.html`, `run.json`, `summary.json`, `weeks.csv`.

## Complete Small Inputs and Their Headers

### weeks.csv

Exact columns, in order: `week_id`, `week_beginning`, `eligible`, `responded`, `status`.

`week_id`
Unique nonblank week identifier; a missing or repeated value suppresses the headline.

`week_beginning`
ISO date of the Monday on which the week begins; a date that is not a valid Monday, or a repeated week, suppresses the headline.

`eligible`
Number of requests eligible for a response that week; a week with zero has no rate.

`responded`
Number of those requests with at least one qualifying recorded response; it cannot exceed `eligible`.

`status`
Exactly `final` or `provisional`; only final weeks enter the pooled sums, and any other value suppresses the headline.

data/ch11/weeks.csv

```
week_id,week_beginning,eligible,responded,status
W01,2026-06-29,2,2,final
W02,2026-07-06,8,4,final
W03,2026-07-13,10,5,final
W04,2026-07-20,10,5,final
W05,2026-07-27,10,5,final
W06,2026-08-03,10,5,final
W07,2026-08-10,10,5,final
W08,2026-08-17,10,5,final
W09,2026-08-24,10,5,final
W10,2026-08-31,10,5,final
W11,2026-09-07,10,8,provisional
W12,2026-09-14,10,2,provisional
```

## One Controlled Experiment

Change only W11's finality flag. Its eight responses and ten eligible requests now enter the pooled final totals. Those totals become fifty-four and one hundred, giving 54%. W12 remains provisional and visible.

The report has changed without any count changing inside a week. Save the status flag with the run parameters and input version. When presenting the two reports, explain that the second includes an additional final period. It is not a correction to the arithmetic of the first. This distinction becomes important whenever a recurring report can be revised after its first issue.

```
python3 tools/run_experiments.py 11 --out my_experiment_11
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/weekly_request_monitor.py

```
"""Chapter 11: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(11)
```

The function `weekly` pools the response rate over the final weeks; provisional weeks are shown but not pooled.

programs/legal_apps.py, lines 720–764

```
# Chapter 11: pool counts, never unweighted percentages.
def weekly(folder: Path, **_) -> Result:
```

```
  """Chapter 11: pooled response rate over the final weeks; provisional weeks shown but not
pooled.

  The headline divides the sum of responded by the sum of eligible over
  final weeks. Any structural problem in any line (non-integer counts,
  responded > eligible, a week not beginning on a Monday, a repeated week,
  an unknown status) suppresses the headline.
  """
  fields = ["week_id", "week_beginning", "eligible", "responded", "status"]
  rows = load(folder, "weeks.csv", fields)
  issues = identity_issues(rows, "week_id")
  week_counts = Counter(r["week_beginning"] for r in rows)
  ledger = []
  en = rn = 0
  structural = False
  for i, r in enumerate(rows):
      e = issues[i]
      n = natural(r["eligible"])
      v = natural(r["responded"])
      d = day(r["week_beginning"])
      if n is None or v is None or v > n:
          e.append("invalid_counts")
      if d is None or d.weekday() != 0:
          e.append("invalid_monday_week")
      if week_counts[r["week_beginning"]] > 1:
          e.append("duplicate_week")
      if r["status"] not in {"final", "provisional"}:
          e.append("unknown_status")
      structural |= bool(e)
      if not e and r["status"] == "final":
          en += n
          rn += v
      ledger.append(dict(r, rate="" if e or not n else v / n * 100, issues=";".join(e)))
  result = Result(
      "Weekly request monitor",
      {
          "eligible_final": en,
          "responded_final": rn,
          "pooled_percentage": None if structural or not en else rn / en * 100,
          "headline_available": not structural and en > 0,
      },
  )
  result.add("weeks.csv", ledger, fields + ["rate", "issues"])
  return result
```

Chapter 12

# Build a Production Index

**Row unit.** One produced document with a supplied Bates range.
**Reference entry point.** `programs/production_index.py`

```
python3 programs/production_index.py --out my_ch12
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch12/report.html`
**Outputs.** `gaps.csv`, `index.csv`, `overlaps.csv`, `report.html`, `run.json`, `summary.json`, `volumes.csv`.

## Complete Small Inputs and Their Headers

### production.csv

Exact columns, in order: `doc_id`, `matter_id`, `volume_id`, `bates_start`, `bates_end`, `page_count`, `family_id`, `doc_type`, `source_id`.

`doc_id`
Unique document identifier; a blank or repeated value holds every occurrence.

`matter_id`
Supplied matter key, checked against the matter register.

`volume_id`
Delivery batch label, required; numbering is checked per prefix across volumes, and volume endpoints are taken from all well-formed ranges.

`bates_start`
First label of the range: two to eight capital letters followed by exactly six digits.

`bates_end`
Last label of the range; it must share the prefix of bates_start and may not precede it.

`page_count`
Supplied page count, which must equal the length of the range.

`family_id`
Document identifier of the family's parent, required; a standalone document names itself, and a parent missing from the register holds the row.

`doc_type`
Supplied document type, such as `email` or `attachment`, required and otherwise carried through.

`source_id`
Supplied source reference, required to be nonblank.

data/ch12/production.csv

```
doc_id,matter_id,volume_id,bates_start,bates_end,page_count,family_id,doc_type,source_id
D01,M01,VOL001,ABC000001,ABC000004,4,D01,email,PRD-S01
D02,M01,VOL001,ABC000005,ABC000006,2,D01,attachment,PRD-S02
D03,M01,VOL001,ABC000007,ABC000007,1,D03,letter,PRD-S03
D04,M01,VOL001,ABC000008,ABC000012,5,D04,agreement draft,PRD-S04
D05,M01,VOL001,ABC000015,ABC000016,2,D05,email,PRD-S05
D06,M01,VOL001,ABC000016,ABC000018,3,D06,memo,PRD-S06
D07,M01,VOL002,ABC000019,ABC000020,3,D07,letter,PRD-S07
D08,M01,VOL002,ABC000021,ABC000021,1,D99,attachment,PRD-S08
D09,M01,VOL002,ABC000022,ABC000025,4,D09,email,PRD-S09
D10,M01,VOL002,ABC000026,ABC000026,1,D09,attachment,PRD-S10
```

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch12/matters.csv

```
matter_id,client_id,description
M01,C01,Agreement documents
M02,C01,Employment records
M03,C02,Correspondence review
```

## How the Function Works

The function first checks each row on its own: identity, required fields, matter, label format, prefix, order, and page count. Every well-formed range then records a claim on each of its numbers in a dictionary keyed by prefix and number. A number with two or more claimants is an overlap, and every claimant is held. Gaps are found by sorting the claimed numbers of each prefix and reporting every jump of more than one. Held documents keep their claims, so the gap list contains only numbers that no row claims. The index is written in Bates order; each row keeps its `source_row`.

## One Controlled Experiment

Change D06's `bates_start` to ABC000017 and its `page_count` to 2. The overlap at ABC000016 disappears, so D05 and D06 are accepted: eight accepted, two held, and 21 accepted pages. The gap ABC000013–ABC000014 remains, because correcting one range says nothing about the missing numbers.

```
python3 tools/run_experiments.py 12 --out my_experiment_12
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/production_index.py

```
"""Chapter 12: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(12)
```

The function `production_index` checks the supplied Bates ranges and lists gaps and overlaps; it never renumbers. The pattern `BATES`, the field list, and the helpers `bates` and `label` just above it parse and print the labels.

programs/legal_apps.py, lines 767–943

```
# Chapter 12: supplied Bates labels; gaps and overlaps are reported, never repaired.
BATES = re.compile(r"([A-Z]{2,8})([0-9]{6})")
PRODUCTION_FIELDS = [
    "doc_id",
    "matter_id",
    "volume_id",
    "bates_start",
    "bates_end",
    "page_count",
    "family_id",
    "doc_type",
    "source_id",
]


def bates(value: str) -> tuple[str, int] | None:
    found = BATES.fullmatch(value)
    return (found.group(1), int(found.group(2))) if found else None


def label(prefix: str, number: int) -> str:
    return f"{prefix}{number:06d}"


def production_index(folder: Path, **_) -> Result:
    """Chapter 12: check supplied Bates ranges; list gaps and overlaps; never renumber.

    A document is held for a blank or duplicated ID, an unregistered matter,
    a blank required field, a malformed label, mixed prefixes, an end before
    its start, a page count that disagrees with the range, any number also
    claimed by another document (every claimant), or a family parent missing
    from the register. Well-formed ranges claim their numbers even when the
    document is held, so a gap is a number that no supplied range claims.
    """
    rows = load(folder, "production.csv", PRODUCTION_FIELDS)
    known = registry(folder)
    issues = identity_issues(rows, "doc_id")
    doc_ids = {r["doc_id"] for r in rows}
    spans = {}
    claims = defaultdict(list)
    for i, r in enumerate(rows):
        e = issues[i]
        require_cells(r, ["volume_id", "family_id", "doc_type", "source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        first = bates(r["bates_start"])
        last = bates(r["bates_end"])
        pages = natural(r["page_count"])
        if first is None or last is None:
            e.append("invalid_bates_label")
        elif first[0] != last[0]:
            e.append("prefix_mismatch")
        elif last[1] < first[1]:
            e.append("end_before_start")
        else:
            spans[i] = (first[0], first[1], last[1])
            for n in range(first[1], last[1] + 1):
                claims[(first[0], n)].append(i)
            if pages is None or pages != last[1] - first[1] + 1:
                e.append("page_count_disagrees_with_range")
```

```
        if r["family_id"].strip() and r["family_id"] not in doc_ids:
            e.append("missing_family_parent")
    overlap_numbers = sorted(k for k, v in claims.items() if len(v) > 1)
    for prefix, number in overlap_numbers:
        for i in claims[(prefix, number)]:
            if "overlapping_range" not in issues[i]:
                issues[i].append("overlapping_range")
    gaps = []
    for prefix in sorted({p for p, _ in claims}):
        numbers = sorted(n for p, n in claims if p == prefix)
        previous = numbers[0]
        for n in numbers[1:]:
            if n > previous + 1:
                gaps.append(
                    {
                        "prefix": prefix,
                        "gap_start": label(prefix, previous + 1),
                        "gap_end": label(prefix, n - 1),
                        "missing_numbers": n - previous - 1,
                    }
                )
            previous = n
    index = []
    accepted_pages = 0
    for i, r in enumerate(rows):
        e = issues[i]
        status = "held" if e else "accepted"
        if not e:
            accepted_pages += spans[i][2] - spans[i][1] + 1
        index.append(
            dict(
                r,
                source_row=i + 2,
                range_length="" if i not in spans else spans[i][2] - spans[i][1] + 1,
                status=status,
                issues=";".join(e),
            )
        )
    index.sort(
        key=lambda x: (spans.get(x["source_row"] - 2, ("~", 10**9, 0))[:2], x["source_row"])
    )
    volumes = []
    for volume in sorted({r["volume_id"] for r in rows}):
        members = [x for x in index if x["volume_id"] == volume]
        valid_spans = [
            spans[x["source_row"] - 2] for x in members if x["source_row"] - 2 in spans
        ]
        prefixes = {span[0] for span in valid_spans}
        available = len(valid_spans) == len(members) and len(prefixes) == 1
        first_label = (
            label(valid_spans[0][0], min(x[1] for x in valid_spans)) if available else ""
        )
        last_label = (
            label(valid_spans[0][0], max(x[2] for x in valid_spans)) if available else ""
        )
        range_note = (
            "supplied_ranges_including_held"
            if available
            else "unavailable_invalid_or_mixed_prefix_ranges"
        )
```

```
        volumes.append(
            {
                "volume_id": volume,
                "documents": len(members),
                "accepted": sum(x["status"] == "accepted" for x in members),
                "held": sum(x["status"] == "held" for x in members),
                "first_label": first_label,
                "last_label": last_label,
                "range_note": range_note,
            }
        )
    accepted = sum(x["status"] == "accepted" for x in index)
    result = Result(
        "Production index",
        {
            "input_rows": len(rows),
            "accepted": accepted,
            "held": len(rows) - accepted,
            "accepted_pages": accepted_pages,
            "supplied_page_count_total": sum(natural(r["page_count"]) or 0 for r in rows),
            "claimed_numbers": len(claims),
            "gap_ranges": len(gaps),
            "gap_numbers": sum(g["missing_numbers"] for g in gaps),
            "overlap_numbers": len(overlap_numbers),
        },
    )
    result.add(
        "index.csv",
        index,
        ["source_row"] + PRODUCTION_FIELDS + ["range_length", "status", "issues"],
    )
    result.add("gaps.csv", gaps, ["prefix", "gap_start", "gap_end", "missing_numbers"])
    result.add(
        "overlaps.csv",
        [
            {
                "label": label(p, n),
                "claimed_by": ";".join(rows[i]["doc_id"] for i in claims[(p, n)]),
            }
            for p, n in overlap_numbers
        ],
        ["label", "claimed_by"],
    )
    result.add(
        "volumes.csv",
        volumes,
        [
            "volume_id",
            "documents",
            "accepted",
            "held",
            "first_label",
            "last_label",
            "range_note",
        ],
    )
    return result
```

Chapter 13

# Prepare Administrative Letters

**Row unit.** One request for a draft meeting confirmation.
**Reference entry point.** `programs/administrative_letters.py`

```
python3 programs/administrative_letters.py --out my_ch13
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch13/report.html`
**Outputs.** `letter_L01.html`, `letter_L02.html`, `letter_L04.html`, `letter_L06.html`, `manifest.csv`, `report.html`, `run.json`, `summary.json`.

## Complete Small Inputs and Their Headers

### letter_template.txt

data/ch13/letter_template.txt

```
DRAFT FOR REVIEW
Meeting confirmation

To: {recipient}
Matter: {matter_id}

Your meeting is recorded for {meeting_date} at {meeting_time} ({time_zone}) by {method}.

{instruction}

Administrative note: {note}
```

### letters.csv

Exact columns, in order: `letter_id`, `matter_id`, `recipient`, `meeting_date`, `meeting_time`, `time_zone`, `method`, `instruction`, `note`.

`letter_id`
Unique request identifier of letters, digits, hyphens, and underscores; it names the draft `letter_<id>.html`, and identifiers differing only in letter case are held.

`matter_id`
Supplied matter key, checked against `matters.csv`.

`recipient`
Supplied recipient text, required; it is escaped when inserted into the draft.

`meeting_date`
Real calendar date in `YYYY-MM-DD` form, required; September 31 holds L05.

`meeting_time`
24-hour time in `HH:MM` form, required.

`time_zone`
Must be exactly `America/Toronto`, the only zone in this exercise.

`method`
Supplied meeting method, such as telephone, required and inserted as text.

`instruction`
Supplied instruction text, required and inserted as text.

`note`
Optional supplied note; it may be blank and is escaped when inserted.

data/ch13/letters.csv

```
letter_id,matter_id,recipient,meeting_date,meeting_time,time_zone,method,instruction,note
L01,M01,Example Recipient 1,2026-09-25,14:00,America/Toronto,telephone,Use the contact
  arrangement already agreed with the office.,
L02,M02,A & B Example,2026-09-25,14:00,America/Toronto,telephone,Use the contact arrangement
  already agreed with the office.,<review>
L03,M03,,2026-09-25,14:00,America/Toronto,telephone,Use the contact arrangement already
  agreed with the office.,
L04,M01,Example Recipient 4,2026-09-25,14:00,America/Toronto,telephone,Use the contact
  arrangement already agreed with the office.,
L05,M02,Example Recipient 5,2026-09-31,14:00,America/Toronto,telephone,Use the contact
  arrangement already agreed with the office.,
L06,M03,Example Recipient 6,2026-09-25,14:00,America/Toronto,telephone,Use the contact
  arrangement already agreed with the office.,
```

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch13/matters.csv

```
matter_id,client_id,description
M01,C01,Agreement documents
M02,C01,Employment records
M03,C02,Correspondence review
```

## One Controlled Experiment

Supply the fictional recipient Example Recipient 3 for L03. The rerun generates five draft letters and holds L05 for its invalid date. Compare L03 with its source row, and confirm that the manifest still has six entries.

Inspect L02 as well. Its ampersand and angle-bracket note must remain literal text. A change to one recipient should not alter another recipient's content or escaping. The reference uses validated request identifiers for filenames and rejects a request identifier containing a path. This keeps source text from deciding where a letter is written.

```
python3 tools/run_experiments.py 13 --out my_experiment_13
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/administrative_letters.py

```
"""Chapter 13: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(13)
```

The function `letters` fills a fixed template for each complete request and holds the rest with reasons.

programs/legal_apps.py, lines 946–1036

```
# Chapter 13: fixed template; all records accounted for, no sending.
def letters(folder: Path, **_) -> Result:
    """Chapter 13: fill a fixed template for each complete request; hold the rest with reasons.


    The template may use only placeholders naming columns of letters.csv
    (other than letter_id) and must contain DRAFT FOR REVIEW. Every inserted
    value is HTML-escaped. Letter files are named letter_<id>.html; identifiers
    that collide case-insensitively are held, and the output writer checks the
    whole set of names before creating the destination.
    """
    fields = [
        "letter_id",
        "matter_id",
        "recipient",
        "meeting_date",
        "meeting_time",
        "time_zone",
        "method",
        "instruction",
        "note",
    ]
    rows = load(folder, "letters.csv", fields)
    known = registry(folder)
    issues = identity_issues(rows, "letter_id")
    template = (folder / "letter_template.txt").read_text(encoding="utf-8")
    manifest = []
    extras = {}
    generated = 0
    filename_spellings = defaultdict(set)
    for row in rows:
        filename_spellings[row["letter_id"].casefold()].add(row["letter_id"])
    allowed = set(fields) - {"letter_id"}
    import string

    for _, field, format_spec, conversion in string.Formatter().parse(template):
        if field is not None and (field not in allowed or format_spec or conversion):
            raise ValueError("Template contains an unapproved substitution")
    if "DRAFT FOR REVIEW" not in template:
        raise ValueError("Template must retain draft label")
    for i, r in enumerate(rows):
        e = issues[i]
        require_cells(
            r,
            ["recipient", "meeting_date", "meeting_time", "time_zone", "method", "instruction
  "],
            e,
        )
        if not re.fullmatch(r"[A-Za-z0-9_-]+", r["letter_id"]):
            e.append("unsafe_letter_id")
        if len(filename_spellings[r["letter_id"].casefold()]) > 1:
            e.append("case_insensitive_filename_collision")
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if day(r["meeting_date"]) is None:
            e.append("invalid_meeting_date")
        if not re.fullmatch(r"(?:[01][0-9]|2[0-3]):[0-5][0-9]", r["meeting_time"]):
            e.append("invalid_time")
        if r["time_zone"] != "America/Toronto":
```

```
            e.append("outside_teaching_time_zone")
        output = "" if e else "letter_" + r["letter_id"] + ".html"
        if not e:
            text = template.format_map(r)
            extras[output] = page(
                "Meeting confirmation", '<div class="letter"><pre>' + esc(text) + "</pre></
div>"
            )
            generated += 1
        manifest.append(
            {
                "source_row": i + 2,
                "letter_id": r["letter_id"],
                "matter_id": r["matter_id"],
                "status": "held" if e else "generated",
                "output": output,
                "issues": ";".join(e),
            }
        )
    result = Result(
        "Draft administrative letter pack",
        {
            "input_rows": len(rows),
            "generated": generated,
            "held": len(rows) - generated,
            "template_sha256": sha(folder / "letter_template.txt"),
        },
        extra=extras,
    )
    result.add(
        "manifest.csv",
        manifest,
        ["source_row", "letter_id", "matter_id", "status", "output", "issues"],
    )
    return result
```

Chapter 14

# Track Professionally Supplied Dates

**Row unit.** One supplied-date record for a matter and task.
**Reference entry point.** `programs/supplied_date_register.py`

```
python3 programs/supplied_date_register.py --out my_ch14
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch14/report.html`
**Outputs.** `date_register.csv`, `report.html`, `run.json`, `summary.json`.

## Complete Small Inputs and Their Headers

### dates.csv

Exact columns, in order: `date_id`, `matter_id`, `task_id`, `supplied_date`, `source_id`, `verifier`, `verification_date`, `owner`, `completed`, `record_status`, `supersedes`.

`date_id`
Unique nonblank identifier of the entry; a blank or repeated value sends it to review.

`matter_id`
Supplied matter key, checked against `matters.csv`.

`task_id`
Supplied task label, required; with matter_id it defines the unit within which current entries may not conflict and supersession links must stay.

`supplied_date`
Date entered from an external professional decision in `YYYY-MM-DD` form; it is compared with the reporting date, never calculated from law.

`source_id`
Supplied source reference, required to be nonblank; not authentication.

`verifier`
Supplied reviewer label, required to be nonblank; not proof of reviewer identity or authority.

`verification_date`
Date of the recorded verification; a missing or invalid date, or one later than the reporting date, sends the entry to review.

`owner`
Supplied assignment label, required to be nonblank and never inferred by the application.

`completed`
Exactly `yes` or `no`; `yes` classifies a usable entry as recorded complete before any date comparison.

`record_status`
Exactly `current` or `superseded`; a superseded entry is kept and must be named by exactly one successor.

`supersedes`
Identifier of the earlier entry this one replaces, or blank; it must name a single superseded entry for the same matter and task, with no self-link or cycle.

data/ch14/dates.csv

```
date_id,matter_id,task_id,supplied_date,source_id,verifier,verification_date,owner,completed,
  record_status,supersedes
D01,M01,TASK01,2026-09-25,DATE-SOURCE-01,Reviewer Example,2026-09-20,A,no,current,
D02,M02,TASK02,2026-09-24,DATE-SOURCE-02,Reviewer Example,2026-09-20,B,no,current,
D03,M03,TASK03,2026-09-23,DATE-SOURCE-03,Reviewer Example,2026-09-20,A,no,current,
D04,M01,TASK04,2026-09-31,DATE-SOURCE-04,Reviewer Example,2026-09-20,B,no,current,
D05,M02,TASK05,2026-09-26,DATE-SOURCE-05,,2026-09-20,A,no,current,
D06,M03,TASK06,2026-09-22,DATE-SOURCE-06,Reviewer Example,2026-09-20,B,yes,current,
```

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch14/matters.csv

```
matter_id,client_id,description
M01,C01,Agreement documents
M02,C01,Employment records
M03,C02,Correspondence review
```

## One Controlled Experiment

Change D01's verification date to September 25 while keeping the report at September 24. D01 moves from upcoming to needs review. The total number needing review becomes three. Its supplied September 25 date has not changed; the problem concerns the evidence available as of the reporting date.

A different experiment marks the old D01 record superseded and adds a new current record for the same matter and task, with a new identity and a link to D01. The reference keeps the superseded row. Adding another current row without superseding the first instead flags both current entries as conflicting. These cases are checked separately in the suite.

```
python3 tools/run_experiments.py 14 --out my_experiment_14
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/supplied_date_register.py

```
"""Chapter 14: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(14)
```

The function `supplied_dates` classifies supplied dates relative to a reporting date, checking completeness first.

programs/legal_apps.py, lines 1039–1150

```
# Chapter 14: record completeness is not authentication of a legal deadline.
def supplied_dates(folder: Path, as_of: str = "2026-09-24", **_) -> Result:
    """Chapter 14: classify supplied dates relative to a reporting date, completeness first.
```

```
    An entry needs review for a blank or duplicated identifier, a missing
    verifier, source or owner, an unregistered matter, an invalid supplied
    date, a verification date that is missing or after the reporting date,
    unexpected flag values, a second current entry for the same matter and
    task, or a broken supersession chain. Only a usable entry is then
    classified as superseded, recorded_complete, past_recorded_date, today or
    upcoming, in that order.
    """
    fields = [
        "date_id",
        "matter_id",
        "task_id",
        "supplied_date",
        "source_id",
        "verifier",
        "verification_date",
        "owner",
        "completed",
        "record_status",
        "supersedes",
    ]
    rows = load(folder, "dates.csv", fields)
    known = registry(folder)
    cutoff = required_day(as_of)
    issues = identity_issues(rows, "date_id")
    current = Counter(
        (r["matter_id"], r["task_id"]) for r in rows if r["record_status"] == "current"
    )
    idcounts = Counter(r["date_id"] for r in rows)
    by_id = {r["date_id"]: r for r in rows}
    ledger = []
    successors = Counter(r["supersedes"] for r in rows if r["supersedes"])
    cycle_ids = set()
    for ident in by_id:
        chain = []
        current_id = ident
        while current_id in by_id and current_id not in chain:
            chain.append(current_id)
            current_id = by_id[current_id]["supersedes"]
        if current_id in chain:
            cycle_ids.update(chain[chain.index(current_id) :])
    for i, r in enumerate(rows):
        e = issues[i]
        d = day(r["supplied_date"])
        v = day(r["verification_date"])
        require_cells(r, ["task_id", "source_id", "verifier", "owner"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if d is None:
            e.append("invalid_supplied_date")
        if v is None or v > cutoff:
            e.append("invalid_or_future_verification")
        if r["completed"] not in {"yes", "no"}:
            e.append("invalid_completion_flag")
        if r["record_status"] not in {"current", "superseded"}:
            e.append("invalid_record_status")
        if r["record_status"] == "current" and current[(r["matter_id"], r["task_id"])] > 1:
            e.append("conflicting_current_dates")
        if r["date_id"] in cycle_ids:
```

```
            e.append("cyclic_supersession")
        if r["supersedes"] == r["date_id"]:
            e.append("self_supersession")
        if r["record_status"] == "superseded" and successors[r["date_id"]] != 1:
            e.append("missing_or_ambiguous_successor")
        if r["supersedes"]:
            old = by_id.get(r["supersedes"])
            if (
                idcounts[r["supersedes"]] != 1
                or old is None
                or old["record_status"] != "superseded"
                or (old["matter_id"], old["task_id"]) != (r["matter_id"], r["task_id"])
            ):
                e.append("invalid_supersession_link")
        status = (
            "needs_review"
            if e
            else "superseded"
            if r["record_status"] == "superseded"
            else "recorded_complete"
            if r["completed"] == "yes"
            else "past_recorded_date"
            if d < cutoff
            else "today"
            if d == cutoff
            else "upcoming"
        )
        ledger.append(dict(r, classification=status, issues=";".join(e)))
    counts = Counter(r["classification"] for r in ledger)
    result = Result(
        "Professionally supplied date register",
        dict(
            input_rows=len(rows),
            reporting_date=as_of,
            **{
                s: counts[s]
                for s in [
                    "needs_review",
                    "superseded",
                    "recorded_complete",
                    "past_recorded_date",
                    "today",
                    "upcoming",
                ]
            },
        ),
    )
    result.add("date_register.csv", ledger, fields + ["classification", "issues"])
    return result
```

Chapter 15

# Search Names and Parties

**Row unit.** One fictional entity, with supplied aliases and relationships.

**Reference entry point.** `programs/party_search.py`

```
python3 programs/party_search.py --out my_ch15
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch15/report.html`

**Outputs.** `candidates.csv`, `report.html`, `run.json`, `summary.json`.

Use `--query "Alex Morgen"` to test the one-edit spelling rule; `--exact-only` excludes spelling candidates while retaining primary-name and alias matching.

## Complete Small Inputs and Their Headers

### parties.csv

Exact columns, in order: `entity_id`, `name`, `aliases`, `role`, `matter_id`, `source_id`.

`entity_id`
Unique nonblank entity identifier; a blank or repeated value stops the search.

`name`
Primary name, required to be nonblank; it is normalized and compared with the query as a whole.

`aliases`
Supplied aliases separated by a vertical bar, or blank; each is compared as a whole name, and none is inferred from the internet.

`role`
Supplied role label in a cell, displayed and not interpreted.

`matter_id`
Supplied matter reference, carried into the candidate row.

`source_id`
Supplied source reference, required to be nonblank; not authentication.

data/ch15/parties.csv

```
entity_id,name,aliases,role,matter_id,source_id
N01,Alex Morgan,A. Morgan,former client,M01,ENTITY01
N02,Alex Morgan,,opposing party,M02,ENTITY02
N03,Maple Services Ltd,Maple Service,associated entity,M03,ENTITY03
N04,Avery Li,,prospective contact,M04,ENTITY04
```

## One Controlled Experiment

Search for Alex Morgen using the optional one-edit rule. Both Alex Morgan entities appear as possible primary-name matches. Their entity IDs, roles, and matter references remain different. Repeat the query with exact-only matching: it returns no candidate for that spelling.

The difference is a changed search rule, not evidence that either entity ceased to exist. Save the query, normalization rule, and register hash. When a reader examines a search result later, those details explain what was and was not attempted. The reference does not infer corporate relationships or query an external database.

```
python3 tools/run_experiments.py 15 --out my_experiment_15
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/party_search.py

```
"""Chapter 15: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(15)
```

The helper `normalize` applies Unicode compatibility normalization (NFKC), then Unicode case folding, then `str.split` and a join with one ordinary space. This removes leading and trailing whitespace and collapses internal whitespace runs. Compatibility normalization can also change compatibility characters, including full-width forms; it is not a promise that every punctuation character stays byte-for-byte unchanged. The rule does not otherwise remove punctuation, corporate suffixes, or parts of names. The helper `one_edit` is true when the normalized strings differ by

exactly one inserted, deleted, or substituted character; `parties` keeps all matching entity identities rather than merging their records.

programs/legal_apps.py, lines 1153–1235

```
# Chapter 15: normalized exact/alias/one-edit candidates, never clearance.
def normalize(text: str) -> str:
    """Chapter 15: NFKC compatibility form, casefold, then trim/collapse Unicode whitespace.
  """
    return " ".join(unicodedata.normalize("NFKC", text).casefold().split())


def one_edit(a: str, b: str) -> bool:
    """Chapter 15: true when a and b differ by exactly one inserted, deleted or substituted
  character."""
    if a == b:
        return False
    if abs(len(a) - len(b)) > 1:
        return False
    if len(a) == len(b):
        return sum(x != y for x, y in zip(a, b)) == 1
    if len(a) > len(b):
        a, b = b, a
    i = j = errors = 0
    while i < len(a) and j < len(b):
        if a[i] == b[j]:
            i += 1
            j += 1
        else:
            errors += 1
            j += 1
        if errors > 1:
            return False
    return True


def parties(folder: Path, query: str = "Alex Morgan", similar: bool = True, **_) -> Result:
    """Chapter 15: list every entity whose primary name or alias matches the query.

    Exact matches compare normalized whole names. With similar=True a name one
    edit away is a possible match. Entities with the same name stay separate.
    The register fingerprint and the scope line are part of every result.
    """
    fields = ["entity_id", "name", "aliases", "role", "matter_id", "source_id"]
    rows = load(folder, "parties.csv", fields)
    if not normalize(query):
        raise ValueError("A nonblank query is required")
    if any(identity_issues(rows, "entity_id")) or any(
        not r["name"].strip() or not r["source_id"].strip() for r in rows
    ):
        raise ValueError("Invalid party register")
    q = normalize(query)
    matches = []
    for r in rows:
        candidates = [("primary_name", r["name"])] + [
            ("alias", s) for s in r["aliases"].split("|") if s.strip()
        ]
        for kind, text in candidates:
            value = normalize(text)
            label = (
```

```
                "exact_" + kind
                if q == value
                else "possible_" + kind
                if similar and one_edit(q, value)
                else ""
            )
            if label:
                matches.append(
                    dict(
                        r,
                        match_type=label,
                        matched_text=text,
                        explanation="normalized exact match"
                        if q == value
                        else "one insertion, deletion, or substitution",
                    )
                )
    result = Result(
        "Name-and-party candidates for review",
        {
            "query": query,
            "matches": len(matches),
            "register_sha256": sha(folder / "parties.csv"),
            "scope": "This register and this rule only; no professional clearance.",
        },
    )
    result.add(
        "candidates.csv", matches, fields + ["match_type", "matched_text", "explanation"]
    )
    return result
```

Chapter 16

# Organize Legal Authorities

**Row unit.** One retrieved authority record; propositions have a separate worksheet.
**Reference entry point.** `programs/authority_register.py`

```
python3 programs/authority_register.py --out my_ch16
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch16/report.html`
**Outputs.** `membership.csv`, `propositions.csv`, `report.html`, `run.json`, `summary.json`, `ungrouped.csv`.

## Complete Small Inputs and Their Headers

### authorities.csv

Exact columns, in order: `record_id`, `authority_id`, `version`, `kind`, `related_authority`, `relationship`, `source_id`, `verification`, `reviewer`, `review_date`.

`record_id`
Unique identifier of one retrieved record; a blank or repeated value lists the record as ungrouped.

`authority_id`
Identifier of the source the record represents, required for grouping; distinct values count the authorities.

`version`
Edition or date of the authority, required for grouping; with authority_id it identifies an authority-version.

`kind`
Supplied kind of authority, such as decision, legislation, or commentary, displayed as supplied.

`related_authority`
Identifier of a different authority this record is related to, or blank; a relationship never merges two authorities.

`relationship`
Supplied description of the link to related_authority, displayed as supplied.

`source_id`
Supplied source reference, required for grouping; not authentication.

`verification`
Supplied verification status, carried through and neither checked nor interpreted.

`reviewer`
Supplied reviewer label, carried through and neither checked nor interpreted.

`review_date`
Supplied date of the review, carried through and neither checked nor interpreted.

data/ch16/authorities.csv

```
record_id,authority_id,version,kind,related_authority,relationship,source_id,verification,
  reviewer,review_date
Q01,DEC01,2026,decision,,,SOURCE-Q01,recorded,Reviewer Example,2026-09-20
Q02,DEC01,2026,decision,,,SOURCE-Q02,recorded,Reviewer Example,2026-09-20
Q03,DEC02,2026,decision,DEC01,separate later decision,SOURCE-Q03,recorded,Reviewer Example
  ,2026-09-20
Q04,LEG01,2024,legislation,,,SOURCE-Q04,recorded,Reviewer Example,2026-09-20
Q05,LEG01,2026,legislation,,,SOURCE-Q05,recorded,Reviewer Example,2026-09-20
Q06,COM01,2026,commentary,,,SOURCE-Q06,recorded,Reviewer Example,2026-09-20
Q07,DEC03,2026,decision,,,SOURCE-Q07,pending,,
Q08,,2026,unknown,,,SOURCE-Q08,pending,,
```

### propositions.csv

Exact columns, in order: `proposition_id`, `authority_id`, `version`, `source_location`, `text`, `text_kind`, `reviewer`, `review_status`.

`proposition_id`
Unique nonblank proposition identifier.

`authority_id`
Authority the proposition relies on; with version it must name a grouped authority-version.

`version`
Version of that authority to which the source location refers.

`source_location`
Precise source pointer supplied by the researcher, required; the organizer does not verify its legal support.

`text`
The proposition's text, required to be nonblank.

`text_kind`
Exactly `exact passage` or `researcher summary`; the two are kept labelled separately.

`reviewer`
Supplied reviewer label; required when the review status is `recorded`.

`review_status`
Supplied status such as `recorded` or `pending`; a recorded proposition needs a reviewer.

data/ch16/propositions.csv

```
proposition_id,authority_id,version,source_location,text,text_kind,reviewer,review_status
P01,DEC01,2026,Synthetic passage 4,Invented worksheet proposition; no real legal claim.,
  researcher summary,Reviewer Example,recorded
P02,LEG01,2024,Synthetic section 2,Invented historical-version note.,researcher summary,,
  pending
```

## One Controlled Experiment

Remove the precise source location from P01 in a copy of the proposition worksheet. The program reports a missing-source-location issue. The authority totals remain five groups and six authority-version groups because the authority register did not change.

This is a useful separation of failures. An authority can be organized correctly while a proposition's support remains incompletely recorded. The worksheet gives the reviewer somewhere to record the missing pinpoint without inventing one. Keep exact passages and researcher summaries labelled separately, and preserve the version to which a location refers.

```
python3 tools/run_experiments.py 16 --out my_experiment_16
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/authority_register.py

```
"""Chapter 16: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(16)
```

The function `authorities` groups retrieved records by authority identifier and version and checks the proposition worksheet.

programs/legal_apps.py, lines 1238–1305

```
# Chapter 16: preserve supplied authority identities and researcher propositions.
def authorities(folder: Path, **_) -> Result:
    """Chapter 16: group retrieved records by (authority_id, version); check the proposition
  worksheet.

    A record is grouped when record_id is present and unique and authority_id,
    version and source_id are present; otherwise it is listed as ungrouped. A
    related authority is displayed, never merged. Each proposition needs a
    source location, a text, a permitted text kind, an authority-version that
    exists in the register, and a reviewer when marked recorded.
    """
    fields = [
        "record_id",
        "authority_id",
        "version",
        "kind",
        "related_authority",
        "relationship",
        "source_id",
        "verification",
        "reviewer",
        "review_date",
    ]
    rows = load(folder, "authorities.csv", fields)
    issues = identity_issues(rows, "record_id")
    membership = []
    held = []
    for i, r in enumerate(rows):
        e = issues[i]
        require_cells(r, ["authority_id", "version", "source_id"], e)
        item = dict(r, issues=";".join(e))
        (held if e else membership).append(item)
    prop_fields = [
        "proposition_id",
        "authority_id",
        "version",
        "source_location",
        "text",
        "text_kind",
        "reviewer",
        "review_status",
    ]
```

```
    props = load(folder, "propositions.csv", prop_fields)
    pissues = identity_issues(props, "proposition_id")
    known = {(r["authority_id"], r["version"]) for r in membership}
    worksheet = []
    for i, r in enumerate(props):
        e = pissues[i]
        require_cells(r, ["source_location", "text", "text_kind"], e)
        if (r["authority_id"], r["version"]) not in known:
            e.append("unknown_authority_version")
        if r["text_kind"] not in {"exact passage", "researcher summary"}:
            e.append("invalid_text_kind")
        if r["review_status"] == "recorded" and not r["reviewer"].strip():
            e.append("missing_reviewer")
        worksheet.append(dict(r, issues=";".join(e)))
    result = Result(
        "Authority and proposition register",
        {
            "records": len(rows),
            "authorities": len({r["authority_id"] for r in membership}),
            "versions": len({(r["authority_id"], r["version"]) for r in membership}),
            "ungrouped": len(held),
        },
    )
    result.add("membership.csv", membership, fields + ["issues"])
    result.add("ungrouped.csv", held, fields + ["issues"])
    result.add("propositions.csv", worksheet, prop_fields + ["issues"])
    return result
```

Chapter 17

# Compare Document Versions

**Row unit.** One explicitly identified paragraph in each complete text.

**Reference entry point.** `programs/document_compare.py`

```
python3 programs/document_compare.py --out my_ch17
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch17/report.html`

**Outputs.** `comparison.csv`, `report.html`, `run.json`, `summary.json`.

Use `--reverse` to exchange original and revised directions. With `--whitespace`, status comparison ignores leading and trailing whitespace and replaces each internal run of Unicode whitespace, including tabs, with one ordinary space. The original texts remain exact in the saved table. The browser's insertion/deletion marks still compare those original texts, so an ignored whitespace difference may remain marked beside an `unchanged` normalized status. The status label names the selected rule; the marks show the literal difference.

## Complete Small Inputs and Their Headers

### original.txt

data/ch17/original.txt

```
P01 The supplier may provide the records within 10 days.
P02 The recipient shall not disclose the draft.
P03 The stated amount is $10,000.
P04 The parties will keep a copy of this document.
```

### revised.txt

data/ch17/revised.txt

```
P01 The supplier must provide the records within 14 days.
P02 The recipient shall disclose the draft.
P03 The stated amount is $100,000.
P04 The parties will keep a copy of this document.
```

## One Controlled Experiment

Reverse the original and revised inputs without changing their bytes. The deleted not in the original direction becomes an insertion. The changed amount and timing phrases reverse as well. There are still three textually changed paragraphs and one unchanged paragraph.

Next compare the original with an identical copy, move an unchanged paragraph, and add an extra space. The tests distinguish exact text, paragraph position, and optional whitespace normalization. A position change is reported separately from changed wording. Identical wording under the selected rule is stated explicitly; an empty page is not used as the no-change result.

```
python3 tools/run_experiments.py 17 --out my_experiment_17
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/document_compare.py

```
"""Chapter 17: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(17)
```

The helper `read_paragraphs` reads one paragraph per line as an identifier, a tab, and the text; `compare_documents` compares the files by identifier and reports movement separately from changed text. Its difference tokens are whitespace runs, runs of Unicode word characters (letters, digits, and underscores), and individual remaining punctuation characters. The literal difference marks use the original tokens even when the paragraph status uses whitespace normalization.

programs/legal_apps.py, lines 1308–1422

```
# Chapter 17: explicit paragraph IDs separate movement from text differences.
def read_paragraphs(path: Path) -> list[tuple[str, str]]:
    """Chapter 17: read one paragraph per line as (identifier, text), separated by a tab."""
    result = []
    ids = set()
    for line in path.read_text(encoding="utf-8").splitlines():
        if "\t" not in line:
            raise ValueError("Each paragraph needs ID, tab, and complete text")
        ident, text = line.split("\t", 1)
        if not ident.strip() or ident in ids:
            raise ValueError("Paragraph IDs must be unique and nonblank")
        ids.add(ident)
        result.append((ident, text))
```

```
    return result


def compare_documents(
    folder: Path, whitespace: bool = False, reverse: bool = False, **_
) -> Result:
    """Chapter 17: compare two paragraph files by identifier, reporting movement separately.

    A paragraph is unchanged, changed, deleted or inserted by exact text
    comparison (or whitespace-normalized with whitespace=True). The changed
    count includes every paragraph that is not unchanged. Word-level marks are
    computed on runs of letters, digits or underscores, punctuation and whitespace runs.
    Status can use normalized whitespace; visible marks always use the original texts.
    """
    left = folder / ("revised.txt" if reverse else "original.txt")
    right = folder / ("original.txt" if reverse else "revised.txt")
    a = read_paragraphs(left)
    b = read_paragraphs(right)
    aa = dict(a)
    bb = dict(b)
    ai = {k: i for i, (k, _) in enumerate(a)}
    bi = {k: i for i, (k, _) in enumerate(b)}
    rows = []
    diffs = []
    for key in list(aa) + [k for k in bb if k not in aa]:
        old = aa.get(key, "")
        new = bb.get(key, "")
        x = " ".join(old.split()) if whitespace else old
        y = " ".join(new.split()) if whitespace else new
        status = (
            "inserted"
            if key not in aa
            else "deleted"
            if key not in bb
            else "unchanged"
            if x == y
            else "changed"
        )
        moved = key in ai and key in bi and ai[key] != bi[key]
        rows.append(
            {
                "paragraph_id": key,
                "original_position": ai.get(key, -1) + 1,
                "revised_position": bi.get(key, -1) + 1,
                "status": status,
                "position_changed": int(moved),
                "original_text": old,
                "revised_text": new,
            }
        )
        at = re.findall(r"\s+|\w+|[^\w\s]", old)
        bt = re.findall(r"\s+|\w+|[^\w\s]", new)
        left_html = []
        right_html = []
        for tag, i, j, k, l in difflib.SequenceMatcher(
            None, at, bt, autojunk=False
        ).get_opcodes():
            ax = esc("".join(at[i:j]))
            bx = esc("".join(bt[k:l]))
            left_html.append("<del>" + ax + "</del>" if tag in {"delete", "replace"} else ax)
```

```
            right_html.append("<ins>" + bx + "</ins>" if tag in {"insert", "replace"} else bx
)
        diffs.append(
            "<h2>"
            + esc(key)
            + " - "
            + esc(status)
            + ("</h2><p>Paragraph position changed.</p>" if moved else "</h2>")
            + "<table><tr><th>Original</th><th>Revised</th></tr><tr><td>"
            + "".join(left_html)
            + "</td><td>"
            + "".join(right_html)
            + "</td></tr></table>"
        )
    result = Result(
        "Textual document comparison",
        {
            "paragraphs": len(rows),
            "changed": sum(r["status"] != "unchanged" for r in rows),
            "unchanged": sum(r["status"] == "unchanged" for r in rows),
            "position_changes": sum(r["position_changed"] for r in rows),
            "original_file": left.name,
            "revised_file": right.name,
            "original_sha256": sha(left),
            "revised_sha256": sha(right),
            "comparison": "whitespace-normalized" if whitespace else "exact paragraph text",
        },
        body="".join(diffs),
    )
    result.add(
        "comparison.csv",
        rows,
        [
            "paragraph_id",
            "original_position",
            "revised_position",
            "status",
            "position_changed",
            "original_text",
            "revised_text",
        ],
    )
    return result
```

Chapter 18

# Check Time Entries and Totals

**Row unit.** One time entry with minutes, rate, currency, and source.

**Reference entry point.** `programs/time_entry_check.py`

```
python3 programs/time_entry_check.py --out my_ch18
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch18/report.html`

**Outputs.** `ledger.csv`, `matter_totals.csv`, `report.html`, `run.json`, `summary.json`.

Line amounts use decimal `ROUND_HALF_UP`; the total is the sum of the individually rounded amounts. CAD 4.005 becomes CAD 4.01.

## Complete Small Inputs and Their Headers

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch18/matters.csv

```
matter_id,client_id,description
M01,C01,Agreement documents
M02,C01,Employment records
M03,C02,Correspondence review
```

### time_entries.csv

Exact columns, in order: `entry_id`, `matter_id`, `minutes`, `hourly_rate`, `currency`, `source_id`.

`entry_id`
Unique nonblank entry identifier; when a value occurs twice, every occurrence is held.

`matter_id`
Supplied matter key, which must be in `matters.csv`.

`minutes`
Whole number of minutes from 0 through 1,000,000 inclusive, written in digits; a blank, a fraction, or a negative value holds the entry, and a blank is missing, not zero. A larger whole number is outside the teaching range and is held.

`hourly_rate`
Finite decimal rate from 0 through 1,000,000 inclusive, with at most four fractional decimal places after trailing fractional zeros are ignored; outside-range or overprecise values are held. This is an arithmetic input, not approval of a charge.

`currency`
Exactly `CAD`; the teaching worksheet accepts nothing else and never mixes currencies in one total.

`source_id`
Supplied source reference, required to be nonblank.

data/ch18/time_entries.csv

```
entry_id,matter_id,minutes,hourly_rate,currency,source_id
T01,M01,30,240,CAD,TIME01
T02,M01,45,240,CAD,TIME02
T03,M02,15,300,CAD,TIME03
T04,M02,,300,CAD,TIME04
T05,M03,60,200,CAD,TIME05
T06,M03,-10,200,CAD,TIME06
```

## One Controlled Experiment

Add T07 for one minute at CAD 240.30 per hour. The half-up line amount is CAD 4.01. Added to the baseline's CAD 575.00, the calculable total is CAD 579.01. The accepted count becomes five while the two original held records remain unresolved.

Two such boundary lines total CAD 8.02 because this exercise sums individually rounded lines. Rounding their combined unrounded amount instead would give CAD 8.01. The difference is intentional and testable. State the convention beside the worksheet so a reader can reproduce the result without guessing when rounding occurred.

```
python3 tools/run_experiments.py 18 --out my_experiment_18
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/time_entry_check.py

```
"""Chapter 18: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(18)
```

The function `time_entries` uses a local 40-digit decimal context to compute minutes times rate divided by sixty, rounds each line half up, and totals the rounded lines. It holds unsupported inputs with `minutes_outside_teaching_range`, `rate_outside_teaching_range`, or `rate_precision_exceeds_four_places`; malformed, negative, and nonfinite values keep their invalid-value reasons. The bounds above are part of this teaching application's contract, not a general limit on Python decimal arithmetic.

programs/legal_apps.py, lines 1425–1521

```
# Chapter 18: Decimal, half-up per line; sum those rounded lines.
def time_entries(folder: Path, **_) -> Result:
    """Chapter 18: minutes x rate / 60 in Decimal, rounded half up per line; total of rounded
  lines.

    An entry is held, with every reason, for a blank or duplicated identifier,
    an unregistered matter, a blank source, minutes that are not a whole number
    from 0 through 1,000,000 written in digits, a rate outside 0 through
    1,000,000 or with more than four fractional places (ignoring trailing
    fractional zeros), or a currency other than CAD. Rates must be finite
    and nonnegative. Calculation uses a 40-digit local Decimal context.
    A held entry has no amount.
    """
```

```
    fields = ["entry_id", "matter_id", "minutes", "hourly_rate", "currency", "source_id"]
    rows = load(folder, "time_entries.csv", fields)
    known = registry(folder)
    issues = identity_issues(rows, "entry_id")
    ledger = []
    total = Decimal("0.00")
    subtotals = defaultdict(lambda: Decimal("0.00"))
    for i, r in enumerate(rows):
        e = issues[i]
        # Compare digit strings before int(): even a very long malformed
        # duration becomes a held row, without Python's integer-digit limit.
        minute_digits = r["minutes"].lstrip("0") or "0"
        n = None
        if not re.fullmatch(r"[0-9]+", r["minutes"]):
            e.append("invalid_minutes")
        elif len(minute_digits) > 7 or (
            len(minute_digits) == 7 and minute_digits > "1000000"
        ):
            e.append("minutes_outside_teaching_range")
        else:
            n = int(minute_digits)
        require_cells(r, ["source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if r["currency"] != "CAD":
            e.append("wrong_currency")
        try:
            rate = Decimal(r["hourly_rate"])
            if not rate.is_finite() or rate < 0:
                raise InvalidOperation
            if rate > Decimal("1000000"):
                e.append("rate_outside_teaching_range")
            elif rate.is_zero():
                rate = Decimal(0)  # Includes signed zero and large zero exponents.
            else:
                digits = rate.as_tuple().digits
                trailing_zeros = 0
                for digit in reversed(digits):
                    if digit != 0:
                        break
                    trailing_zeros += 1
                fractional_places = max(0, -rate.as_tuple().exponent - trailing_zeros)
                if fractional_places > 4:
                    e.append("rate_precision_exceeds_four_places")
        except InvalidOperation:
            e.append("invalid_rate")
            rate = None
        amount = ""
        if not e:
            # The stated magnitude/scale bounds leave ample guard digits,
            # independent of any Decimal context set by a caller.
            with localcontext(Context(prec=40)):
                value = (Decimal(n) * rate / Decimal(60)).quantize(
                    Decimal(".01"), rounding=ROUND_HALF_UP
                )
                total += value
                subtotals[r["matter_id"]] += value
            amount = f"{value:.2f}"
        ledger.append(
            dict(
```

```
            r,
            source_row=i + 2,
            amount=amount,
            status="held" if e else "calculable",
            issues=";".join(e),
        )
    )
result = Result(
    "Draft time-entry arithmetic worksheet",
    {
        "input_rows": len(rows),
        "accepted": sum(r["status"] == "calculable" for r in ledger),
        "held": sum(r["status"] == "held" for r in ledger),
        "total": f"{total:.2f}",
        "currency": "CAD",
        "rounding": "ROUND_HALF_UP per line; total is sum of rounded lines",
    },
)
result.add("ledger.csv", ledger, fields + ["source_row", "amount", "status", "issues"])
result.add(
    "matter_totals.csv",
    [{"matter_id": m, "amount": f"{v:.2f}"} for m, v in sorted(subtotals.items())],
    ["matter_id", "amount"],
)
return result
```

Chapter 19

# Review Files Recorded as Closed

**Row unit.** One file-closing register row; review history is separate.
**Reference entry point.** `programs/closing_review.py`

```
python3 programs/closing_review.py --out my_ch19
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch19/report.html`
**Outputs.** `closing_review.csv`, `history.csv`, `report.html`, `run.json`, `summary.json`.

## Complete Small Inputs and Their Headers

### closing.csv

Exact columns, in order: `file_id`, `matter_id`, `closure_date`, `review_date`, `hold`, `owner`, `source_id`.

`file_id`
Unique nonblank file identifier; a blank or repeated value sends the entry to review.

`matter_id`
Supplied matter key, which must be in the matter register.

`closure_date`
Supplied closure date; a missing or invalid date, or one later than the reporting date, sends the entry to review, and it is never replaced by the review date.

`review_date`
Supplied review date, never calculated; on or before the reporting date is due for review, later is upcoming, and missing, invalid, or earlier than the closure date is review.

`hold`
Exactly `yes` or `no`; `yes` makes the entry a hold before anything else, and any other value sends it to a person.

`owner`
Supplied owner label, required to be nonblank and never inferred.

`source_id`
Supplied source reference, required to be nonblank; not authentication.

data/ch19/closing.csv

```
file_id,matter_id,closure_date,review_date,hold,owner,source_id
F01,M01,2026-08-01,2026-09-20,no,A,CLOSING01
F02,M02,2026-08-01,2026-10-01,no,B,CLOSING02
F03,M03,2026-08-01,2026-09-01,yes,A,CLOSING03
F04,M01,2026-08-01,,no,B,CLOSING04
F05,M02,,2026-09-10,no,A,CLOSING05
```

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch19/matters.csv

```
matter_id,client_id,description
M01,C01,Agreement documents
M02,C01,Employment records
M03,C02,Correspondence review
```

### review__history.csv

Exact columns, in order: `history_id`, `file_id`, `review_date`, `reviewer`, `source_version`, `outcome`, `source_id`.

`history_id`
Unique nonblank identifier of one recorded review; a duplicate stops the run.

`file_id`
File the review concerns; the history is reported beside the register and does not classify files.

`review_date`
Date of the recorded review, which must be a valid date.

`reviewer`
Supplied reviewer label, required to be nonblank.

`source_version`
Version of the source record the review examined, required to be nonblank.

`outcome`
Supplied outcome text, such as a request for another administrative review; it authorizes no disposition.

`source_id`
Supplied source reference, carried through; not authentication.

data/ch19/review_history.csv

```
history_id,file_id,review_date,reviewer,source_version,outcome,source_id
H01,F01,2026-09-20,Reviewer Example,1,request administrative review,HISTORY01
```

## One Controlled Experiment

For F01, supply a new review date of October 15 with source CLOSING01-REV2 and append history entry H02. Its recorded outcome asks for another administrative review; it authorizes no disposition. Keep H01 unchanged.

At the same September 24 reporting date, the due-for-review count falls to zero and upcoming becomes two. The hold and two incomplete records remain. The history output contains H01 and H02. This is a controlled, two-table revision: the register shows the current supplied value and the history explains the transition. A one-table overwrite would not preserve both functions.

```
python3 tools/run_experiments.py 19 --out my_experiment_19
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/closing_review.py

```
"""Chapter 19: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(19)
```

The function `closing` classifies closed files in a fixed order: hold, then needs review, then the date comparison.

programs/legal_apps.py, lines 1524–1602

```
# Chapter 19: hold precedence; history is retained, not an instruction to delete.
def closing(folder: Path, as_of: str = "2026-09-24", **_) -> Result:
    """Chapter 19: classify closed files in a fixed order: hold, then needs_review, then dates.


    A hold flag of yes always wins. Otherwise an unrecognised flag, a blank or
    repeated file ID, an unregistered matter, a blank owner or source, a
    closure date missing, invalid or after the reporting date, or a review
    date missing, invalid or before the closure date sends the entry to a
    person. Otherwise a review date on or before the reporting date is due and
    a later one is upcoming. History rows are checked and returned unchanged.
    """
    fields = [
        "file_id",
        "matter_id",
        "closure_date",
        "review_date",
        "hold",
        "owner",
        "source_id",
    ]
    rows = load(folder, "closing.csv", fields)
    known = registry(folder)
    cutoff = required_day(as_of)
    issues = identity_issues(rows, "file_id")
    ledger = []
    for i, r in enumerate(rows):
        e = issues[i]
        c = day(r["closure_date"])
        d = day(r["review_date"])
        require_cells(r, ["owner", "source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if r["hold"] not in {"yes", "no"}:
            e.append("unknown_hold_flag")
        if c is None or c > cutoff:
            e.append("invalid_closure_date")
        if d is None or c is not None and d < c:
            e.append("invalid_review_date")
        status = (
            "hold"
            if r["hold"] == "yes"
```

```
            else "needs_review"
            if e
            else "due_for_review"
            if d <= cutoff
            else "upcoming"
        )
        ledger.append(dict(r, classification=status, issues=";".join(e)))
    hfields = [
        "history_id",
        "file_id",
        "review_date",
        "reviewer",
        "source_version",
        "outcome",
        "source_id",
    ]
    history = load(folder, "review_history.csv", hfields)
    if any(identity_issues(history, "history_id")):
        raise ValueError("Duplicate or missing history identity")
    for r in history:
        if (
            day(r["review_date"]) is None
            or not r["reviewer"].strip()
            or not r["source_version"].strip()
        ):
            raise ValueError("Incomplete review-history record")
    counts = Counter(r["classification"] for r in ledger)
    result = Result(
        "File-closing administrative review",
        dict(
            input_rows=len(rows),
            reporting_date=as_of,
            **{s: counts[s] for s in ["hold", "needs_review", "due_for_review", "upcoming"]},
        ),
    )
    result.add("closing_review.csv", ledger, fields + ["classification", "issues"])
    result.add("history.csv", history, hfields)
    return result
```

Chapter 20

# Test the Applications

**Row unit.** One declared test case and its expected behaviour.

**Reference entry point.** `programs/test_legal_apps.py`

```
python3 -m unittest discover -s programs -p "test_*.py" -v
```

The complete test listings follow in the appendices. A deliberately failing mutation is created only in a temporary copy.

## Files to Inspect

The saved release log is `verification/regression_tests.txt`; the command above prints a fresh result and does not update it. To keep your run, save its complete output under a new name, such as `my_regression_tests.txt`, note the interpreter version beside it, and preserve the supplied log for comparison.

## Complete Small Inputs and Their Headers

Tests read the canonical fixture folders and hand-derived expectations; the complete baseline expectation file is printed in the verification appendix.

## One Controlled Experiment

The executable chapter experiment copies the programs and inputs, changes the inclusive upper endpoint to an exclusive one, and runs the endpoint test against the faulty copy. The saved test result is a failure: the expected outcome of this experiment, not a failed release test. The baseline implementation remains unchanged, and its test suite is run separately after the experiment; a passing baseline suite and a deliberately failing mutant establish different facts. A second saved mutation changes half-up rounding to half-even and is caught by the half-cent boundary test; a third changes the window-open comparison. Exercise 20.3 in the main book works through the saved failure logs of all three. Compare the observed result with the prediction in the main book and with the saved `experiment.json`; the helper never changes the canonical data folder.

```
python3 tools/run_experiments.py 20 --out my_experiment_20
```

Chapter 21

# Package a Reproducible Demonstration

**Row unit.** One allowlisted archive member.

**Reference entry point.** `programs/make_demo_package.py`

```
python3 programs/make_demo_package.py --root . --out my_matter_demo.zip
```

Inspect the actual member list and manifest. Extract to a new folder and run its opening tests. The small demo contains the opening application, not all later chapters.

## Files to Inspect

The saved baseline is `Matter_Demo.zip`. The command above creates the new archive `my_matter_demo.zip`; another `--out` filename creates an archive under that name. Inspect the new archive rather than mistaking the unchanged supplied baseline for your fresh result.

## Complete Small Inputs and Their Headers

### README_DEMO.txt

README_DEMO.txt

```
VIBE CODING FOR LEGAL PROFESSIONALS -- OPENING PROJECT
Synthetic teaching data only. Not approved for real legal-office use.

Requires Python 3.10 or later; standard library only.
Open a terminal in the extracted directory. On systems where the Python
command is python rather than python3, substitute that command.

Run the small example to a new folder:
python3 programs/matter_run.py --input data/matter_events_demo.csv --matters data/
  matters_demo.csv --out first_run

Open first_run/report.html in a browser. Expected: 18 accepted, 0 held.
Choose M01. Expected events: E01, E02, E03.

Run the troublesome example:
```

```
python3 programs/matter_check.py --input data/matter_events_problem.csv --matters data/
  matters_demo.csv --out problem_run
Expected: 11 accepted, 7 held. Inspect review.csv and run.json.

Run the large input, returning only one matter's events:
python3 programs/matter_run.py --input data/matter_events_large.csv --matters data/
  matters_large.csv --out large_m0001 --matter M0001
Expected: 12,000 accepted overall; 4 selected events, E00001--E00004.
Whole-file validation precedes selection. The run record contains internal
whole-file counts and is not automatically suitable for external delivery.

Run the regression tests:
python3 programs/test_matter_tools.py

The three entry points use the SAME shared matter_tools.py implementation.
Keep that file beside them. No inputs are modified. Existing output folders
are not overwritten. A failed partial output folder is not a completed run;
inspect all four expected output files and use a new destination after diagnosis.
No messages are sent, no client systems are contacted, and no legal dates are
calculated. Browser filtering does not remove other data from an HTML file;
use a separately selected run when a single-matter file is required.

Interpreter note: commands use python3. On Windows, use python or py if that is the installed
   interpreter command.
```

The explicit member list in the source below, rather than a recursive copy of the working directory, defines the demonstration payload.

## One Controlled Experiment

Add an unrelated invented note to a copy of the package source folder, then run the unchanged maker. The demonstration still has twelve archive members. The unrelated note is absent because it was never on the allowlist.

Open the archive member list before extracting it. After extraction, compare every payload member's size and hash with the manifest, then run the included opening tests using only the extracted files. The full release's fresh-extraction check is separate and also exercises the later applications. Do not confuse a small demonstration's twelve members with the full book package.

```
python3 tools/run_experiments.py 21 --out my_experiment_21
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

This entry point is the complete program; it does not use `legal_apps.py`.

programs/make_demo_package.py

```
"""Build an allowlisted synthetic demonstration; no network or input mutation."""

from __future__ import annotations
import argparse
import hashlib
import json
from pathlib import Path
from zipfile import ZipFile, ZIP_DEFLATED

MEMBERS = [
    "programs/matter_tools.py",
    "programs/matter_view.py",
    "programs/matter_check.py",
    "programs/matter_run.py",
    "programs/test_matter_tools.py",
    "data/matters_demo.csv",
    "data/matter_events_demo.csv",
    "data/matter_events_problem.csv",
    "data/matters_large.csv",
    "data/matter_events_large.csv",
    "README_DEMO.txt",
]


def make(root: Path, output: Path) -> None:
    payload = []
    for name in MEMBERS:
        path = root / name
        if not path.is_file() or path.is_symlink():
            raise ValueError(f"Missing or symlinked required file: {name}")
        content = path.read_bytes()
        payload.append((name, content))
    manifest = {
        "purpose": "Synthetic teaching demonstration, not a production legal tool",
        "members": [
            {"path": name, "bytes": len(content), "sha256": hashlib.sha256(content).hexdigest
  ()}
            for name, content in payload
        ],
    }
    # Exclusive mode refuses to overwrite an existing destination.
    with ZipFile(output, "x", compression=ZIP_DEFLATED) as archive:
        for name, content in payload:
            archive.writestr(name, content)
        archive.writestr("MANIFEST.json", json.dumps(manifest, indent=2, sort_keys=True) + "\
  n")


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--root", required=True, type=Path)
    parser.add_argument("--out", required=True, type=Path)
    args = parser.parse_args()
```

```
    try:
        make(args.root, args.out)
    except (OSError, ValueError) as exc:
        parser.exit(2, str(exc) + "\n")
    print(f"Created {args.out.name}: {len(MEMBERS) + 1} members")


if __name__ == "__main__":
    main()
```

Chapter 22

# Check Application-Review Records

**Row unit.** One recorded review of a required category.
**Reference entry point.** `programs/application_review_register.py`

```
python3 programs/application_review_register.py --out my_ch22
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch22/report.html`
**Outputs.** `report.html`, `review_register.csv`, `run.json`, `summary.json`.

## Complete Small Inputs and Their Headers

### reviews.csv

Exact columns, in order: `category`, `artifact_version`, `rule_version`, `reviewer`, `review_date`, `expiry_date`, `source_id`.

`category`
One of `specification`, `program`, `tests`, `data handling`, or `user guide`; each must appear exactly once, and an absent category is added with `missing_category`.

`artifact_version`
Version of the application that was reviewed; it must equal the declared version, 2 by default.

`rule_version`
Version of the rules the review applied; it must equal the declared version, 1 by default.

`reviewer`
Supplied reviewer label, required to be nonblank.

`review_date`
Valid date of the review, on or before the reporting date.

`expiry_date`
Inclusive last date on which the review is current; it must be on or after the reporting date.

`source_id`
Supplied source reference, required to be nonblank; not authentication.

data/ch22/reviews.csv

```
category,artifact_version,rule_version,reviewer,review_date,expiry_date,source_id
specification,2,1,Reviewer Example,2026-09-20,2026-12-31,REVIEW-1
program,1,1,Reviewer Example,2026-09-20,2026-12-31,REVIEW-2
tests,2,1,Reviewer Example,2026-09-20,2026-12-31,REVIEW-3
data handling,2,1,,2026-09-20,2026-12-31,REVIEW-4
user guide,2,1,Reviewer Example,2026-09-20,2026-09-24,REVIEW-5
```

## One Controlled Experiment

Supply Reviewer Example in the synthetic data-handling record. Four of the five review records now meet the declared checks. The program entry still names artifact version 1 rather than 2 and remains an attention item.

This change demonstrates why a single green approval badge is inadequate. Completing one field does not repair another category's version mismatch. The output retains all categories, and the action list explains each unresolved condition. If a required category is absent, the program adds a missing-category result rather than quietly reducing the number of expected reviews.

```
python3 tools/run_experiments.py 22 --out my_experiment_22
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/application_review_register.py

```
"""Chapter 22: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(22)
```

The function `reviews` asks whether each required review record is complete for the declared delivery.

programs/legal_apps.py, lines 1605–1675

```
# Chapter 22: exactly one required category, explicit versions and dates.
def reviews(
    folder: Path, as_of: str = "2026-09-24", artifact: str = "2", rule: str = "1", **_
) -> Result:
    """Chapter 22: is each required review record complete for the declared delivery?

    Exactly one row per required category; reviewer and source present;
    artifact and rule versions equal to the declared ones; valid review date
    on or before the reporting date and expiry date on or after it (expiry is
    inclusive). A missing category is added as a row with missing_category.
    """
    fields = [
        "category",
        "artifact_version",
        "rule_version",
        "reviewer",
        "review_date",
        "expiry_date",
        "source_id",
    ]
    rows = load(folder, "reviews.csv", fields)
    required = ["specification", "program", "tests", "data handling", "user guide"]
    cutoff = required_day(as_of)
    counts = Counter(r["category"] for r in rows)
    ledger = []
    for i, r in enumerate(rows):
        e = []
        require_cells(r, ["reviewer", "source_id"], e)
        d = day(r["review_date"])
        x = day(r["expiry_date"])
        if r["category"] not in required:
            e.append("unknown_category")
        if counts[r["category"]] != 1:
            e.append("duplicate_category")
        if r["artifact_version"] != artifact:
            e.append("artifact_version_mismatch")
        if r["rule_version"] != rule:
            e.append("rule_version_mismatch")
        if d is None or x is None or not d <= cutoff <= x:
            e.append("invalid_or_outdated_dates")
        ledger.append(
            dict(
                r,
                source_row=i + 2,
                status="needs_attention" if e else "review_record_complete",
                issues=";".join(e),
            )
        )
    for category in required:
        if counts[category] == 0:
            ledger.append(
                dict.fromkeys(fields, "")
                | {
                    "category": category,
                    "source_row": "",
                    "status": "needs_attention",
                    "issues": "missing_category",
                    }
```

```
        )
    result = Result(
        "Application-review record check",
        {
            "complete": sum(r["status"] == "review_record_complete" for r in ledger),
            "needs_attention": sum(r["status"] != "review_record_complete" for r in ledger),
            "artifact_version": artifact,
            "rule_version": rule,
            "reporting_date": as_of,
        },
    )
    result.add("review_register.csv", ledger, fields + ["source_row", "status", "issues"])
    return result
```

Chapter 23

# Build an Integrated Matter Worklist

**Row unit.** One request, joined to its response candidates and supplied owner.
**Reference entry point.** `programs/matter_worklist.py`

```
python3 programs/matter_worklist.py --out my_ch23
```

## Files to Inspect

**Saved baseline.** `reference_outputs/ch23/report.html`
**Outputs.** `candidates.csv`, `internal_validation.json`, `report.html`, `run.json`, `summary.json`, `worklist.csv`.

Use `--window 21` to change the inclusive window, `--as-of 2026-09-25` to change the historical date, and, for the worklist, `--matter M01` for a genuinely selected export. Future requests and responses remain explicitly distinguished from observed records.

## Complete Small Inputs and Their Headers

### matters.csv

Exact columns, in order: `matter_id`, `client_id`, `description`.

`matter_id`
Unique matter key; the register must support an unambiguous lookup, or the run stops.

`client_id`
Designated synthetic client of the matter, exactly one per matter in this teaching convention; not a substitute for the matter key.

`description`
Short supplied description of the matter; it is required as a column but takes no part in any check.

data/ch23/matters.csv

```
matter_id,client_id,description
M01,C01,Agreement documents
M02,C01,Employment records
M03,C02,Correspondence review
```

### requests.csv

Exact columns, in order: `request_id`, `matter_id`, `owner`, `request_date`, `source_id`.

`request_id`
Request label; with matter_id it forms the key a response must match, and a repeated key is flagged on every occurrence.

`matter_id`
Supplied matter key, checked against the register; a response must agree on it as well as on the request label.

`owner`
Supplied owner of the request, copied into the worklist and used by the owner filter; never inferred from a name or document.

`request_date`
Supplied ISO date of the request; a date after the reporting date makes the request not yet requested, and a blank or invalid date sends it to review.

`source_id`
Supplied source reference, required to be nonblank; not authentication.

data/ch23/requests.csv

```
request_id,matter_id,owner,request_date,source_id
Q01,M01,A,2026-09-10,CQ01
Q02,M01,B,2026-09-20,CQ02
Q03,M02,A,2026-09-01,CQ03
Q04,M02,B,,CQ04
Q05,M03,A,2026-09-10,CQ05
Q06,M03,B,2026-09-01,CQ06
```

### responses.csv

Exact columns, in order: `response_id`, `matter_id`, `request_id`, `response_date`, `source_id`.

`response_id`
Unique identifier of one response; a repeated value is flagged on every occurrence.

`matter_id`
Matter the response names; it must agree with the request's matter for the response to link.

`request_id`
Request label the response names; with matter_id it must match a request, or the response is unmatched.

`response_date`
Supplied ISO date of the response; it decides the elapsed days, the reporting-date cutoff, and the window test.

`source_id`
Supplied source reference; a nonblank value is not authentication.

data/ch23/responses.csv

```
response_id,matter_id,request_id,response_date,source_id
CS01,M01,Q01,2026-09-12,CR01
CS06,M03,Q06,2026-09-18,CR06
```

## One Controlled Experiment

Move Q02's request date to September 25 while retaining a September 24 report. Q02 becomes not yet requested; it does not enter the open-window group simply because a negative elapsed time is less than fourteen. Q05 remains the one open-window request, and Q01 and Q06 still have recorded responses.

The retained input copy, change description, result, and independently stated expectations are saved together. Reverse that one change and rerun to another folder. Q02 returns to window open. The comparison is small enough to inspect request by request and specific enough to catch the original boundary omission.

```
python3 tools/run_experiments.py 23 --out my_experiment_23
```

Compare the observed result with the prediction in the main book and with the saved `experiment.json`. The helper never changes the canonical data folder.

## Entry Point and Chapter Function

programs/matter_worklist.py

```
"""Chapter 23: canonical entry point; logic in legal_apps.py."""

from run_chapter import main

if __name__ == "__main__":
    main(23)
```

The function `worklist` assigns one status per request and keeps three evidence fields beside it, using the Chapter 8 matcher. All the rules in Chapter 8's *Input Validity and Evidence Fields* (page 35) apply unchanged, including the supplied owner requirement, the registered-matter check, and the distinction between review status and retained usable evidence. The function `match_records` that it calls is printed in Chapter 8.

programs/legal_apps.py, lines 1678–1762

```
# Chapter 23: integrate the matcher without changing its definitions.
def worklist(
    folder: Path, as_of: str = "2026-09-24", window: int = 14, matter: str | None = None, **_
) -> Result:
    """Chapter 23: one status per request plus three evidence fields, using the Chapter 8
  matcher.

    Status order: needs_review, not_yet_requested, response_recorded,
    window_open, window_passed. The owner is copied from the request row.
    With a selected matter the whole file is matched first and the delivered
    tables are then restricted to that matter.
    """
    req = load(folder, "requests.csv", REQUEST_FIELDS)
    resp = load(folder, "responses.csv", RESPONSE_FIELDS)
    known = registry(folder)
    if matter is not None and matter not in known:
        raise ValueError("Unknown selected matter")
    rows, candidates = match_records(req, resp, known, required_day(as_of), window)
    whole = Counter(r["status"] for r in rows)
    if matter is not None:
        rows = [r for r in rows if r["matter_id"] == matter]
        candidates = [r for r in candidates if r["matter_id"] == matter]
    counts = Counter(r["status"] for r in rows)
    summary = {
        "requests": len(rows),
        "reporting_date": as_of,
        "window_days": window,
        **{
            s: counts[s]
            for s in [
                "response_recorded",
                "window_open",
                "window_passed",
                "needs_review",
                "not_yet_requested",
            ]
```

```
      },
      "in_window_yes": sum(r["in_window"] == "yes" for r in rows),
      "ambiguous_earliest": sum(r["earliest"] == "ambiguous" for r in rows),
  }
  result = Result("Integrated matter worklist", summary)
  result.add("worklist.csv", rows, MATCH_FIELDS)
  result.add("candidates.csv", candidates, CANDIDATE_FIELDS)
  # Whole-file counts are an internal result, not part of the selected report.
  result.extra["internal_validation.json"] = (
      json.dumps({"whole_requests": len(req), "whole_status_counts": dict(whole)}, indent
=2)
      + "\n"
  )
  owners = sorted({r["owner"] for r in rows})
  statuses = sorted(counts)
  controls = (
      '<p>These filters hide rows; they do not remove data from this delivered file.</p><
label>Owner <select id="owner"><option value="">All</option>'
      + "".join("<option>" + esc(x) + "</option>" for x in owners)
      + '</select></label> <label>Status <select id="status"><option value="">All</option>'
      + "".join("<option>" + esc(x) + "</option>" for x in statuses)
      + '</select></label><p id="visible"></p>'
  )
  display_fields = [
      "request_id",
      "matter_id",
      "owner",
      "status",
      "any_response",
      "in_window",
      "earliest",
      "selected_response",
  ]
  table = (
      '<table id="work"><thead><tr>'
      + "".join("<th>" + esc(x.replace("_", " ")) + "</th>" for x in display_fields)
      + "</tr></thead><tbody>"
      + "".join(
          '<tr data-owner="'
          + esc(r["owner"])
          + '" data-status="'
          + esc(r["status"])
          + '">'
          + "".join("<td>" + esc(r[f]) + "</td>" for f in display_fields)
          + "</tr>"
          for r in rows
      )
      + "</tbody></table>"
  )
  script = """<script>const owner=document.getElementById('owner'),status=document.
getElementById('status');function filter(){let n=0;document.querySelectorAll('#work tbody tr
').forEach(r=>{r.hidden=!!((owner.value&&r.dataset.owner!==owner.value)||(status.value&&r.
dataset.status!==status.value));if(!r.hidden)n++;});document.getElementById('visible').
textContent=n?n+' matching requests':'No matching rows';}owner.addEventListener('change',
filter);status.addEventListener('change',filter);filter();</script>"""
  result.body = controls + table + script
  return result
```

## The Large Conversion and Integration

From a new working destination, first create the request and response registers:

```
python3 programs/convert_matter_events.py --events
data/matter_events_large.csv --matters data/matters_large.csv --assignments
data/assignments_large.csv --out my_conversion
```

Then run the worklist on the conversion:

```
python3 programs/matter_worklist.py --data my_conversion --as-of 2026-12-31
--out my_large_worklist
```

For a selected deliverable, add `--matter M0001` and choose a different new destination. The supplied assignment table has one owner for every matter. Missing assignments stop the conversion; no owner is guessed. The source ledger retains a disposition for all 12,000 events.

Exact columns of `data/assignments_large.csv`, in order: `matter_id`, `owner`. The file has 3,000 data rows; its header and first four rows are printed in the main book.

`matter_id`
Registered matter key; the table must give exactly one nonblank owner for every registered matter, or the conversion stops.

`owner`
Owner assigned to every request of the matter; it is copied into the request register and never guessed.

Appendix A

# The Shared Opening Implementation

programs/matter_tools.py

```
"""Synthetic teaching application, not a production legal-record system.
Python standard library only. Inputs are never modified. No network access.
"""

from __future__ import annotations
import argparse
import csv
import hashlib
import html
import json
import re
from collections import Counter
from datetime import date
from pathlib import Path

VERSION = "legal-copyedit-2026-10-24.2"
EVENT_FIELDS = [
    "event_id",
    "client_id",
    "matter_id",
    "event_date",
    "event_type",
    "request_id",
    "source_id",
    "summary",
]
MATTER_FIELDS = ["matter_id", "client_id", "description"]


def load_csv(path: Path, expected: list[str]) -> list[dict[str, str]]:
    with path.open("r", encoding="utf-8-sig", newline="") as stream:
        reader = csv.DictReader(stream, strict=True)
        if reader.fieldnames != expected:
            raise ValueError(
                f"{path.name}: expected columns {expected}; found {reader.fieldnames}"
            )
        try:
            rows = list(reader)
        except csv.Error as error:
            raise ValueError(f"Malformed CSV: {error}") from error
    for number, row in enumerate(rows, 2):
        if None in row or any(value is None for value in row.values()):
```

```
            raise ValueError(f"{path.name}: malformed CSV record at record position {number}"
  )
    return rows


def valid_date(value: str) -> bool:
    if not re.fullmatch(r"\d{4}-\d{2}-\d{2}", value):
        return False
    try:
        date.fromisoformat(value)
    except ValueError:
        return False
    return True


def check(
    events: list[dict[str, str]], matters: list[dict[str, str]]
) -> tuple[list[dict], list[dict]]:
    ids = [row["matter_id"] for row in matters]
    if any(
        not row["matter_id"].strip() or not row["client_id"].strip() for row in matters
    ) or len(set(ids)) != len(ids):
        raise ValueError(
            "Matter register needs nonblank matter/client IDs and unique matter IDs"
        )
    registry = {row["matter_id"]: row["client_id"] for row in matters}
    counts = Counter(row["event_id"] for row in events)
    reasons: list[list[str]] = [[] for _ in events]
    for i, row in enumerate(events):
        for field in ["event_id", "client_id", "matter_id", "source_id", "summary"]:
            if not row[field].strip():
                reasons[i].append("missing_" + field)
        if row["event_id"] and counts[row["event_id"]] > 1:
            reasons[i].append("duplicate_event_id")
        if not valid_date(row["event_date"]):
            reasons[i].append("invalid_or_missing_date")
        if row["matter_id"] not in registry:
            reasons[i].append("unknown_matter")
        elif registry[row["matter_id"]] != row["client_id"]:
            reasons[i].append("client_matter_mismatch")
        if row["event_type"] not in {"request", "response", "note"}:
            reasons[i].append("unknown_event_type")
        if row["event_type"] in {"request", "response"} and not row["request_id"].strip():
            reasons[i].append("missing_request_id")
        if row["event_type"] == "note" and row["request_id"]:
            reasons[i].append("note_has_request_id")
    request_positions: dict[tuple[str, str], list[int]] = {}
    for i, row in enumerate(events):
        if row["event_type"] == "request":
            request_positions.setdefault((row["matter_id"], row["request_id"]), []).append(i)
    for positions in request_positions.values():
        if len(positions) > 1:
            for i in positions:
                reasons[i].append("duplicate_request_key")
    for i, row in enumerate(events):
        if row["event_type"] != "response":
            continue
        positions = request_positions.get((row["matter_id"], row["request_id"]), [])
        if len(positions) != 1:
            reasons[i].append("missing_or_ambiguous_request")
```

```
            continue
        j = positions[0]
        if reasons[j]:
            reasons[i].append("request_needs_review")
        elif valid_date(row["event_date"]) and row["event_date"] < events[j]["event_date"]:
            reasons[i].append("response_before_request")
    accepted, held = [], []
    for i, row in enumerate(events):
        enriched = dict(row, source_row=i + 2)
        if reasons[i]:
            held.append(dict(enriched, reasons=";".join(reasons[i])))
        else:
            accepted.append(enriched)
    accepted.sort(key=lambda row: (row["matter_id"], row["event_date"], row["event_id"]))
    return accepted, held


def write_csv(path: Path, rows: list[dict], fields: list[str]) -> None:
    with path.open("w", encoding="utf-8", newline="") as stream:
        writer = csv.DictWriter(stream, fieldnames=fields, lineterminator="\n")
        writer.writeheader()
        writer.writerows(rows)


def browser_page(accepted: list[dict], held: list[dict], matter_ids: list[str]) -> str:
    data = (
        json.dumps(accepted, ensure_ascii=True)
        .replace("<", "\\u003c")
        .replace(">", "\\u003e")
        .replace("&", "\\u0026")
    )
    options = "".join(f"<option>{html.escape(m)}</option>" for m in matter_ids)
    if not options:
        options = '<option value="">No matters</option>'

    def table(rows: list[dict], fields: list[str]) -> str:
        headings = "".join("<th>" + html.escape(f) + "</th>" for f in fields)
        body = "".join(
            '<tr data-matter="'
            + html.escape(row["matter_id"], quote=True)
            + '">'
            + "".join("<td>" + html.escape(str(row[f])) + "</td>" for f in fields)
            + "</tr>"
            for row in rows
        )
        return (
            "<table><thead><tr>" + headings + "</tr></thead><tbody>" + body + "</tbody></
  table>"
        )

    return (
        """<!doctype html><html lang="en"><meta charset="utf-8"><title>Synthetic matter
  timeline</title>
<style>body{font:18px system-ui;max-width:1200px;margin:2em auto;padding:0 1em}table{border-
  collapse:collapse;width:100%;font-size:14px}td,th{border:1px solid #bbb;padding:8px;text-
  align:left}svg{width:100%;border:1px solid #bbb}button,select{font:inherit;padding:.3em}.
  scroll{overflow:auto}#detail{padding:1em;border:1px solid #888;white-space:pre-wrap}circle{
  cursor:pointer}caption{text-align:left}</style>
<h1>Matter timeline</h1><p>SYNTHETIC TEACHING DATA. A recorded event is not an established
  legal fact.</p>
```

```
<label>Matter <select id="matter">"""
        + options
        + """</select></label>
<p id="count"></p><svg id="timeline" role="img" aria-label="Recorded events on a timeline"></
  svg>
<p>Select an event marker or use the event buttons below it.</p><div id="buttons"></div><p id
  ="detail">No event selected.</p>
<h2>Accepted records</h2><div class="scroll">"""
        + table(accepted, EVENT_FIELDS + ["source_row"])
        + """</div>
<h2>Records needing review</h2><p id="reviewcount"></p><div class="scroll">"""
        + table(held, EVENT_FIELDS + ["source_row", "reasons"])
        + """</div>
<script>const records="""
        + data
        + """;
const selector=document.getElementById('matter'),svg=document.getElementById('timeline');
function element(tag,attrs){const e=document.createElementNS('http://www.w3.org/2000/svg',tag)
  ;for(const [k,v] of Object.entries(attrs))e.setAttribute(k,v);return e;}
function show(r){document.getElementById('detail').textContent=Object.entries(r).map(([k,v])=>
  k+': '+v).join('\\n');}
function draw(){const rows=records.filter(r=>r.matter_id===selector.value);svg.replaceChildren
  ();const b=document.getElementById('buttons');b.replaceChildren();document.getElementById('
  detail').textContent='No event selected.';document.getElementById('count').textContent=rows.
  length+' accepted records for '+selector.value;
const height=Math.max(100,rows.length*44+50);svg.setAttribute('viewBox','0 0 1000 '+height);
const times=records.map(r=>Date.parse(r.event_date+'T00:00:00Z')),lo=Math.min(...times),hi=
  Math.max(...times),span=Math.max(1,hi-lo);
rows.forEach((r,i)=>{const x=70+800*(Date.parse(r.event_date+'T00:00:00Z')-lo)/span,y=34+i*44;
  const line=element('line',{x1:60,y1:y,x2:930,y2:y,stroke:'#ddd'});svg.append(line);const dot
  =element('circle',{cx:x,cy:y,r:7,fill:'#222'});dot.addEventListener('click',()=>show(r));svg.
  append(dot);const t=element('text',{x:10,y:y+5,'font-size':14});t.textContent=r.event_id;svg.
  append(t);const d=element('text',{x:Math.min(x+12,880),y:y-12,'font-size':12});d.textContent
  =r.event_date;svg.append(d);const button=document.createElement('button');button.textContent
  =r.event_id;button.addEventListener('click',()=>show(r));b.append(button);});
let held=0;document.querySelectorAll('tbody tr').forEach(row=>{row.hidden=row.dataset.matter
  !==selector.value;if(!row.hidden&&row.cells.length===10)held++;});document.getElementById('
  reviewcount').textContent=held+' records need review for this matter.';}
selector.addEventListener('change',draw);draw();</script></html>"""
    )


def run(
    input_path: Path, register_path: Path, output: Path, selected: str | None = None
) -> dict:
    if output.exists():
        raise FileExistsError(f"Refusing to overwrite existing output: {output}")
    events = load_csv(input_path, EVENT_FIELDS)
    matters = load_csv(register_path, MATTER_FIELDS)
    accepted, held = check(events, matters)
    known = sorted({row["matter_id"] for row in matters})
    if selected is not None and selected not in known:
        raise ValueError(f"Unknown selected matter: {selected}")
    selected_a = [row for row in accepted if selected is None or row["matter_id"] == selected
  ]
    selected_h = [row for row in held if selected is None or row["matter_id"] == selected]
    display_ids = (
        [selected] if selected else sorted(set(known) | {row["matter_id"] for row in held})
    )
    record = {
```

```
        "version": VERSION,
        "input_file": input_path.name,
        "register_file": register_path.name,
        "input_sha256": hashlib.sha256(input_path.read_bytes()).hexdigest(),
        "register_sha256": hashlib.sha256(register_path.read_bytes()).hexdigest(),
        "program_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
        "input_rows": len(events),
        "accepted_rows": len(accepted),
        "held_rows": len(held),
        "selected_matter": selected,
        "selected_accepted": len(selected_a),
        "selected_held": len(selected_h),
    }
    output.mkdir(parents=True, exist_ok=False)
    write_csv(output / "accepted.csv", selected_a, EVENT_FIELDS + ["source_row"])
    write_csv(output / "review.csv", selected_h, EVENT_FIELDS + ["source_row", "reasons"])
    (output / "report.html").write_text(
        browser_page(selected_a, selected_h, display_ids), encoding="utf-8"
    )
    (output / "run.json").write_text(
        json.dumps(record, indent=2, sort_keys=True) + "\n", encoding="utf-8"
    )
    return record


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", required=True, type=Path)
    parser.add_argument("--matters", required=True, type=Path)
    parser.add_argument("--out", required=True, type=Path)
    parser.add_argument("--matter")
    args = parser.parse_args()
    try:
        result = run(args.input, args.matters, args.out, args.matter)
    except (ValueError, OSError, csv.Error) as exc:
        parser.exit(2, str(exc) + "\n")
    print(json.dumps(result, indent=2, sort_keys=True))


if __name__ == "__main__":
    main()
```

Appendix B

# The Shared Application Infrastructure

programs/legal_common.py

```
"""Shared, standard-library-only helpers for synthetic legal-office lessons.

No network calls. No input rewriting. The HTML writer escapes source text.
These teaching checks are not professional or security certification.
"""

from __future__ import annotations
import csv
import hashlib
import html
import json
import re
from collections import Counter
from dataclasses import dataclass, field
from datetime import date
from pathlib import Path
from typing import Any

VERSION = "legal-copyedit-2026-10-24.2"
MATTER_FIELDS = ["matter_id", "client_id", "description"]
REQUEST_FIELDS = ["request_id", "matter_id", "owner", "request_date", "source_id"]
RESPONSE_FIELDS = ["response_id", "matter_id", "request_id", "response_date", "source_id"]
APPOINTMENT_FIELDS = [
    "appointment_id",
    "matter_id",
    "scheduled_date",
    "status",
    "completion_date",
    "name",
    "note",
]
DOCUMENT_FIELDS = [
    "file_id",
    "matter_id",
    "document_id",
    "version",
    "type",
    "source_id",
    "title",
]


def load(folder: Path, filename: str, fields: list[str]) -> list[dict[str, str]]:
```

```
    """Read exact columns; positions are logical records, not physical lines."""
    with (folder / filename).open(encoding="utf-8-sig", newline="") as stream:
        reader = csv.DictReader(stream, strict=True)
        if reader.fieldnames != fields:
            raise ValueError(f"{filename}: expected columns {fields}; got {reader.fieldnames}
  ")
        try:
            rows = list(reader)
        except csv.Error as error:
            raise ValueError(f"Malformed CSV: {error}") from error
    if any(None in row or any(value is None for value in row.values()) for row in rows):
        raise ValueError(f"{filename}: malformed CSV record")
    return rows


def day(value: str) -> date | None:
    if not re.fullmatch(r"\d{4}-\d{2}-\d{2}", value):
        return None
    try:
        return date.fromisoformat(value)
    except ValueError:
        return None


def required_day(value: str) -> date:
    result = day(value)
    if result is None:
        raise ValueError(f"Invalid reporting date: {value}")
    return result


def natural(value: str) -> int | None:
    return int(value) if re.fullmatch(r"[0-9]+", value) else None


def registry(folder: Path) -> dict[str, str]:
    rows = load(folder, "matters.csv", MATTER_FIELDS)
    keys = [r["matter_id"] for r in rows]
    if len(keys) != len(set(keys)) or any(
        not r["matter_id"].strip() or not r["client_id"].strip() for r in rows
    ):
        raise ValueError(
            "Matter register requires unique, nonblank matter IDs and nonblank client IDs"
        )
    return {r["matter_id"]: r["client_id"] for r in rows}


def identity_issues(rows: list[dict], key: str) -> list[list[str]]:
    counts = Counter(r[key] for r in rows)
    return [
        ["missing_" + key]
        if not r[key].strip()
        else ["duplicate_" + key]
        if counts[r[key]] > 1
        else []
        for r in rows
    ]


def require_cells(row: dict, fields: list[str], issues: list[str]) -> None:
```

```
    issues.extend("missing_" + field for field in fields if not str(row[field]).strip())


def sha(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def esc(value: Any) -> str:
    return html.escape(str(value), quote=True)


def html_table(rows: list[dict], fields: list[str]) -> str:
    header = "".join("<th>" + esc(f.replace("_", " ")) + "</th>" for f in fields)
    body = "".join(
        "<tr>" + "".join("<td>" + esc(row.get(f, "")) + "</td>" for f in fields) + "</tr>"
        for row in rows
    )
    if not rows:
        body = f'<tr><td colspan="{max(1, len(fields))}">No records in this table.</td></tr>'
    return (
        '<div class="scroll"><table><thead><tr>'
        + header
        + "</tr></thead><tbody>"
        + body
        + "</tbody></table></div>"
    )


def page(title: str, body: str) -> str:
    return (
        """<!doctype html><html lang="en"><meta charset="utf-8"><meta name="viewport" content
  ="width=device-width, initial-scale=1"><title>"""
        + esc(title)
        + """</title><style>
body{font:18px/1.45 system-ui,sans-serif;max-width:1150px;margin:28px auto;padding:0 24px;
  color:#161616;background:white}h1{font-size:30px}h2{font-size:22px;margin-top:28px}table{
  border-collapse:collapse;width:100%;font-size:15px}th,td{border-bottom:1px solid #999;
  padding:9px;text-align:left;vertical-align:top}th{background:#f2f2f2}.scroll{overflow:auto}
  pre{white-space:pre-wrap;overflow-wrap:anywhere}select{font:inherit;padding:5px}ins{
  background:#e5e5e5;text-decoration:underline}del{text-decoration:line-through}small{font-
  size:13px}.tag{border:1px solid;padding:4px 8px;display:inline-block}svg{max-width:100%}.
  letter{max-width:720px;border:1px solid #aaa;padding:32px;margin:18px 0} @media print{body{
  margin:0;font-size:11pt}.scroll{overflow:visible}table{font-size:9pt}tr{break-inside:avoid}}
</style><h1>"""
        + esc(title)
        + '</h1><p class="tag">SYNTHETIC TEACHING DATA</p>'
        + body
        + "</html>"
    )


@dataclass
class Result:
    title: str
    summary: dict[str, Any]
    tables: dict[str, tuple[list[str], list[dict]]] = field(default_factory=dict)
    extra: dict[str, str] = field(default_factory=dict)
    body: str = ""

    def add(self, filename: str, rows: list[dict], fields: list[str]) -> None:
```

```
        self.tables[filename] = (fields, rows)

    def report(self) -> str:
        summary_rows = [
            {"item": k, "value": v if v is not None else "unavailable"}
            for k, v in self.summary.items()
        ]
        tables = "".join(
            "<h2>" + esc(name) + "</h2>" + html_table(rows, fields)
            for name, (fields, rows) in self.tables.items()
        )
        return page(
            self.title,
            self.body
            + "<h2>Summary</h2>"
            + html_table(summary_rows, ["item", "value"])
            + tables,
        )


def save_result(result: Result, folder: Path, output: Path, parameters: dict) -> None:
    """Compute first, then reserve a new destination. Never overwrite a run.

    A writing error can leave an incomplete destination; run.json is written
    last. A complete run needs every member named there, not just the folder.
    """
    if output.exists() or output.is_symlink():
        raise FileExistsError(f"Refusing to overwrite {output}")
    # Validate the complete namespace before reserving the destination.
    names = (
        list(result.tables) + list(result.extra) + ["summary.json", "report.html", "run.json"
 ]
    )
    if any(Path(name).name != name or name in {"", ".", ".."} for name in names):
        raise ValueError("Only direct output members are permitted")
    if len({name.casefold() for name in names}) != len(names):
        raise ValueError("Output names collide, including on a case-insensitive filesystem")
    output.mkdir(parents=True, exist_ok=False)
    for filename, (fields, rows) in result.tables.items():
        with (output / filename).open("w", encoding="utf-8", newline="") as stream:
            writer = csv.DictWriter(stream, fieldnames=fields, lineterminator="\n")
            writer.writeheader()
            writer.writerows(rows)
    for filename, text in result.extra.items():
        # Generated names are fixed or validated identifiers, never source paths.
        if Path(filename).name != filename:
            raise ValueError("Only direct output members are permitted")
        (output / filename).write_text(text, encoding="utf-8")
    (output / "summary.json").write_text(
        json.dumps(result.summary, indent=2, sort_keys=True) + "\n", encoding="utf-8"
    )
    (output / "report.html").write_text(result.report(), encoding="utf-8")
    # RUN.txt is a reader's instruction sheet in a chapter folder, not an input.
    inputs = {
        p.name: sha(p)
        for p in sorted(folder.iterdir())
        if p.is_file() and p.suffix in {".csv", ".txt"} and p.name != "RUN.txt"
    }
    sources = {p.name: sha(p) for p in sorted(Path(__file__).parent.glob("*.py"))}
    outputs = {p.name: sha(p) for p in sorted(output.iterdir()) if p.is_file()}
```

```
    record = {
        "version": VERSION,
        "parameters": parameters,
        "input_hashes": inputs,
        "program_hashes": sources,
        "output_hashes": outputs,
        "note": "Hash identity is not professional approval. run.json does not hash itself.",
    }
    (output / "run.json").write_text(
        json.dumps(record, indent=2, sort_keys=True) + "\n", encoding="utf-8"
    )
```

programs/run_chapter.py

```
"""Run a supplied chapter reference without editing code.

From the project folder: python3 programs/run_chapter.py 4 --out my_run_04
Inside a chapter folder the inputs beside this program are used. In the
release's programs/ folder, the matching data/chNN folder is used.
"""

from __future__ import annotations
import argparse
import inspect
import csv
import json
import sys
from pathlib import Path
from legal_common import save_result
from legal_apps import APPLICATIONS

NAMES = {
    4: "consultation_report",
    5: "document_catalogue",
    6: "document_register_check",
    7: "management_extract",
    8: "request_response_match",
    9: "matter_population",
    10: "turnaround_report",
    11: "weekly_request_monitor",
    12: "production_index",
    13: "administrative_letters",
    14: "supplied_date_register",
    15: "party_search",
    16: "authority_register",
    17: "document_compare",
    18: "time_entry_check",
    19: "closing_review",
    22: "application_review_register",
    23: "matter_worklist",
}


def main(chapter: int | None = None) -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    if chapter is None:
        parser.add_argument("chapter", type=int, choices=sorted(APPLICATIONS))
    parser.add_argument(
        "--data", type=Path, help="Folder containing the complete chapter input files"
```

```
    )
    parser.add_argument("--out", type=Path, required=True, help="A NEW output folder")
    parser.add_argument("--as-of", help="Explicit historical reporting date YYYY-MM-DD")
    parser.add_argument("--window", type=int, default=14)
    parser.add_argument("--query", default="Alex Morgan")
    parser.add_argument("--exact-only", action="store_true")
    parser.add_argument("--whitespace", action="store_true")
    parser.add_argument("--reverse", action="store_true")
    parser.add_argument("--artifact", default="2")
    parser.add_argument("--rule", default="1")
    parser.add_argument("--matter")
    args = parser.parse_args()
    ch = chapter if chapter is not None else args.chapter
    here = Path(__file__).resolve().parent
    # In a chapter folder the inputs sit beside the program: always use them.
    # Only in the release's programs/ folder (no inputs beside it) use data/chNN.
    beside = any(p.suffix in {".csv", ".txt"} and p.name != "RUN.txt" for p in here.iterdir()
)
    data = args.data or (here if beside else here.parent / "data" / f"ch{ch:02}")
    options = {
        "window": args.window,
        "query": args.query,
        "similar": not args.exact_only,
        "whitespace": args.whitespace,
        "reverse": args.reverse,
        "artifact": args.artifact,
        "rule": args.rule,
        "matter": args.matter,
    }
    options["as_of"] = args.as_of or (
        "2026-09-15" if ch == 9 else "2026-12-31" if ch == 10 else "2026-09-24"
    )
    # Record and pass only the settings this chapter's application uses.
    accepted = inspect.signature(APPLICATIONS[ch]).parameters
    parameters = {"chapter": ch, **{k: v for k, v in options.items() if k in accepted}}
    try:
        if args.out.exists() or args.out.is_symlink():
            raise FileExistsError("Choose a new output folder; the destination already exists
"
)
        result = APPLICATIONS[ch](data, **parameters)
        save_result(result, data, args.out, parameters)
    except (ValueError, OSError, csv.Error) as exc:
        parser.exit(2, f"{type(exc).__name__}: {exc}\n")
    print(json.dumps(result.summary, indent=2, sort_keys=True))


if __name__ == "__main__":
    main()
```

## Reading the Shared Runner

The runner selects the application before performing its work. The `APPLICATIONS` dictionary maps the chapter number to a function. The named entry points supply that number automatically; the universal command asks you for it. Both routes reach the same implementation, so changing the entry-point name does not change

a chapter's rules.

The default data path is resolved relative to the installed script, not a remembered working directory. An explicit `--data` selects your copied fixture. The output argument names a new destination. The common parser offers several options; the chapter's instructions identify which ones affect its application. A search query does not change a date-register rule merely because both commands use the same parser.

Each application first returns its summary and tables. The saving function then creates the destination and writes the outputs, with the run record last. This separation lets a test compare row identities and calculated values before it inspects files. It also makes an unsuccessful write distinguishable from a successful computation: a partly written folder is not a completed delivery.

The error handler reports an input or operating-system problem and exits unsuccessfully. It does not silently rewrite an input, suppress a failed check, or remove an existing output folder. Preserve the diagnostic and choose the next action from its cause. A wrong header needs the correct input; an existing destination needs a new name; a permission failure needs an authorized location. These are different repairs, not reasons to request another plausible-looking report.

**Appendix C**

# The Later Reference Implementations

Each named chapter wrapper calls a function in this canonical module. Shared matching rules are implemented once and reused by the request matcher and worklist. Functions return a summary, tables, optional extra files, and a browser report; the infrastructure handles saved output and run records. The chapter listings let you read one application beside its inputs and experiment. The complete listing below deliberately repeats those functions so that imports, helpers, and the application map can also be read as one contiguous file. You need not read it a second time to work a chapter. The stated line ranges identify this release's canonical source; they are regenerated when the implementation changes.

programs/legal_apps.py

```
"""Reference applications for Chapters 4--19, 22 and 23.

All inputs are synthetic teaching records. The functions return data before
any files are written. Rules and tests, not AI calls, determine the results.
"""

from __future__ import annotations
import difflib
import math
import json
import re
import unicodedata
from collections import Counter, defaultdict
from datetime import date, timedelta
from decimal import Decimal, InvalidOperation, ROUND_HALF_UP, localcontext, Context
from pathlib import Path
from statistics import mean, median
from legal_common import (
    Result,
    load,
    day,
    required_day,
    natural,
    registry,
    identity_issues,
    require_cells,
    sha,
    esc,
    html_table,
    page,
```

```
    MATTER_FIELDS,
    REQUEST_FIELDS,
    RESPONSE_FIELDS,
    APPOINTMENT_FIELDS,
    DOCUMENT_FIELDS,
)


# Chapter 4: documented completions, not just completed status labels.
def consultation(folder: Path, as_of: str = "2026-09-24", **_) -> Result:
    """Documented consultation completion as of a reporting date.

    Rule (Section 4.1 of the book). The denominator holds every appointment
    scheduled on or before the reporting date except cancelled ones. The
    numerator holds the denominator rows marked completed that also have a
    completion date between the scheduled date and the reporting date.
    An appointment scheduled after the reporting date is outside the report:
    it is labelled not_yet_scheduled and takes part in no count. A structural
    defect (duplicate ID, unknown status or matter, invalid scheduled date)
    suppresses the headline percentage but keeps every row in the ledger.
    """
    cutoff = required_day(as_of)
    known = registry(folder)
    rows = load(folder, "appointments.csv", APPOINTMENT_FIELDS)
    issues = identity_issues(rows, "appointment_id")
    ledger = []
    structural = False
    for i, r in enumerate(rows):
        e = issues[i]
        scheduled = day(r["scheduled_date"])
        completed = day(r["completion_date"])
        # Structural checks first: these suppress the headline.
        if r["status"] not in {"completed", "cancelled", "missed", "scheduled"}:
            e.append("unknown_status")
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if scheduled is None:
            e.append("invalid_scheduled_date")
        structural |= bool(e)
        # Then the population: is the row in the report at all, and in the denominator?
        in_report = not e and scheduled <= cutoff
        eligible = in_report and r["status"] != "cancelled"
        documented = (
            completed is not None and scheduled is not None and scheduled <= completed <=
  cutoff
        )
        numerator = eligible and r["status"] == "completed" and documented
        # Review labels apply only to rows the report covers.
        if in_report and r["status"] == "completed" and not documented:
            e.append("completion_date_needs_review")
        if in_report and r["status"] != "completed" and r["completion_date"]:
            e.append("date_on_noncompleted_status")
        if e:
            explanation = ";".join(e)
        elif not in_report:
            explanation = "not_yet_scheduled"
        elif r["status"] == "cancelled":
            explanation = "cancelled"
        elif numerator:
            explanation = "documented_completion"
```

```
        elif r["status"] == "scheduled":
            explanation = "unresolved"
        else:
            explanation = "denominator_only"
        ledger.append(
            {
                "source_row": i + 2,
                "appointment_id": r["appointment_id"],
                "matter_id": r["matter_id"],
                "status": r["status"],
                "scheduled_date": r["scheduled_date"],
                "completion_date": r["completion_date"],
                "eligible": int(eligible),
                "numerator": int(numerator),
                "explanation": explanation,
            }
        )
    den = sum(r["eligible"] for r in ledger)
    num = sum(r["numerator"] for r in ledger)
    covered = [r for r in ledger if r["explanation"] != "not_yet_scheduled"]
    result = Result(
        "Documented consultation completion",
        {
            "input_rows": len(rows),
            "reporting_date": as_of,
            "numerator": num,
            "denominator": den,
            "percentage": None if structural or den == 0 else num / den * 100,
            "headline_available": not structural and den > 0,
            "cancelled": sum(r["status"] == "cancelled" for r in covered),
            "unresolved": sum(
                r["explanation"] == "unresolved" or "review" in r["explanation"]
                for r in covered
            ),
        },
    )
    result.add(
        "ledger.csv",
        ledger,
        [
            "source_row",
            "appointment_id",
            "matter_id",
            "status",
            "scheduled_date",
            "completion_date",
            "eligible",
            "numerator",
            "explanation",
        ],
    )
    return result


# Chapters 5 and 6: common register checks, different deliverables.
def document_rows(folder: Path) -> tuple[list[dict], list[dict]]:
    """Shared loader for Chapters 5 and 6: validate the document register row by row.

    Returns (accepted, held). A row is held, with every reason, when its file
    ID is blank or duplicated, its document ID, type or source reference is
```

```
    blank, its matter is not in matters.csv, or its version is not a whole
    number of at least 1 written in digits. Nothing is corrected.
    """
    rows = load(folder, "documents.csv", DOCUMENT_FIELDS)
    known = registry(folder)
    issues = identity_issues(rows, "file_id")
    accepted = []
    held = []
    for i, r in enumerate(rows):
        e = issues[i]
        require_cells(r, ["document_id", "source_id", "type"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        v = natural(r["version"])
        if v is None or v < 1:
            e.append("invalid_version")
        enriched = dict(r, source_row=i + 2, issues=";".join(e))
        (held if e else accepted).append(enriched)
    return accepted, held


def catalogue(folder: Path, **_) -> Result:
    """Chapter 5: count file records, documents, and labelled versions.

    A document is the pair (matter, document ID); a labelled version is the
    triple (matter, document ID, version). Two rows with the same triple are
    two copies of one version and both stay in the membership table.
    """
    accepted, held = document_rows(folder)
    groups = defaultdict(list)
    for r in accepted:
        groups[r["matter_id"]].append(r)
    summaries = []
    for m, rr in sorted(groups.items()):
        summaries.append(
            {
                "matter_id": m,
                "file_records": len(rr),
                "documents": len({r["document_id"] for r in rr}),
                "versions": len({(r["document_id"], r["version"]) for r in rr}),
            }
        )
    result = Result(
        "Matter-document catalogue",
        {
            "file_records": len(accepted),
            "held": len(held),
            "documents": len({(r["matter_id"], r["document_id"]) for r in accepted}),
            "versions": len(
                {(r["matter_id"], r["document_id"], r["version"]) for r in accepted}
            ),
        },
    )
    result.add(
        "catalogue.csv", summaries, ["matter_id", "file_records", "documents", "versions"]
    )
    result.add("membership.csv", accepted, DOCUMENT_FIELDS + ["source_row", "issues"])
    result.add("review.csv", held, DOCUMENT_FIELDS + ["source_row", "issues"])
    return result
```

```
def document_check(folder: Path, **_) -> Result:
    """Chapter 6: accepted and held lists for the register, plus a consistency list.

    The consistency check groups every row, held or not, by (matter, document,
    version) and lists groups that mix a draft type with an executed type. It
    reports; it never holds or repairs a row.
    """
    accepted, held = document_rows(folder)
    allrows = sorted(accepted + held, key=lambda r: r["source_row"])
    consistency = []
    grouped = defaultdict(list)
    for r in allrows:
        grouped[(r["matter_id"], r["document_id"], r["version"])].append(r)
    for key, rr in grouped.items():
        labels = {r["type"] for r in rr}
        if any("draft" in s for s in labels) and any("executed" in s for s in labels):
            for r in rr:
                consistency.append(
                    {
                        "source_row": r["source_row"],
                        "file_id": r["file_id"],
                        "issue": "conflicting_supplied_types",
                        "original_type": r["type"],
                    }
                )
    result = Result(
        "Document-register review",
        {
            "input_rows": len(allrows),
            "accepted": len(accepted),
            "held": len(held),
            "consistency_rows": len(consistency),
        },
    )
    result.add("accepted.csv", accepted, DOCUMENT_FIELDS + ["source_row", "issues"])
    result.add("review.csv", held, DOCUMENT_FIELDS + ["source_row", "issues"])
    result.add(
        "consistency.csv", consistency, ["source_row", "file_id", "issue", "original_type"]
    )
    return result


# Chapter 7: allowlist output fields; do not copy private rows to diagnostics.
def management_extract(folder: Path, **_) -> Result:
    """Chapter 7: weekly status counts built from three permitted fields only.

    Weeks begin on Monday. Every week present in the input gets one row for
    each of the four statuses, zeros included. An unknown status, invalid date
    or duplicate ID stops the run with a message that quotes no source value.
    """
    rows = load(folder, "appointments.csv", APPOINTMENT_FIELDS)
    categories = ["completed", "cancelled", "missed", "scheduled"]
    counts = Counter()
    weeks = set()
    if any(identity_issues(rows, "appointment_id")):
        raise ValueError("Appointment identity error; inspect the source locally")
    for r in rows:
        d = day(r["scheduled_date"])
        if d is None or r["status"] not in categories:
```

```
            raise ValueError("Invalid reporting date or status; inspect the source locally")
        week = str(d - timedelta(days=d.weekday()))
        weeks.add(week)
        counts[(week, r["status"])] += 1
    output = [
        {"week_beginning": w, "status": s, "count": counts[(w, s)]}
        for w in sorted(weeks)
        for s in categories
    ]
    result = Result(
        "Limited weekly status extract",
        {
            "input_rows": len(rows),
            "output_rows": len(output),
            "total_count": sum(r["count"] for r in output),
        },
    )
    result.add("extract.csv", output, ["week_beginning", "status", "count"])
    return result


# Chapter 8 and Chapter 23 share this relationship and historical-cutoff logic.
def match_records(
    requests: list[dict], responses: list[dict], known: dict, cutoff: date, window: int = 14
) -> tuple[list[dict], list[dict]]:
    """Chapters 8 and 23: link responses to requests and select the earliest in-window one.

    A response belongs to a request only when matter_id and request_id both
    agree. Elapsed days are calendar days; a negative value is invalid and
    sends the request to review. A response after the reporting date is kept
    but not observed. An observed response is eligible when 0 <= elapsed <=
    window, both ends included. Among eligible responses the earliest date is
    selected; a tie makes the request ambiguous and selects nothing.
    """
    if not isinstance(window, int) or window < 0:
        raise ValueError("Window must be a nonnegative integer")
    keys = Counter((r["matter_id"], r["request_id"]) for r in requests)
    request_issues = []
    request_index = {}
    for i, r in enumerate(requests):
        e = []
        require_cells(r, ["request_id", "owner", "source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if day(r["request_date"]) is None:
            e.append("invalid_request_date")
        key = (r["matter_id"], r["request_id"])
        if keys[key] > 1:
            e.append("duplicate_request_key")
        request_issues.append(e)
        if keys[key] == 1:
            request_index[key] = i
    issues = identity_issues(responses, "response_id")
    candidates = []
    by_request = defaultdict(list)
    for i, r in enumerate(responses):
        e = issues[i]
        require_cells(r, ["source_id", "request_id"], e)
        d = day(r["response_date"])
        if d is None:
```

```
            e.append("invalid_response_date")
        key = (r["matter_id"], r["request_id"])
        j = request_index.get(key)
        if keys[key] > 1:
            e.append("ambiguous_request_key")
        elif j is None:
            e.append("unknown_request_key")
        elif request_issues[j]:
            e.append("request_needs_review")
        elapsed = None
        if j is not None and not request_issues[j] and d is not None:
            elapsed = (d - required_day(requests[j]["request_date"])).days
            if elapsed < 0:
                e.append("response_before_request")
        linked_valid = not e
        observed = (
            linked_valid and d <= cutoff and required_day(requests[j]["request_date"]) <=
cutoff
        )
        eligible = bool(observed and elapsed is not None and 0 <= elapsed <= window)
        reason = (
            ";".join(e)
            if e
            else "after_as_of"
            if not observed
            else "in_window"
            if eligible
            else "outside_window"
        )
        row = dict(
            r,
            source_row=i + 2,
            elapsed_days="" if elapsed is None else elapsed,
            valid=int(linked_valid),
            observed=int(observed),
            eligible=int(eligible),
            selection="not_selected",
            explanation=reason,
        )
        candidates.append(row)
        if j is not None:
            by_request[j].append(row)
    results = []
    for i, r in enumerate(requests):
        e = request_issues[i]
        d = day(r["request_date"])
        rr = by_request[i]
        observed = [x for x in rr if x["observed"]]
        eligible = [x for x in rr if x["eligible"]]
        linked_problems = any(not x["valid"] for x in rr)
        future = not e and d > cutoff
        evidence_unavailable = bool(e) or future
        any_response = "unavailable" if evidence_unavailable else "yes" if observed else "no"
        in_window = "unavailable" if evidence_unavailable else "yes" if eligible else "no"
        selected = ""
        earliest = "unavailable" if evidence_unavailable else "none"
        if eligible:
            earliest_date = min(x["response_date"] for x in eligible)
            tied = [x for x in eligible if x["response_date"] == earliest_date]
            earliest = "unique" if len(tied) == 1 else "ambiguous"
```

```
            for x in tied:
                x["selection"] = "selected" if len(tied) == 1 else "tied_earliest"
            if len(tied) == 1:
                selected = tied[0]["response_id"]
        if e or linked_problems:
            status = "needs_review"
        elif future:
            status = "not_yet_requested"
        elif observed:
            status = "response_recorded"
        elif (cutoff - d).days <= window:
            status = "window_open"
        else:
            status = "window_passed"
        results.append(
            dict(
                r,
                source_row=i + 2,
                status=status,
                any_response=any_response,
                in_window=in_window,
                earliest=earliest,
                selected_response=selected,
                issues=";".join(
                    e + (["linked_response_needs_review"] if linked_problems else [])
                ),
            )
        )
    return results, candidates


MATCH_FIELDS = REQUEST_FIELDS + [
    "source_row",
    "status",
    "any_response",
    "in_window",
    "earliest",
    "selected_response",
    "issues",
]
CANDIDATE_FIELDS = RESPONSE_FIELDS + [
    "source_row",
    "elapsed_days",
    "valid",
    "observed",
    "eligible",
    "selection",
    "explanation",
]


def matching(folder: Path, as_of: str = "2026-09-24", window: int = 14, **_) -> Result:
    """Chapter 8: one result row per request and a candidate ledger with one row per response.
  """
    req = load(folder, "requests.csv", REQUEST_FIELDS)
    resp = load(folder, "responses.csv", RESPONSE_FIELDS)
    matched, candidates = match_records(
        req, resp, registry(folder), required_day(as_of), window
    )
    result = Result(
```

```
        "Document request-response matching",
        {
            "requests": len(req),
            "responses": len(resp),
            "matched": sum(r["earliest"] == "unique" for r in matched),
            "ambiguous": sum(r["earliest"] == "ambiguous" for r in matched),
            "reporting_date": as_of,
            "window_days": window,
        },
    )
    result.add("matches.csv", matched, MATCH_FIELDS)
    result.add("candidates.csv", candidates, CANDIDATE_FIELDS)
    result.add(
        "unmatched.csv",
        [
            r
            for r in candidates
            if "unknown_request_key" in r["explanation"]
            or "ambiguous_request_key" in r["explanation"]
        ],
        CANDIDATE_FIELDS,
    )
    return result


# Chapter 9: population membership and observation eligibility are separate.
def population(folder: Path, as_of: str = "2026-09-15", **_) -> Result:
    """Chapter 9: which matters are active on the reporting date, and their observed delays.

    Included: opened on or before the reporting date and closed after it (or
    not closed). Excluded: valid dates that fail that test. Review: invalid or
    inconsistent dates, duplicate or unregistered identifiers, blank source.
    Delay statistics use included matters with a whole-number delay; a blank
    is missing and a non-integer is invalid. Neither is zero.
    """
    rows = load(
        folder,
        "population.csv",
        ["matter_id", "opening_date", "closure_date", "response_days", "source_id"],
    )
    known = registry(folder)
    cutoff = required_day(as_of)
    issues = identity_issues(rows, "matter_id")
    ledger = []
    values = []
    included = excluded = review = 0
    for i, r in enumerate(rows):
        e = issues[i]
        o = day(r["opening_date"])
        c = day(r["closure_date"])
        require_cells(r, ["source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if o is None:
            e.append("invalid_opening")
        if r["closure_date"] and (c is None or o is not None and c < o):
            e.append("invalid_closure")
        status = (
            "review"
            if e
```

```
            else "included"
            if o <= cutoff and (c is None or c > cutoff)
            else "excluded"
        )
        included += status == "included"
        excluded += status == "excluded"
        review += status == "review"
        value = natural(r["response_days"])
        obs = status == "included" and value is not None
        if obs:
            values.append(value)
        observation = (
            "observed"
            if obs
            else "not_in_population"
            if status != "included"
            else "missing"
            if not r["response_days"]
            else "invalid_value"
        )
        ledger.append(
            dict(
                r,
                source_row=i + 2,
                classification=status,
                observation=observation,
                issues=";".join(e),
            )
        )
    result = Result(
        "Active-register population",
        {
            "included": included,
            "excluded": excluded,
            "review": review,
            "observations": len(values),
            "missing_or_invalid_observations": included - len(values),
            "mean": mean(values) if values else None,
            "median": median(values) if values else None,
            "reporting_date": as_of,
        },
    )
    result.add(
        "population.csv",
        ledger,
        [
            "matter_id",
            "opening_date",
            "closure_date",
            "response_days",
            "source_id",
            "source_row",
            "classification",
            "observation",
            "issues",
        ],
    )
    return result
```

```
# Chapter 10: chart is derived only from the saved monthly table.
def turnaround_svg(summary: list[dict]) -> str:
    """Derive scale, missing-month notes and accessible text from the table."""
    values = [float(r[f]) for r in summary for f in ("mean", "median") if r[f] != ""]
    maximum = max(values, default=0)
    # Leave room above the largest marker, including when every value is zero.
    raw = max(maximum * 1.1, 1) / 4
    unit = 10 ** math.floor(math.log10(raw))
    step = next(n * unit for n in (1, 2, 5, 10) if n * unit >= raw)
    top = 4 * step
    missing = [r["month"] for r in summary if r["count"] == 0]
    note = (
        "No observed pairs: " + ", ".join(missing) + ". Gaps are not zero days."
        if missing
        else "Every displayed month has at least one observed pair."
    )
    description = (
        "Monthly mean and median recorded turnaround, in days, grouped by request month. "
        + note
    )
    pieces = [
        '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 900 450" role="img" aria-label
="'
        + esc(description)
        + '"><title>'
        + esc(description)
        + '</title><rect width="900" height="450" fill="white"/>'
    ]
    for tick in range(5):
        value = tick * step
        yp = 320 - value / top * 260
        pieces.append(
            f'<line x1="70" y1="{yp}" x2="870" y2="{yp}" stroke="#ddd"/><text x="8" y="{yp +
5}" font-size="18">{value:g}</text>'
        )
    spacing = 780 / max(1, len(summary) - 1)
    for i, r in enumerate(summary):
        pieces.append(
            f'<text x="{78 + i * spacing}" y="350" font-size="18">{esc(r["month"][5:])}</text
>'
        )
    for field, marker in [("mean", "circle"), ("median", "rect")]:
        previous = None
        for i, r in enumerate(summary):
            if r[field] == "":
                previous = None
                continue
            x = 85 + i * spacing
            y = 320 - float(r[field]) / top * 260
            if previous:
                dash = ' stroke-dasharray="5 4"' if field == "median" else ""
                pieces.append(
                    f'<line x1="{previous[0]}" y1="{previous[1]}" x2="{x}" y2="{y}" stroke="
black"{dash}/>'
                )
            pieces.append(
                f'<circle cx="{x}" cy="{y}" r="5" fill="white" stroke="black"/>'
                if marker == "circle"
                else f'<rect x="{x - 4}" y="{y - 4}" width="8" height="8" fill="black"/>'
            )
```

```
            previous = (x, y)
    pieces.append(
        '<text x="70" y="25" font-size="21">Days: circle / solid = mean; square / dashed =
  median</text>'
    )
    pieces.append(
        '<text x="70" y="380" font-size="18">Request month in 2026; the scale follows the
  displayed values.</text>'
    )
    import textwrap

    for line_number, line in enumerate(textwrap.wrap(note, width=87)):
        pieces.append(
            f'<text x="70" y="{406 + line_number * 23}" font-size="17">{esc(line)}</text>'
        )
    pieces.append("</svg>")
    return "".join(pieces)


def turnaround(folder: Path, as_of: str = "2026-12-31", **_) -> Result:
    """Chapter 10: monthly count, mean and median of elapsed days, grouped by request month.

    A pair is observed when its matter is registered, its identifiers are
    present and unique, both dates are valid and ordered, the supplied
    elapsed_days equals the calendar difference, the request is in 2026, and
    both dates are on or before the observation cutoff. A month with no
    observation keeps count 0 and blank statistics; nothing is plotted there.
    """
    fields = [
        "pair_id",
        "matter_id",
        "request_id",
        "request_date",
        "response_date",
        "elapsed_days",
        "source_id",
    ]
    rows = load(folder, "pairs.csv", fields)
    known = registry(folder)
    cutoff = required_day(as_of)
    issues = identity_issues(rows, "pair_id")
    keys = Counter((r["matter_id"], r["request_id"]) for r in rows)
    groups = defaultdict(list)
    ledger = []
    for i, r in enumerate(rows):
        e = issues[i]
        d = day(r["request_date"])
        v = day(r["response_date"])
        n = natural(r["elapsed_days"])
        require_cells(r, ["request_id", "source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if keys[(r["matter_id"], r["request_id"])] > 1:
            e.append("duplicate_selected_pair")
        if d is None or v is None or v < d:
            e.append("invalid_dates")
        elif n is None or n != (v - d).days:
            e.append("elapsed_disagrees_with_dates")
        if not e and d.year != 2026:
            e.append("outside_teaching_year")
```

```
        status = "review" if e else "after_as_of" if v > cutoff or d > cutoff else "observed"
        if status == "observed":
            groups[r["request_date"][:7]].append(n)
        ledger.append(dict(r, classification=status, issues=";".join(e)))
    summary = []
    for month in range(1, 13):
        key = f"2026-{month:02}"
        values = groups[key]
        summary.append(
            {
                "month": key,
                "count": len(values),
                "mean": mean(values) if values else "",
                "median": median(values) if values else "",
            }
        )
    result = Result(
        "Monthly recorded turnaround",
        {
            "observations": sum(r["count"] for r in summary),
            "months": 12,
            "reporting_date": as_of,
            "grouping": "request month",
            "outstanding_requests": "not derivable from observed-pair input",
        },
    )
    result.add("monthly.csv", summary, ["month", "count", "mean", "median"])
    result.add("pairs_checked.csv", ledger, fields + ["classification", "issues"])
    result.extra["turnaround.svg"] = turnaround_svg(summary)
    result.body = (
        "<p>Observation cutoff: "
        + esc(as_of)
        + ". Missing pairs do not supply an outstanding-request denominator.</p>"
        + result.extra["turnaround.svg"]
    )
    return result


# Chapter 11: pool counts, never unweighted percentages.
def weekly(folder: Path, **_) -> Result:
    """Chapter 11: pooled response rate over the final weeks; provisional weeks shown but not
  pooled.

    The headline divides the sum of responded by the sum of eligible over
    final weeks. Any structural problem in any line (non-integer counts,
    responded > eligible, a week not beginning on a Monday, a repeated week,
    an unknown status) suppresses the headline.
    """
    fields = ["week_id", "week_beginning", "eligible", "responded", "status"]
    rows = load(folder, "weeks.csv", fields)
    issues = identity_issues(rows, "week_id")
    week_counts = Counter(r["week_beginning"] for r in rows)
    ledger = []
    en = rn = 0
    structural = False
    for i, r in enumerate(rows):
        e = issues[i]
        n = natural(r["eligible"])
        v = natural(r["responded"])
        d = day(r["week_beginning"])
```

```
        if n is None or v is None or v > n:
            e.append("invalid_counts")
        if d is None or d.weekday() != 0:
            e.append("invalid_monday_week")
        if week_counts[r["week_beginning"]] > 1:
            e.append("duplicate_week")
        if r["status"] not in {"final", "provisional"}:
            e.append("unknown_status")
        structural |= bool(e)
        if not e and r["status"] == "final":
            en += n
            rn += v
        ledger.append(dict(r, rate="" if e or not n else v / n * 100, issues=";".join(e)))
    result = Result(
        "Weekly request monitor",
        {
            "eligible_final": en,
            "responded_final": rn,
            "pooled_percentage": None if structural or not en else rn / en * 100,
            "headline_available": not structural and en > 0,
        },
    )
    result.add("weeks.csv", ledger, fields + ["rate", "issues"])
    return result


# Chapter 12: supplied Bates labels; gaps and overlaps are reported, never repaired.
BATES = re.compile(r"([A-Z]{2,8})([0-9]{6})")
PRODUCTION_FIELDS = [
    "doc_id",
    "matter_id",
    "volume_id",
    "bates_start",
    "bates_end",
    "page_count",
    "family_id",
    "doc_type",
    "source_id",
]


def bates(value: str) -> tuple[str, int] | None:
    found = BATES.fullmatch(value)
    return (found.group(1), int(found.group(2))) if found else None


def label(prefix: str, number: int) -> str:
    return f"{prefix}{number:06d}"


def production_index(folder: Path, **_) -> Result:
    """Chapter 12: check supplied Bates ranges; list gaps and overlaps; never renumber.

    A document is held for a blank or duplicated ID, an unregistered matter,
    a blank required field, a malformed label, mixed prefixes, an end before
    its start, a page count that disagrees with the range, any number also
    claimed by another document (every claimant), or a family parent missing
    from the register. Well-formed ranges claim their numbers even when the
    document is held, so a gap is a number that no supplied range claims.
    """
```

```
    rows = load(folder, "production.csv", PRODUCTION_FIELDS)
    known = registry(folder)
    issues = identity_issues(rows, "doc_id")
    doc_ids = {r["doc_id"] for r in rows}
    spans = {}
    claims = defaultdict(list)
    for i, r in enumerate(rows):
        e = issues[i]
        require_cells(r, ["volume_id", "family_id", "doc_type", "source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        first = bates(r["bates_start"])
        last = bates(r["bates_end"])
        pages = natural(r["page_count"])
        if first is None or last is None:
            e.append("invalid_bates_label")
        elif first[0] != last[0]:
            e.append("prefix_mismatch")
        elif last[1] < first[1]:
            e.append("end_before_start")
        else:
            spans[i] = (first[0], first[1], last[1])
            for n in range(first[1], last[1] + 1):
                claims[(first[0], n)].append(i)
            if pages is None or pages != last[1] - first[1] + 1:
                e.append("page_count_disagrees_with_range")
        if r["family_id"].strip() and r["family_id"] not in doc_ids:
            e.append("missing_family_parent")
    overlap_numbers = sorted(k for k, v in claims.items() if len(v) > 1)
    for prefix, number in overlap_numbers:
        for i in claims[(prefix, number)]:
            if "overlapping_range" not in issues[i]:
                issues[i].append("overlapping_range")
    gaps = []
    for prefix in sorted({p for p, _ in claims}):
        numbers = sorted(n for p, n in claims if p == prefix)
        previous = numbers[0]
        for n in numbers[1:]:
            if n > previous + 1:
                gaps.append(
                    {
                        "prefix": prefix,
                        "gap_start": label(prefix, previous + 1),
                        "gap_end": label(prefix, n - 1),
                        "missing_numbers": n - previous - 1,
                    }
                )
            previous = n
    index = []
    accepted_pages = 0
    for i, r in enumerate(rows):
        e = issues[i]
        status = "held" if e else "accepted"
        if not e:
            accepted_pages += spans[i][2] - spans[i][1] + 1
        index.append(
            dict(
                r,
                source_row=i + 2,
                range_length="" if i not in spans else spans[i][2] - spans[i][1] + 1,
```

```
            status=status,
            issues=";".join(e),
        )
    )
index.sort(
    key=lambda x: (spans.get(x["source_row"] - 2, ("~", 10**9, 0))[:2], x["source_row"])
)
volumes = []
for volume in sorted({r["volume_id"] for r in rows}):
    members = [x for x in index if x["volume_id"] == volume]
    valid_spans = [
        spans[x["source_row"] - 2] for x in members if x["source_row"] - 2 in spans
    ]
    prefixes = {span[0] for span in valid_spans}
    available = len(valid_spans) == len(members) and len(prefixes) == 1
    first_label = (
        label(valid_spans[0][0], min(x[1] for x in valid_spans)) if available else ""
    )
    last_label = (
        label(valid_spans[0][0], max(x[2] for x in valid_spans)) if available else ""
    )
    range_note = (
        "supplied_ranges_including_held"
        if available
        else "unavailable_invalid_or_mixed_prefix_ranges"
    )
    volumes.append(
        {
            "volume_id": volume,
            "documents": len(members),
            "accepted": sum(x["status"] == "accepted" for x in members),
            "held": sum(x["status"] == "held" for x in members),
            "first_label": first_label,
            "last_label": last_label,
            "range_note": range_note,
        }
    )
accepted = sum(x["status"] == "accepted" for x in index)
result = Result(
    "Production index",
    {
        "input_rows": len(rows),
        "accepted": accepted,
        "held": len(rows) - accepted,
        "accepted_pages": accepted_pages,
        "supplied_page_count_total": sum(natural(r["page_count"]) or 0 for r in rows),
        "claimed_numbers": len(claims),
        "gap_ranges": len(gaps),
        "gap_numbers": sum(g["missing_numbers"] for g in gaps),
        "overlap_numbers": len(overlap_numbers),
    },
)
result.add(
    "index.csv",
    index,
    ["source_row"] + PRODUCTION_FIELDS + ["range_length", "status", "issues"],
)
result.add("gaps.csv", gaps, ["prefix", "gap_start", "gap_end", "missing_numbers"])
result.add(
    "overlaps.csv",
```

```
        [
            {
                "label": label(p, n),
                "claimed_by": ";".join(rows[i]["doc_id"] for i in claims[(p, n)]),
            }
            for p, n in overlap_numbers
        ],
        ["label", "claimed_by"],
    )
    result.add(
        "volumes.csv",
        volumes,
        [
            "volume_id",
            "documents",
            "accepted",
            "held",
            "first_label",
            "last_label",
            "range_note",
        ],
    )
    return result


# Chapter 13: fixed template; all records accounted for, no sending.
def letters(folder: Path, **_) -> Result:
    """Chapter 13: fill a fixed template for each complete request; hold the rest with reasons.


    The template may use only placeholders naming columns of letters.csv
    (other than letter_id) and must contain DRAFT FOR REVIEW. Every inserted
    value is HTML-escaped. Letter files are named letter_<id>.html; identifiers
    that collide case-insensitively are held, and the output writer checks the
    whole set of names before creating the destination.
    """
    fields = [
        "letter_id",
        "matter_id",
        "recipient",
        "meeting_date",
        "meeting_time",
        "time_zone",
        "method",
        "instruction",
        "note",
    ]
    rows = load(folder, "letters.csv", fields)
    known = registry(folder)
    issues = identity_issues(rows, "letter_id")
    template = (folder / "letter_template.txt").read_text(encoding="utf-8")
    manifest = []
    extras = {}
    generated = 0
    filename_spellings = defaultdict(set)
    for row in rows:
        filename_spellings[row["letter_id"].casefold()].add(row["letter_id"])
    allowed = set(fields) - {"letter_id"}
    import string
```

```
  for _, field, format_spec, conversion in string.Formatter().parse(template):
      if field is not None and (field not in allowed or format_spec or conversion):
          raise ValueError("Template contains an unapproved substitution")
  if "DRAFT FOR REVIEW" not in template:
      raise ValueError("Template must retain draft label")
  for i, r in enumerate(rows):
      e = issues[i]
      require_cells(
          r,
          ["recipient", "meeting_date", "meeting_time", "time_zone", "method", "instruction
"],
          e,
      )
      if not re.fullmatch(r"[A-Za-z0-9_-]+", r["letter_id"]):
          e.append("unsafe_letter_id")
      if len(filename_spellings[r["letter_id"].casefold()]) > 1:
          e.append("case_insensitive_filename_collision")
      if r["matter_id"] not in known:
          e.append("unknown_matter")
      if day(r["meeting_date"]) is None:
          e.append("invalid_meeting_date")
      if not re.fullmatch(r"(?:[01][0-9]|2[0-3]):[0-5][0-9]", r["meeting_time"]):
          e.append("invalid_time")
      if r["time_zone"] != "America/Toronto":
          e.append("outside_teaching_time_zone")
      output = "" if e else "letter_" + r["letter_id"] + ".html"
      if not e:
          text = template.format_map(r)
          extras[output] = page(
              "Meeting confirmation", '<div class="letter"><pre>' + esc(text) + "</pre></
div>"
          )
          generated += 1
      manifest.append(
          {
              "source_row": i + 2,
              "letter_id": r["letter_id"],
              "matter_id": r["matter_id"],
              "status": "held" if e else "generated",
              "output": output,
              "issues": ";".join(e),
          }
      )
  result = Result(
      "Draft administrative letter pack",
      {
          "input_rows": len(rows),
          "generated": generated,
          "held": len(rows) - generated,
          "template_sha256": sha(folder / "letter_template.txt"),
      },
      extra=extras,
  )
  result.add(
      "manifest.csv",
      manifest,
      ["source_row", "letter_id", "matter_id", "status", "output", "issues"],
  )
  return result
```

```
# Chapter 14: record completeness is not authentication of a legal deadline.
def supplied_dates(folder: Path, as_of: str = "2026-09-24", **_) -> Result:
    """Chapter 14: classify supplied dates relative to a reporting date, completeness first.

    An entry needs review for a blank or duplicated identifier, a missing
    verifier, source or owner, an unregistered matter, an invalid supplied
    date, a verification date that is missing or after the reporting date,
    unexpected flag values, a second current entry for the same matter and
    task, or a broken supersession chain. Only a usable entry is then
    classified as superseded, recorded_complete, past_recorded_date, today or
    upcoming, in that order.
    """
    fields = [
        "date_id",
        "matter_id",
        "task_id",
        "supplied_date",
        "source_id",
        "verifier",
        "verification_date",
        "owner",
        "completed",
        "record_status",
        "supersedes",
    ]
    rows = load(folder, "dates.csv", fields)
    known = registry(folder)
    cutoff = required_day(as_of)
    issues = identity_issues(rows, "date_id")
    current = Counter(
        (r["matter_id"], r["task_id"]) for r in rows if r["record_status"] == "current"
    )
    idcounts = Counter(r["date_id"] for r in rows)
    by_id = {r["date_id"]: r for r in rows}
    ledger = []
    successors = Counter(r["supersedes"] for r in rows if r["supersedes"])
    cycle_ids = set()
    for ident in by_id:
        chain = []
        current_id = ident
        while current_id in by_id and current_id not in chain:
            chain.append(current_id)
            current_id = by_id[current_id]["supersedes"]
        if current_id in chain:
            cycle_ids.update(chain[chain.index(current_id) :])
    for i, r in enumerate(rows):
        e = issues[i]
        d = day(r["supplied_date"])
        v = day(r["verification_date"])
        require_cells(r, ["task_id", "source_id", "verifier", "owner"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if d is None:
            e.append("invalid_supplied_date")
        if v is None or v > cutoff:
            e.append("invalid_or_future_verification")
        if r["completed"] not in {"yes", "no"}:
            e.append("invalid_completion_flag")
        if r["record_status"] not in {"current", "superseded"}:
```

```
            e.append("invalid_record_status")
        if r["record_status"] == "current" and current[(r["matter_id"], r["task_id"])] > 1:
            e.append("conflicting_current_dates")
        if r["date_id"] in cycle_ids:
            e.append("cyclic_supersession")
        if r["supersedes"] == r["date_id"]:
            e.append("self_supersession")
        if r["record_status"] == "superseded" and successors[r["date_id"]] != 1:
            e.append("missing_or_ambiguous_successor")
        if r["supersedes"]:
            old = by_id.get(r["supersedes"])
            if (
                idcounts[r["supersedes"]] != 1
                or old is None
                or old["record_status"] != "superseded"
                or (old["matter_id"], old["task_id"]) != (r["matter_id"], r["task_id"])
            ):
                e.append("invalid_supersession_link")
        status = (
            "needs_review"
            if e
            else "superseded"
            if r["record_status"] == "superseded"
            else "recorded_complete"
            if r["completed"] == "yes"
            else "past_recorded_date"
            if d < cutoff
            else "today"
            if d == cutoff
            else "upcoming"
        )
        ledger.append(dict(r, classification=status, issues=";".join(e)))
    counts = Counter(r["classification"] for r in ledger)
    result = Result(
        "Professionally supplied date register",
        dict(
            input_rows=len(rows),
            reporting_date=as_of,
            **{
                s: counts[s]
                for s in [
                    "needs_review",
                    "superseded",
                    "recorded_complete",
                    "past_recorded_date",
                    "today",
                    "upcoming",
                ]
            },
        ),
    )
    result.add("date_register.csv", ledger, fields + ["classification", "issues"])
    return result


# Chapter 15: normalized exact/alias/one-edit candidates, never clearance.
def normalize(text: str) -> str:
    """Chapter 15: NFKC compatibility form, casefold, then trim/collapse Unicode whitespace.
  """
    return " ".join(unicodedata.normalize("NFKC", text).casefold().split())
```

```
def one_edit(a: str, b: str) -> bool:
    """Chapter 15: true when a and b differ by exactly one inserted, deleted or substituted
  character."""
    if a == b:
        return False
    if abs(len(a) - len(b)) > 1:
        return False
    if len(a) == len(b):
        return sum(x != y for x, y in zip(a, b)) == 1
    if len(a) > len(b):
        a, b = b, a
    i = j = errors = 0
    while i < len(a) and j < len(b):
        if a[i] == b[j]:
            i += 1
            j += 1
        else:
            errors += 1
            j += 1
        if errors > 1:
            return False
    return True


def parties(folder: Path, query: str = "Alex Morgan", similar: bool = True, **_) -> Result:
    """Chapter 15: list every entity whose primary name or alias matches the query.

    Exact matches compare normalized whole names. With similar=True a name one
    edit away is a possible match. Entities with the same name stay separate.
    The register fingerprint and the scope line are part of every result.
    """
    fields = ["entity_id", "name", "aliases", "role", "matter_id", "source_id"]
    rows = load(folder, "parties.csv", fields)
    if not normalize(query):
        raise ValueError("A nonblank query is required")
    if any(identity_issues(rows, "entity_id")) or any(
        not r["name"].strip() or not r["source_id"].strip() for r in rows
    ):
        raise ValueError("Invalid party register")
    q = normalize(query)
    matches = []
    for r in rows:
        candidates = [("primary_name", r["name"])] + [
            ("alias", s) for s in r["aliases"].split("|") if s.strip()
        ]
        for kind, text in candidates:
            value = normalize(text)
            label = (
                "exact_" + kind
                if q == value
                else "possible_" + kind
                if similar and one_edit(q, value)
                else ""
            )
            if label:
                matches.append(
                    dict(
                        r,
```

```
                        match_type=label,
                        matched_text=text,
                        explanation="normalized exact match"
                        if q == value
                        else "one insertion, deletion, or substitution",
                    )
                )
    result = Result(
        "Name-and-party candidates for review",
        {
            "query": query,
            "matches": len(matches),
            "register_sha256": sha(folder / "parties.csv"),
            "scope": "This register and this rule only; no professional clearance.",
        },
    )
    result.add(
        "candidates.csv", matches, fields + ["match_type", "matched_text", "explanation"]
    )
    return result


# Chapter 16: preserve supplied authority identities and researcher propositions.
def authorities(folder: Path, **_) -> Result:
    """Chapter 16: group retrieved records by (authority_id, version); check the proposition
  worksheet.

    A record is grouped when record_id is present and unique and authority_id,
    version and source_id are present; otherwise it is listed as ungrouped. A
    related authority is displayed, never merged. Each proposition needs a
    source location, a text, a permitted text kind, an authority-version that
    exists in the register, and a reviewer when marked recorded.
    """
    fields = [
        "record_id",
        "authority_id",
        "version",
        "kind",
        "related_authority",
        "relationship",
        "source_id",
        "verification",
        "reviewer",
        "review_date",
    ]
    rows = load(folder, "authorities.csv", fields)
    issues = identity_issues(rows, "record_id")
    membership = []
    held = []
    for i, r in enumerate(rows):
        e = issues[i]
        require_cells(r, ["authority_id", "version", "source_id"], e)
        item = dict(r, issues=";".join(e))
        (held if e else membership).append(item)
    prop_fields = [
        "proposition_id",
        "authority_id",
        "version",
        "source_location",
        "text",
```

```
        "text_kind",
        "reviewer",
        "review_status",
    ]
    props = load(folder, "propositions.csv", prop_fields)
    pissues = identity_issues(props, "proposition_id")
    known = {(r["authority_id"], r["version"]) for r in membership}
    worksheet = []
    for i, r in enumerate(props):
        e = pissues[i]
        require_cells(r, ["source_location", "text", "text_kind"], e)
        if (r["authority_id"], r["version"]) not in known:
            e.append("unknown_authority_version")
        if r["text_kind"] not in {"exact passage", "researcher summary"}:
            e.append("invalid_text_kind")
        if r["review_status"] == "recorded" and not r["reviewer"].strip():
            e.append("missing_reviewer")
        worksheet.append(dict(r, issues=";".join(e)))
    result = Result(
        "Authority and proposition register",
        {
            "records": len(rows),
            "authorities": len({r["authority_id"] for r in membership}),
            "versions": len({(r["authority_id"], r["version"]) for r in membership}),
            "ungrouped": len(held),
        },
    )
    result.add("membership.csv", membership, fields + ["issues"])
    result.add("ungrouped.csv", held, fields + ["issues"])
    result.add("propositions.csv", worksheet, prop_fields + ["issues"])
    return result


# Chapter 17: explicit paragraph IDs separate movement from text differences.
def read_paragraphs(path: Path) -> list[tuple[str, str]]:
    """Chapter 17: read one paragraph per line as (identifier, text), separated by a tab."""
    result = []
    ids = set()
    for line in path.read_text(encoding="utf-8").splitlines():
        if "\t" not in line:
            raise ValueError("Each paragraph needs ID, tab, and complete text")
        ident, text = line.split("\t", 1)
        if not ident.strip() or ident in ids:
            raise ValueError("Paragraph IDs must be unique and nonblank")
        ids.add(ident)
        result.append((ident, text))
    return result


def compare_documents(
    folder: Path, whitespace: bool = False, reverse: bool = False, **_
) -> Result:
    """Chapter 17: compare two paragraph files by identifier, reporting movement separately.

    A paragraph is unchanged, changed, deleted or inserted by exact text
    comparison (or whitespace-normalized with whitespace=True). The changed
    count includes every paragraph that is not unchanged. Word-level marks are
    computed on runs of letters, digits or underscores, punctuation and whitespace runs.
    Status can use normalized whitespace; visible marks always use the original texts.
    """
```

```
    left = folder / ("revised.txt" if reverse else "original.txt")
    right = folder / ("original.txt" if reverse else "revised.txt")
    a = read_paragraphs(left)
    b = read_paragraphs(right)
    aa = dict(a)
    bb = dict(b)
    ai = {k: i for i, (k, _) in enumerate(a)}
    bi = {k: i for i, (k, _) in enumerate(b)}
    rows = []
    diffs = []
    for key in list(aa) + [k for k in bb if k not in aa]:
        old = aa.get(key, "")
        new = bb.get(key, "")
        x = " ".join(old.split()) if whitespace else old
        y = " ".join(new.split()) if whitespace else new
        status = (
            "inserted"
            if key not in aa
            else "deleted"
            if key not in bb
            else "unchanged"
            if x == y
            else "changed"
        )
        moved = key in ai and key in bi and ai[key] != bi[key]
        rows.append(
            {
                "paragraph_id": key,
                "original_position": ai.get(key, -1) + 1,
                "revised_position": bi.get(key, -1) + 1,
                "status": status,
                "position_changed": int(moved),
                "original_text": old,
                "revised_text": new,
            }
        )
        at = re.findall(r"\s+|\w+|[^\w\s]", old)
        bt = re.findall(r"\s+|\w+|[^\w\s]", new)
        left_html = []
        right_html = []
        for tag, i, j, k, l in difflib.SequenceMatcher(
            None, at, bt, autojunk=False
        ).get_opcodes():
            ax = esc("".join(at[i:j]))
            bx = esc("".join(bt[k:l]))
            left_html.append("<del>" + ax + "</del>" if tag in {"delete", "replace"} else ax)
            right_html.append("<ins>" + bx + "</ins>" if tag in {"insert", "replace"} else bx
)
        diffs.append(
            "<h2>"
            + esc(key)
            + " - "
            + esc(status)
            + ("</h2><p>Paragraph position changed.</p>" if moved else "</h2>")
            + "<table><tr><th>Original</th><th>Revised</th></tr><tr><td>"
            + "".join(left_html)
            + "</td><td>"
            + "".join(right_html)
            + "</td></tr></table>"
        )
```

```
    result = Result(
        "Textual document comparison",
        {
            "paragraphs": len(rows),
            "changed": sum(r["status"] != "unchanged" for r in rows),
            "unchanged": sum(r["status"] == "unchanged" for r in rows),
            "position_changes": sum(r["position_changed"] for r in rows),
            "original_file": left.name,
            "revised_file": right.name,
            "original_sha256": sha(left),
            "revised_sha256": sha(right),
            "comparison": "whitespace-normalized" if whitespace else "exact paragraph text",
        },
        body="".join(diffs),
    )
    result.add(
        "comparison.csv",
        rows,
        [
            "paragraph_id",
            "original_position",
            "revised_position",
            "status",
            "position_changed",
            "original_text",
            "revised_text",
        ],
    )
    return result


# Chapter 18: Decimal, half-up per line; sum those rounded lines.
def time_entries(folder: Path, **_) -> Result:
    """Chapter 18: minutes x rate / 60 in Decimal, rounded half up per line; total of rounded
  lines.

    An entry is held, with every reason, for a blank or duplicated identifier,
    an unregistered matter, a blank source, minutes that are not a whole number
    from 0 through 1,000,000 written in digits, a rate outside 0 through
    1,000,000 or with more than four fractional places (ignoring trailing
    fractional zeros), or a currency other than CAD. Rates must be finite
    and nonnegative. Calculation uses a 40-digit local Decimal context.
    A held entry has no amount.
    """
    fields = ["entry_id", "matter_id", "minutes", "hourly_rate", "currency", "source_id"]
    rows = load(folder, "time_entries.csv", fields)
    known = registry(folder)
    issues = identity_issues(rows, "entry_id")
    ledger = []
    total = Decimal("0.00")
    subtotals = defaultdict(lambda: Decimal("0.00"))
    for i, r in enumerate(rows):
        e = issues[i]
        # Compare digit strings before int(): even a very long malformed
        # duration becomes a held row, without Python's integer-digit limit.
        minute_digits = r["minutes"].lstrip("0") or "0"
        n = None
        if not re.fullmatch(r"[0-9]+", r["minutes"]):
            e.append("invalid_minutes")
        elif len(minute_digits) > 7 or (
```

```
            len(minute_digits) == 7 and minute_digits > "1000000"
        ):
            e.append("minutes_outside_teaching_range")
        else:
            n = int(minute_digits)
        require_cells(r, ["source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if r["currency"] != "CAD":
            e.append("wrong_currency")
        try:
            rate = Decimal(r["hourly_rate"])
            if not rate.is_finite() or rate < 0:
                raise InvalidOperation
            if rate > Decimal("1000000"):
                e.append("rate_outside_teaching_range")
            elif rate.is_zero():
                rate = Decimal(0)  # Includes signed zero and large zero exponents.
            else:
                digits = rate.as_tuple().digits
                trailing_zeros = 0
                for digit in reversed(digits):
                    if digit != 0:
                        break
                    trailing_zeros += 1
                fractional_places = max(0, -rate.as_tuple().exponent - trailing_zeros)
                if fractional_places > 4:
                    e.append("rate_precision_exceeds_four_places")
        except InvalidOperation:
            e.append("invalid_rate")
            rate = None
        amount = ""
        if not e:
            # The stated magnitude/scale bounds leave ample guard digits,
            # independent of any Decimal context set by a caller.
            with localcontext(Context(prec=40)):
                value = (Decimal(n) * rate / Decimal(60)).quantize(
                    Decimal(".01"), rounding=ROUND_HALF_UP
                )
                total += value
                subtotals[r["matter_id"]] += value
            amount = f"{value:.2f}"
        ledger.append(
            dict(
                r,
                source_row=i + 2,
                amount=amount,
                status="held" if e else "calculable",
                issues=";".join(e),
            )
        )
    result = Result(
        "Draft time-entry arithmetic worksheet",
        {
            "input_rows": len(rows),
            "accepted": sum(r["status"] == "calculable" for r in ledger),
            "held": sum(r["status"] == "held" for r in ledger),
            "total": f"{total:.2f}",
            "currency": "CAD",
            "rounding": "ROUND_HALF_UP per line; total is sum of rounded lines",
```

```
        },
    )
    result.add("ledger.csv", ledger, fields + ["source_row", "amount", "status", "issues"])
    result.add(
        "matter_totals.csv",
        [{"matter_id": m, "amount": f"{v:.2f}"} for m, v in sorted(subtotals.items())],
        ["matter_id", "amount"],
    )
    return result


# Chapter 19: hold precedence; history is retained, not an instruction to delete.
def closing(folder: Path, as_of: str = "2026-09-24", **_) -> Result:
    """Chapter 19: classify closed files in a fixed order: hold, then needs_review, then dates.


    A hold flag of yes always wins. Otherwise an unrecognised flag, a blank or
    repeated file ID, an unregistered matter, a blank owner or source, a
    closure date missing, invalid or after the reporting date, or a review
    date missing, invalid or before the closure date sends the entry to a
    person. Otherwise a review date on or before the reporting date is due and
    a later one is upcoming. History rows are checked and returned unchanged.
    """
    fields = [
        "file_id",
        "matter_id",
        "closure_date",
        "review_date",
        "hold",
        "owner",
        "source_id",
    ]
    rows = load(folder, "closing.csv", fields)
    known = registry(folder)
    cutoff = required_day(as_of)
    issues = identity_issues(rows, "file_id")
    ledger = []
    for i, r in enumerate(rows):
        e = issues[i]
        c = day(r["closure_date"])
        d = day(r["review_date"])
        require_cells(r, ["owner", "source_id"], e)
        if r["matter_id"] not in known:
            e.append("unknown_matter")
        if r["hold"] not in {"yes", "no"}:
            e.append("unknown_hold_flag")
        if c is None or c > cutoff:
            e.append("invalid_closure_date")
        if d is None or c is not None and d < c:
            e.append("invalid_review_date")
        status = (
            "hold"
            if r["hold"] == "yes"
            else "needs_review"
            if e
            else "due_for_review"
            if d <= cutoff
            else "upcoming"
        )
        ledger.append(dict(r, classification=status, issues=";".join(e)))
```

```
    hfields = [
        "history_id",
        "file_id",
        "review_date",
        "reviewer",
        "source_version",
        "outcome",
        "source_id",
    ]
    history = load(folder, "review_history.csv", hfields)
    if any(identity_issues(history, "history_id")):
        raise ValueError("Duplicate or missing history identity")
    for r in history:
        if (
            day(r["review_date"]) is None
            or not r["reviewer"].strip()
            or not r["source_version"].strip()
        ):
            raise ValueError("Incomplete review-history record")
    counts = Counter(r["classification"] for r in ledger)
    result = Result(
        "File-closing administrative review",
        dict(
            input_rows=len(rows),
            reporting_date=as_of,
            **{s: counts[s] for s in ["hold", "needs_review", "due_for_review", "upcoming"]},
        ),
    )
    result.add("closing_review.csv", ledger, fields + ["classification", "issues"])
    result.add("history.csv", history, hfields)
    return result


# Chapter 22: exactly one required category, explicit versions and dates.
def reviews(
    folder: Path, as_of: str = "2026-09-24", artifact: str = "2", rule: str = "1", **_
) -> Result:
    """Chapter 22: is each required review record complete for the declared delivery?

    Exactly one row per required category; reviewer and source present;
    artifact and rule versions equal to the declared ones; valid review date
    on or before the reporting date and expiry date on or after it (expiry is
    inclusive). A missing category is added as a row with missing_category.
    """
    fields = [
        "category",
        "artifact_version",
        "rule_version",
        "reviewer",
        "review_date",
        "expiry_date",
        "source_id",
    ]
    rows = load(folder, "reviews.csv", fields)
    required = ["specification", "program", "tests", "data handling", "user guide"]
    cutoff = required_day(as_of)
    counts = Counter(r["category"] for r in rows)
    ledger = []
    for i, r in enumerate(rows):
        e = []
```

```
        require_cells(r, ["reviewer", "source_id"], e)
        d = day(r["review_date"])
        x = day(r["expiry_date"])
        if r["category"] not in required:
            e.append("unknown_category")
        if counts[r["category"]] != 1:
            e.append("duplicate_category")
        if r["artifact_version"] != artifact:
            e.append("artifact_version_mismatch")
        if r["rule_version"] != rule:
            e.append("rule_version_mismatch")
        if d is None or x is None or not d <= cutoff <= x:
            e.append("invalid_or_outdated_dates")
        ledger.append(
            dict(
                r,
                source_row=i + 2,
                status="needs_attention" if e else "review_record_complete",
                issues=";".join(e),
            )
        )
    for category in required:
        if counts[category] == 0:
            ledger.append(
                dict.fromkeys(fields, "")
                | {
                    "category": category,
                    "source_row": "",
                    "status": "needs_attention",
                    "issues": "missing_category",
                }
            )
    result = Result(
        "Application-review record check",
        {
            "complete": sum(r["status"] == "review_record_complete" for r in ledger),
            "needs_attention": sum(r["status"] != "review_record_complete" for r in ledger),
            "artifact_version": artifact,
            "rule_version": rule,
            "reporting_date": as_of,
        },
    )
    result.add("review_register.csv", ledger, fields + ["source_row", "status", "issues"])
    return result


# Chapter 23: integrate the matcher without changing its definitions.
def worklist(
    folder: Path, as_of: str = "2026-09-24", window: int = 14, matter: str | None = None, **_
) -> Result:
    """Chapter 23: one status per request plus three evidence fields, using the Chapter 8
  matcher.

    Status order: needs_review, not_yet_requested, response_recorded,
    window_open, window_passed. The owner is copied from the request row.
    With a selected matter the whole file is matched first and the delivered
    tables are then restricted to that matter.
    """
    req = load(folder, "requests.csv", REQUEST_FIELDS)
    resp = load(folder, "responses.csv", RESPONSE_FIELDS)
```

```
    known = registry(folder)
    if matter is not None and matter not in known:
        raise ValueError("Unknown selected matter")
    rows, candidates = match_records(req, resp, known, required_day(as_of), window)
    whole = Counter(r["status"] for r in rows)
    if matter is not None:
        rows = [r for r in rows if r["matter_id"] == matter]
        candidates = [r for r in candidates if r["matter_id"] == matter]
    counts = Counter(r["status"] for r in rows)
    summary = {
        "requests": len(rows),
        "reporting_date": as_of,
        "window_days": window,
        **{
            s: counts[s]
            for s in [
                "response_recorded",
                "window_open",
                "window_passed",
                "needs_review",
                "not_yet_requested",
            ]
        },
        "in_window_yes": sum(r["in_window"] == "yes" for r in rows),
        "ambiguous_earliest": sum(r["earliest"] == "ambiguous" for r in rows),
    }
    result = Result("Integrated matter worklist", summary)
    result.add("worklist.csv", rows, MATCH_FIELDS)
    result.add("candidates.csv", candidates, CANDIDATE_FIELDS)
    # Whole-file counts are an internal result, not part of the selected report.
    result.extra["internal_validation.json"] = (
        json.dumps({"whole_requests": len(req), "whole_status_counts": dict(whole)}, indent
=2)
        + "\n"
    )
    owners = sorted({r["owner"] for r in rows})
    statuses = sorted(counts)
    controls = (
        '<p>These filters hide rows; they do not remove data from this delivered file.</p><
label>Owner <select id="owner"><option value="">All</option>'
        + "".join("<option>" + esc(x) + "</option>" for x in owners)
        + '</select></label> <label>Status <select id="status"><option value="">All</option>'
        + "".join("<option>" + esc(x) + "</option>" for x in statuses)
        + '</select></label><p id="visible"></p>'
    )
    display_fields = [
        "request_id",
        "matter_id",
        "owner",
        "status",
        "any_response",
        "in_window",
        "earliest",
        "selected_response",
    ]
    table = (
        '<table id="work"><thead><tr>'
        + "".join("<th>" + esc(x.replace("_", " ")) + "</th>" for x in display_fields)
        + "</tr></thead><tbody>"
        + "".join(
```

```
            '<tr data-owner="'
            + esc(r["owner"])
            + '" data-status="'
            + esc(r["status"])
            + '">'
            + "".join("<td>" + esc(r[f]) + "</td>" for f in display_fields)
            + "</tr>"
            for r in rows
        )
        + "</tbody></table>"
    )
    script = """<script>const owner=document.getElementById('owner'),status=document.
  getElementById('status');function filter(){let n=0;document.querySelectorAll('#work tbody tr
  ').forEach(r=>{r.hidden=!!((owner.value&&r.dataset.owner!==owner.value)||(status.value&&r.
  dataset.status!==status.value));if(!r.hidden)n++;});document.getElementById('visible').
  textContent=n?n+' matching requests':'No matching rows';}owner.addEventListener('change',
  filter);status.addEventListener('change',filter);filter();</script>"""
    result.body = controls + table + script
    return result


APPLICATIONS = {
    4: consultation,
    5: catalogue,
    6: document_check,
    7: management_extract,
    8: matching,
    9: population,
    10: turnaround,
    11: weekly,
    12: production_index,
    13: letters,
    14: supplied_dates,
    15: parties,
    16: authorities,
    17: compare_documents,
    18: time_entries,
    19: closing,
    22: reviews,
    23: worklist,
}
```

Appendix D

# Large-File Conversion

programs/convert_matter_events.py

```
"""Convert event records into requests and responses using supplied assignments.

Retains a conversion ledger for every source event. Notes are deliberately
outside this row unit. Held source events are never silently converted.
"""

from __future__ import annotations
import argparse
import json
from pathlib import Path
from matter_tools import load_csv, check, write_csv, EVENT_FIELDS, MATTER_FIELDS
from legal_common import REQUEST_FIELDS, RESPONSE_FIELDS, sha


def convert(
    events_path: Path, matters_path: Path, assignments_path: Path, output: Path
) -> dict:
    if output.exists() or output.is_symlink():
        raise FileExistsError("Choose a new conversion folder")
    events = load_csv(events_path, EVENT_FIELDS)
    matters = load_csv(matters_path, MATTER_FIELDS)
    assignments = load_csv(assignments_path, ["matter_id", "owner"])
    known = {r["matter_id"] for r in matters}
    owners = {r["matter_id"]: r["owner"] for r in assignments}
    if (
        len(owners) != len(assignments)
        or set(owners) != known
        or any(not v.strip() for v in owners.values())
    ):
        raise ValueError(
            "Assignments must supply exactly one nonblank owner for every registered matter"
        )
    accepted, held = check(events, matters)
    by_position = {r["source_row"]: r for r in accepted}
    held_by = {r["source_row"]: r for r in held}
    requests = []
    responses = []
    ledger = []
    for position, r in enumerate(events, 2):
        if position in held_by:
            disposition = "held"
            reason = held_by[position]["reasons"]
        elif r["event_type"] == "request":
            disposition = "request"
            reason = "converted"
            requests.append(
```

```
                {
                    "request_id": r["request_id"],
                    "matter_id": r["matter_id"],
                    "owner": owners[r["matter_id"]],
                    "request_date": r["event_date"],
                    "source_id": r["source_id"],
                }
            )
        elif r["event_type"] == "response":
            disposition = "response"
            reason = "converted"
            responses.append(
                {
                    "response_id": r["event_id"],
                    "matter_id": r["matter_id"],
                    "request_id": r["request_id"],
                    "response_date": r["event_date"],
                    "source_id": r["source_id"],
                }
            )
        else:
            disposition = "note"
            reason = "outside_request_response_row_unit"
        ledger.append(
            {
                "source_row": position,
                "event_id": r["event_id"],
                "matter_id": r["matter_id"],
                "disposition": disposition,
                "reason": reason,
            }
        )
    output.mkdir(parents=True, exist_ok=False)
    write_csv(output / "requests.csv", requests, REQUEST_FIELDS)
    write_csv(output / "responses.csv", responses, RESPONSE_FIELDS)
    write_csv(output / "matters.csv", matters, MATTER_FIELDS)
    write_csv(
        output / "conversion_ledger.csv",
        ledger,
        ["source_row", "event_id", "matter_id", "disposition", "reason"],
    )
    summary = {
        "source_events": len(events),
        "requests": len(requests),
        "responses": len(responses),
        "notes": sum(r["disposition"] == "note" for r in ledger),
        "held": len(held),
        "input_hashes": {p.name: sha(p) for p in [events_path, matters_path, assignments_path
 ]},
    }
    (output / "conversion.json").write_text(
        json.dumps(summary, indent=2, sort_keys=True) + "\n"
    )
    return summary


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--events", type=Path, required=True)
    parser.add_argument("--matters", type=Path, required=True)
```

```
    parser.add_argument("--assignments", type=Path, required=True)
    parser.add_argument("--out", type=Path, required=True)
    a = parser.parse_args()
    try:
        result = convert(a.events, a.matters, a.assignments, a.out)
    except (ValueError, OSError) as exc:
        parser.exit(2, str(exc) + "\n")
    print(json.dumps(result, indent=2, sort_keys=True))


if __name__ == "__main__":
    main()
```

Appendix E

# Regression Tests and Expectations

expected/baselines.json

```
{
  "4": {
    "input_rows": 8,
    "numerator": 3,
    "denominator": 6,
    "percentage": 50.0
  },
  "5": {
    "file_records": 8,
    "documents": 5,
    "versions": 7
  },
  "6": {
    "input_rows": 8,
    "accepted": 3,
    "held": 5
  },
  "7": {
    "input_rows": 8,
    "output_rows": 8,
    "total_count": 8
  },
  "8": {
    "requests": 4,
    "responses": 7,
    "matched": 2,
    "ambiguous": 0
  },
  "9": {
    "included": 3,
    "excluded": 3,
    "review": 2,
    "observations": 2,
    "mean": 6.0,
    "median": 6.0
  },
  "10": {
    "observations": 16,
    "months": 12
  },
  "11": {
    "eligible_final": 90,
    "responded_final": 46
```

```
  },
  "12": {
    "input_rows": 10,
    "accepted": 6,
    "held": 4,
    "accepted_pages": 17,
    "gap_numbers": 2,
    "overlap_numbers": 1
  },
  "13": {
    "input_rows": 6,
    "generated": 4,
    "held": 2
  },
  "14": {
    "input_rows": 6,
    "needs_review": 2,
    "upcoming": 1,
    "today": 1,
    "past_recorded_date": 1,
    "recorded_complete": 1
  },
  "15": {
    "matches": 2
  },
  "16": {
    "records": 8,
    "authorities": 5,
    "versions": 6,
    "ungrouped": 1
  },
  "17": {
    "paragraphs": 4,
    "changed": 3,
    "unchanged": 1
  },
  "18": {
    "input_rows": 6,
    "accepted": 4,
    "held": 2,
    "total": "575.00"
  },
  "19": {
    "input_rows": 5,
    "due_for_review": 1,
    "upcoming": 1,
    "hold": 1,
    "needs_review": 2
  },
  "22": {
    "complete": 3,
    "needs_attention": 2
  },
  "23": {
    "requests": 6,
    "response_recorded": 2,
    "window_open": 2,
    "window_passed": 1,
    "needs_review": 1,
    "not_yet_requested": 0,
```

```
    "in_window_yes": 1
  }
}
```

programs/test_matter_tools.py

```
"""Regression tests for the synthetic opening project. No network access."""

from __future__ import annotations
import csv
import hashlib
import json
import tempfile
import unittest
from copy import deepcopy
from pathlib import Path
from matter_tools import (
    EVENT_FIELDS,
    MATTER_FIELDS,
    check,
    load_csv,
    run,
    valid_date,
    write_csv,
)

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data"


class MatterTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.clean = load_csv(DATA / "matter_events_demo.csv", EVENT_FIELDS)
        cls.matters = load_csv(DATA / "matters_demo.csv", MATTER_FIELDS)
        cls.problem = load_csv(DATA / "matter_events_problem.csv", EVENT_FIELDS)
        cls.large = load_csv(DATA / "matter_events_large.csv", EVENT_FIELDS)
        cls.large_matters = load_csv(DATA / "matters_large.csv", MATTER_FIELDS)

    def setUp(self):
        self.temp = tempfile.TemporaryDirectory(prefix="legal-test-")
        self.addCleanup(self.temp.cleanup)
        self.work = Path(self.temp.name)

    def small_run(self, selected=None, events=None, output="out"):
        ep = self.work / "events.csv"
        mp = self.work / "matters.csv"
        write_csv(ep, self.clean if events is None else events, EVENT_FIELDS)
        write_csv(mp, self.matters, MATTER_FIELDS)
        return run(ep, mp, self.work / output, selected)

    def test_01_clean_counts_and_identities(self):
        accepted, held = check(self.clean, self.matters)
        self.assertEqual((len(accepted), len(held)), (18, 0))
        self.assertEqual({r["event_id"] for r in accepted}, {f"E{i:02}" for i in range(1, 19)
  })

    def test_02_problem_counts_and_source_positions(self):
        accepted, held = check(self.problem, self.matters)
```

```
        self.assertEqual((len(accepted), len(held)), (11, 7))
        self.assertEqual({r["source_row"] for r in held}, {3, 6, 9, 11, 12, 15, 18})
        self.assertEqual(sum(r["event_id"] == "E10" for r in held), 2)

    def test_03_reversed_response_is_held(self):
        rows = deepcopy(self.clean)
        rows[1]["event_date"] = "2026-08-31"
        accepted, held = check(rows, self.matters)
        self.assertEqual((len(accepted), len(held)), (17, 1))
        self.assertIn("response_before_request", held[0]["reasons"])

    def test_04_same_day_response_allowed(self):
        rows = deepcopy(self.clean)
        rows[1]["event_date"] = rows[0]["event_date"]
        self.assertEqual(len(check(rows, self.matters)[0]), 18)

    def test_05_invalid_date(self):
        for value in ["2026-02-30", "2026-9-01", "", "2026-13-01", " 2026-09-01"]:
            self.assertFalse(valid_date(value), value)
        self.assertTrue(valid_date("2024-02-29"))

    def test_06_wrong_client(self):
        rows = deepcopy(self.clean)
        rows[2]["client_id"] = "C99"
        accepted, held = check(rows, self.matters)
        self.assertEqual(len(held), 1)
        self.assertIn("client_matter_mismatch", held[0]["reasons"])

    def test_07_bad_request_holds_dependent_response(self):
        rows = deepcopy(self.clean)
        rows[0]["source_id"] = ""
        accepted, held = check(rows, self.matters)
        self.assertEqual({r["event_id"] for r in held}, {"E01", "E02"})
        self.assertIn("request_needs_review", held[1]["reasons"])

    def test_08_duplicate_request_pair(self):
        rows = deepcopy(self.clean)
        extra = dict(rows[0], event_id="E99")
        rows.append(extra)
        accepted, held = check(rows, self.matters)
        self.assertEqual((len(accepted), len(held)), (16, 3))

    def test_09_request_ids_can_repeat_in_different_matters(self):
        rows = deepcopy(self.clean)
        rows[3]["request_id"] = "R01"
        rows[4]["request_id"] = "R01"
        self.assertEqual(len(check(rows, self.matters)[0]), 18)

    def test_10_multiple_responses_allowed(self):
        rows = deepcopy(self.clean)
        rows.append(dict(rows[1], event_id="E99", source_id="SRC99"))
        self.assertEqual(len(check(rows, self.matters)[0]), 19)

    def test_11_note_cannot_carry_request_id(self):
        rows = deepcopy(self.clean)
        rows[2]["request_id"] = "R01"
        self.assertIn("note_has_request_id", check(rows, self.matters)[1][0]["reasons"])

    def test_12_duplicate_matter_register_stops(self):
        with self.assertRaises(ValueError):
```

```
            check(self.clean, self.matters + [self.matters[0]])

    def test_13_blank_matter_register_identifier_stops(self):
        matters = deepcopy(self.matters)
        matters[0]["client_id"] = " "
        with self.assertRaises(ValueError):
            check(self.clean, matters)

    def test_14_bad_headers_stop(self):
        path = self.work / "bad.csv"
        path.write_text("wrong,columns\na,b\n")
        with self.assertRaises(ValueError):
            load_csv(path, EVENT_FIELDS)

    def test_15_malformed_csv_record_stops(self):
        path = self.work / "bad.csv"
        path.write_text(",".join(EVENT_FIELDS) + "\nx,y\n")
        with self.assertRaises(ValueError):
            load_csv(path, EVENT_FIELDS)

    def test_16_selection_has_only_named_matter(self):
        result = self.small_run("M01")
        self.assertEqual((result["input_rows"], result["selected_accepted"]), (18, 3))
        rows = load_csv(self.work / "out" / "accepted.csv", EVENT_FIELDS + ["source_row"])
        self.assertEqual([r["event_id"] for r in rows], ["E01", "E02", "E03"])
        page = (self.work / "out" / "report.html").read_text()
        self.assertNotIn("M02", page)
        self.assertNotIn("SRC04", page)

    def test_17_unknown_selection_stops_without_output(self):
        with self.assertRaises(ValueError):
            self.small_run("M77")
        self.assertFalse((self.work / "out").exists())

    def test_18_output_protection(self):
        self.small_run()
        before = {p.name: p.read_bytes() for p in (self.work / "out").iterdir()}
        with self.assertRaises(FileExistsError):
            self.small_run()
        self.assertEqual(
            before, {p.name: p.read_bytes() for p in (self.work / "out").iterdir()}
        )

    def test_19_html_source_text_is_escaped(self):
        rows = deepcopy(self.clean)
        payload = '</script><script>alert("canary-83")</script>'
        rows[2]["summary"] = payload
        self.small_run(events=rows)
        page = (self.work / "out" / "report.html").read_text()
        self.assertNotIn(payload, page)
        self.assertIn("&lt;/script&gt;", page)
        self.assertIn("\\u003c/script\\u003e", page)

    def test_20_input_files_preserved(self):
        ep = self.work / "events.csv"
        mp = self.work / "matters.csv"
        write_csv(ep, self.clean, EVENT_FIELDS)
        write_csv(mp, self.matters, MATTER_FIELDS)
        original = (ep.read_bytes(), mp.read_bytes())
        run(ep, mp, self.work / "out")
```

```
        self.assertEqual(original, (ep.read_bytes(), mp.read_bytes()))

    def test_21_empty_event_input_accounted_for(self):
        result = self.small_run(events=[])
        self.assertEqual(
            (result["input_rows"], result["accepted_rows"], result["held_rows"]), (0, 0, 0)
        )
        self.assertTrue((self.work / "out" / "report.html").is_file())

    def test_22_quoted_multiline_field_preserved(self):
        rows = deepcopy(self.clean)
        rows[2]["summary"] = "First line, with comma\nSecond line"
        self.small_run(events=rows)
        actual = load_csv(self.work / "out" / "accepted.csv", EVENT_FIELDS + ["source_row"])
        self.assertEqual(actual[2]["summary"], rows[2]["summary"])
        self.assertEqual(actual[3]["source_row"], "5")

    def test_23_large_totals(self):
        accepted, held = check(self.large, self.large_matters)
        self.assertEqual((len(accepted), len(held)), (12000, 0))
        self.assertEqual(len({r["client_id"] for r in accepted}), 1000)
        self.assertEqual(len({r["matter_id"] for r in accepted}), 3000)
        counts = {
            k: sum(r["event_type"] == k for r in accepted)
            for k in ["request", "response", "note"]
        }
        self.assertEqual(counts, {"request": 3000, "response": 3000, "note": 6000})

    def test_24_large_selected_identities(self):
        result = run(
            DATA / "matter_events_large.csv",
            DATA / "matters_large.csv",
            self.work / "out",
            "M0001",
        )
        self.assertEqual(result["selected_accepted"], 4)
        rows = load_csv(self.work / "out" / "accepted.csv", EVENT_FIELDS + ["source_row"])
        self.assertEqual(
            [r["event_id"] for r in rows], ["E00001", "E00002", "E00003", "E00004"]
        )

    def test_25_global_validation_before_selection(self):
        rows = deepcopy(self.large)
        rows[6]["client_id"] = "C9999"
        ep = self.work / "large.csv"
        write_csv(ep, rows, EVENT_FIELDS)
        result = run(ep, DATA / "matters_large.csv", self.work / "out", "M0001")
        self.assertEqual(
            (result["accepted_rows"], result["held_rows"], result["selected_accepted"]),
            (11999, 1, 4),
        )

    def test_26_unknown_matter_visible_in_full_review(self):
        self.small_run(events=self.problem)
        page = (self.work / "out" / "report.html").read_text()
        self.assertIn("<option>M99</option>", page)

    def test_27_repeat_run_bytes(self):
        self.small_run(output="one")
        self.small_run(output="two")
```

```
        first = {p.name: p.read_bytes() for p in (self.work / "one").iterdir()}
        second = {p.name: p.read_bytes() for p in (self.work / "two").iterdir()}
        self.assertEqual(first, second)

    def test_28_blank_summary_held(self):
        rows = deepcopy(self.clean)
        rows[2]["summary"] = "  "
        accepted, held = check(rows, self.matters)
        self.assertEqual(len(held), 1)
        self.assertIn("missing_summary", held[0]["reasons"])


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

programs/test_legal_apps.py

```
"""Independent expectations and boundary tests for the later reference apps."""

from __future__ import annotations
import copy
import csv
import json
import shutil
import tempfile
import unittest
from datetime import date
from pathlib import Path
from legal_apps import *
from legal_common import save_result, sha
from convert_matter_events import convert

ROOT = Path(__file__).resolve().parents[1]


class LegalTests(unittest.TestCase):
    def setUp(self):
        self.temp = tempfile.TemporaryDirectory()
        self.addCleanup(self.temp.cleanup)
        self.root = Path(self.temp.name)

    def fixture(self, ch):
        dest = self.root / f"ch{ch:02}"
        shutil.copytree(ROOT / "data" / f"ch{ch:02}", dest)
        return dest

    def rows(self, path):
        with path.open(newline="", encoding="utf-8") as f:
            return list(csv.DictReader(f))

    def change(self, path, modify):
        rows = self.rows(path)
        fields = list(rows[0])
        modify(rows)
        with path.open("w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=fields, lineterminator="\n")
            w.writeheader()
            w.writerows(rows)
```

```
    def test_01_all_hand_derived_baselines(self):
        expectations = json.loads((ROOT / "expected/baselines.json").read_text())
        for ch, expected in expectations.items():
            with self.subTest(chapter=ch):
                actual = APPLICATIONS[int(ch)](ROOT / "data" / f"ch{int(ch):02}").summary
                for key, value in expected.items():
                    if isinstance(value, float):
                        self.assertAlmostEqual(actual[key], value)
                    else:
                        self.assertEqual(actual[key], value, key)

    def test_02_completion_identities(self):
        r = consultation(ROOT / "data/ch04")
        rows = r.tables["ledger.csv"][1]
        self.assertEqual(
            [x["appointment_id"] for x in rows if x["numerator"]], ["A01", "A02", "A06"]
        )
        self.assertEqual(
            [x["appointment_id"] for x in rows if x["eligible"]],
            ["A01", "A02", "A04", "A05", "A06", "A07"],
        )

    def test_03_completion_repair(self):
        p = self.fixture(4)
        self.change(
            p / "appointments.csv", lambda rr: rr[4].update(completion_date="2026-09-05")
        )
        r = consultation(p)
        self.assertEqual(r.summary["numerator"], 4)
        self.assertEqual(r.summary["denominator"], 6)

    def test_04_completion_cancelled_only(self):
        p = self.fixture(4)
        self.change(
            p / "appointments.csv",
            lambda rr: rr.__setitem__(
                slice(None), [r for r in rr if r["status"] == "cancelled"]
            ),
        )
        self.assertIsNone(consultation(p).summary["percentage"])

    def test_05_completion_unknown_status_suppresses_rate(self):
        p = self.fixture(4)
        self.change(p / "appointments.csv", lambda rr: rr[0].update(status="maybe"))
        self.assertIsNone(consultation(p).summary["percentage"])

    def test_06_catalogue_copy_is_still_a_file(self):
        p = self.fixture(5)
        self.change(
            p / "documents.csv",
            lambda rr: rr.append(dict(rr[1], file_id="F09", source_id="S09")),
        )
        r = catalogue(p)
        self.assertEqual(
            [r.summary[k] for k in ["file_records", "documents", "versions"]], [9, 5, 7]
        )

    def test_07_catalogue_title_not_identity(self):
        p = self.fixture(5)
        self.change(p / "documents.csv", lambda rr: rr[4].update(title=rr[0]["title"]))
```

```
        self.assertEqual(catalogue(p).summary["documents"], 5)

    def test_08_document_review_positions(self):
        r = document_check(ROOT / "data/ch06")
        held = r.tables["review.csv"][1]
        self.assertEqual([r["source_row"] for r in held], [4, 5, 6, 7, 9])

    def test_09_document_one_repair(self):
        p = self.fixture(6)
        self.change(p / "documents.csv", lambda rr: rr[2].update(source_id="S03"))
        r = document_check(p)
        self.assertEqual((r.summary["accepted"], r.summary["held"]), (4, 4))

    def test_10_document_contradiction_preserved(self):
        p = self.fixture(6)
        shutil.copyfile(p / "documents_clean.csv", p / "documents.csv")
        self.change(p / "documents.csv", lambda rr: rr[2].update(type="recorded as executed")
)
        r = document_check(p)
        self.assertEqual(r.summary["consistency_rows"], 2)
        self.assertEqual(r.summary["accepted"], 8)

    def test_11_extract_exact_output_columns_and_canary(self):
        p = ROOT / "data/ch07"
        r = management_extract(p)
        out = self.root / "export"
        save_result(r, p, out, {"chapter": 7})
        self.assertEqual(r.tables["extract.csv"][0], ["week_beginning", "status", "count"])
        for path in out.iterdir():
            self.assertNotIn("PRIVATE-NOTE-X83", path.read_text())
            self.assertNotIn("Example recipient", path.read_text())

    def test_12_extract_weeks(self):
        rr = management_extract(ROOT / "data/ch07").tables["extract.csv"][1]
        self.assertEqual([r["count"] for r in rr], [4, 1, 1, 0, 0, 1, 0, 1])

    def test_13_extract_rejects_unknown_status(self):
        p = self.fixture(7)
        self.change(p / "appointments.csv", lambda rr: rr[0].update(status="other"))
        with self.assertRaises(ValueError):
            management_extract(p)

    def test_14_matching_baseline_identities(self):
        rr = matching(ROOT / "data/ch08").tables["matches.csv"][1]
        self.assertEqual(
            [(r["request_id"], r["selected_response"]) for r in rr],
            [("R01", "S01"), ("R02", ""), ("R03", "S04"), ("R04", "")],
        )

    def test_15_matching_day14_and_day15(self):
        rr = matching(ROOT / "data/ch08").tables["candidates.csv"][1]
        self.assertEqual((rr[1]["eligible"], rr[2]["eligible"]), (1, 0))

    def test_16_matching_window21(self):
        r = matching(ROOT / "data/ch08", window=21)
        self.assertEqual(r.summary["matched"], 3)
        self.assertEqual(r.tables["matches.csv"][1][1]["selected_response"], "S03")

    def test_17_matching_tie(self):
        p = self.fixture(8)
```

```
        self.change(p / "responses.csv", lambda rr: rr.append(dict(rr[3], response_id="S08"))
)
        r = matching(p)
        rr = r.tables["matches.csv"][1][2]
        self.assertEqual(
            (rr["any_response"], rr["in_window"], rr["earliest"], rr["selected_response"]),
            ("yes", "yes", "ambiguous", ""),
        )

    def test_18_matching_wrong_matter_key(self):
        r = matching(ROOT / "data/ch08")
        rr = r.tables["candidates.csv"][1][4]
        self.assertEqual(rr["explanation"], "unknown_request_key")
        self.assertFalse(rr["eligible"])

    def test_19_matching_duplicate_request_pair(self):
        p = self.fixture(8)
        self.change(p / "requests.csv", lambda rr: rr.append(dict(rr[0])))
        r = matching(p)
        self.assertEqual(r.summary["matched"], 1)
        self.assertTrue(
            all(
                "duplicate_request_key" in x["issues"]
                for x in r.tables["matches.csv"][1]
                if x["request_id"] == "R01"
            )
        )

    def test_20_matching_duplicate_response_id(self):
        p = self.fixture(8)
        self.change(p / "responses.csv", lambda rr: rr.append(dict(rr[0])))
        r = matching(p)
        candidate = r.tables["candidates.csv"][1]
        self.assertEqual(sum("duplicate_response_id" in x["explanation"] for x in candidate),
 2)
        self.assertEqual(r.tables["matches.csv"][1][0]["selected_response"], "S02")

    def test_21_matching_request_dependency(self):
        p = self.fixture(8)
        self.change(p / "requests.csv", lambda rr: rr[0].update(source_id=""))
        r = matching(p)
        self.assertEqual(r.tables["matches.csv"][1][0]["in_window"], "unavailable")
        self.assertIn("request_needs_review", r.tables["candidates.csv"][1][0]["explanation"
])

    def test_22_matching_future_response_excluded(self):
        r = matching(ROOT / "data/ch08", as_of="2026-09-05")
        self.assertEqual(r.summary["matched"], 1)
        self.assertEqual(r.tables["candidates.csv"][1][3]["explanation"], "after_as_of")

    def test_23_population_ids_and_missing_observation(self):
        r = population(ROOT / "data/ch09")
        rr = r.tables["population.csv"][1]
        self.assertEqual(
            [x["matter_id"] for x in rr if x["classification"] == "included"],
            ["M01", "M03", "M06"],
        )
        self.assertEqual(r.summary["missing_or_invalid_observations"], 1)

    def test_24_population_later_date(self):
```

```
        self.assertEqual(
            population(ROOT / "data/ch09", as_of="2026-10-02").summary["included"], 4
        )

    def test_25_turnaround_actual_dates_and_gap(self):
        r = turnaround(ROOT / "data/ch10")
        rr = r.tables["monthly.csv"][1]
        self.assertEqual(
            [rr[0]["mean"], rr[0]["median"], rr[1]["mean"], rr[1]["median"]], [8, 8, 8, 2]
        )
        self.assertEqual((rr[3]["count"], rr[3]["mean"], rr[3]["median"]), (0, "", ""))

    def test_26_turnaround_april_added(self):
        p = self.fixture(10)
        self.change(
            p / "pairs.csv",
            lambda rr: rr.append(
                dict(
                    rr[0],
                    pair_id="APRIL",
                    request_id="APRIL",
                    request_date="2026-04-01",
                    response_date="2026-04-10",
                    elapsed_days="9",
                )
            ),
        )
        r = turnaround(p)
        self.assertEqual(r.tables["monthly.csv"][1][3]["median"], 9)

    def test_27_turnaround_disagreement_held(self):
        p = self.fixture(10)
        self.change(p / "pairs.csv", lambda rr: rr[0].update(elapsed_days="400"))
        self.assertEqual(turnaround(p).summary["observations"], 15)

    def test_28_weekly_finalize_one(self):
        p = self.fixture(11)
        self.change(p / "weeks.csv", lambda rr: rr[10].update(status="final"))
        r = weekly(p)
        self.assertEqual(
            (
                r.summary["eligible_final"],
                r.summary["responded_final"],
                r.summary["pooled_percentage"],
            ),
            (100, 54, 54),
        )

    def test_29_weekly_zero_denominators(self):
        p = self.fixture(11)
        self.change(
            p / "weeks.csv", lambda rr: [r.update(eligible="0", responded="0") for r in rr]
        )
        self.assertIsNone(weekly(p).summary["pooled_percentage"])

    def test_30_weekly_invalid_counts_suppress(self):
        p = self.fixture(11)
        self.change(p / "weeks.csv", lambda rr: rr[0].update(responded="3"))
        self.assertIsNone(weekly(p).summary["pooled_percentage"])
```

```
    def test_31_production_index_identities(self):
        r = production_index(ROOT / "data/ch12")
        rr = r.tables["index.csv"][1]
        self.assertEqual(
            [x["doc_id"] for x in rr if x["status"] == "held"], ["D05", "D06", "D07", "D08"]
        )
        self.assertEqual(
            r.tables["gaps.csv"][1],
            [
                {
                    "prefix": "ABC",
                    "gap_start": "ABC000013",
                    "gap_end": "ABC000014",
                    "missing_numbers": 2,
                }
            ],
        )
        self.assertEqual(
            r.tables["overlaps.csv"][1], [{"label": "ABC000016", "claimed_by": "D05;D06"}]
        )
        # The tempting reconciliation agrees by coincidence: 26 supplied pages, 26-number span.

        self.assertEqual(
            (r.summary["supplied_page_count_total"], r.summary["claimed_numbers"]), (26, 24)
        )

    def test_32_production_repair_overlap_keeps_gap(self):
        p = self.fixture(12)
        self.change(
            p / "production.csv",
            lambda rr: rr[5].update(bates_start="ABC000017", page_count="2"),
        )
        r = production_index(p)
        self.assertEqual(
            (r.summary["accepted"], r.summary["overlap_numbers"], r.summary["gap_numbers"]),
            (8, 0, 2),
        )

    def test_32b_production_malformed_label_and_prefix(self):
        p = self.fixture(12)
        self.change(
            p / "production.csv",
            lambda rr: (
                rr[0].update(bates_end="ABC00004"),
                rr[2].update(bates_end="XYZ000007"),
            ),
        )
        rr = production_index(p).tables["index.csv"][1]
        held = {x["doc_id"]: x["issues"] for x in rr if x["status"] == "held"}
        self.assertIn("invalid_bates_label", held["D01"])
        self.assertIn("prefix_mismatch", held["D03"])

    def test_33_letters_held_and_escaped(self):
        r = letters(ROOT / "data/ch13")
        self.assertEqual(
            set(r.extra),
            {"letter_L01.html", "letter_L02.html", "letter_L04.html", "letter_L06.html"},
        )
        self.assertIn("A &amp;amp; B Example", r.extra["letter_L02.html"])
        self.assertIn("&amp;lt;review&amp;gt;", r.extra["letter_L02.html"])
```

```
    def test_34_letters_path_rejected(self):
        p = self.fixture(13)
        self.change(p / "letters.csv", lambda rr: rr[0].update(letter_id="../outside"))
        r = letters(p)
        self.assertEqual(r.summary["generated"], 3)
        self.assertFalse(any("/" in s for s in r.extra))

    def test_35_letters_template_not_code(self):
        p = self.fixture(13)
        (p / "letter_template.txt").write_text("DRAFT FOR REVIEW {recipient.__class__}")
        with self.assertRaises(ValueError):
            letters(p)

    def test_36_dates_future_verification(self):
        p = self.fixture(14)
        self.change(p / "dates.csv", lambda rr: rr[0].update(verification_date="2026-09-25"))
        self.assertEqual(supplied_dates(p).summary["needs_review"], 3)

    def test_37_dates_invalid_flag(self):
        p = self.fixture(14)
        self.change(p / "dates.csv", lambda rr: rr[5].update(completed="maybe"))
        self.assertEqual(supplied_dates(p).summary["needs_review"], 3)

    def test_38_dates_supersession(self):
        p = self.fixture(14)

        def modify(rr):
            rr[0]["record_status"] = "superseded"
            rr.append(
                dict(
                    rr[0],
                    date_id="D07",
                    supplied_date="2026-09-29",
                    record_status="current",
                    supersedes="D01",
                )
            )

        self.change(p / "dates.csv", modify)
        r = supplied_dates(p)
        self.assertEqual(r.summary["superseded"], 1)
        self.assertEqual(r.summary["upcoming"], 1)

    def test_39_dates_conflicting_current(self):
        p = self.fixture(14)
        self.change(
            p / "dates.csv",
            lambda rr: rr.append(dict(rr[0], date_id="D07", supplied_date="2026-09-29")),
        )
        r = supplied_dates(p)
        self.assertEqual(r.summary["needs_review"], 4)

    def test_40_dates_boundary(self):
        r = supplied_dates(ROOT / "data/ch14", as_of="2026-09-25")
        self.assertEqual((r.summary["today"], r.summary["past_recorded_date"]), (1, 2))

    def test_41_party_alias(self):
        rr = parties(ROOT / "data/ch15", query="A. Morgan", similar=False).tables[
            "candidates.csv"
```

```
        ][1]
        self.assertEqual(
            [(r["entity_id"], r["match_type"]) for r in rr], [("NO1", "exact_alias")]
        )

    def test_42_party_similar_and_exact_only(self):
        self.assertEqual(parties(ROOT / "data/ch15", query="Alex Morgen").summary["matches"],
 2)
        self.assertEqual(
            parties(ROOT / "data/ch15", query="Alex Morgen", similar=False).summary["matches"
],
            0,
        )

    def test_43_party_distinct_entities(self):
        rr = parties(ROOT / "data/ch15").tables["candidates.csv"][1]
        self.assertEqual([r["entity_id"] for r in rr], ["NO1", "NO2"])

    def test_44_one_edit_boundaries(self):
        for a, b, expected in [
            ("a", "ab", True),
            ("a", "b", True),
            ("a", "a", False),
            ("a", "abc", False),
            ("ab", "ba", False),
            ("abc", "ac", True),
        ]:
            self.assertEqual(one_edit(a, b), expected)

    def test_45_authority_identity_and_pending(self):
        r = authorities(ROOT / "data/ch16")
        rr = r.tables["membership.csv"][1]
        self.assertEqual(
            {x["authority_id"] for x in rr}, {"DEC01", "DEC02", "LEG01", "COM01", "DEC03"}
        )
        self.assertEqual(
            [x["record_id"] for x in rr if x["verification"] == "pending"], ["Q07"]
        )

    def test_46_proposition_precise_location(self):
        p = self.fixture(16)
        self.change(p / "propositions.csv", lambda rr: rr[0].update(source_location=""))
        rr = authorities(p).tables["propositions.csv"][1]
        self.assertIn("missing_source_location", rr[0]["issues"])

    def test_47_comparison_negation(self):
        r = compare_documents(ROOT / "data/ch17")
        self.assertRegex(r.body, r"<del>\s*not\s*</del>")
        self.assertEqual(r.tables["comparison.csv"][1][3]["status"], "unchanged")

    def test_48_comparison_reverse(self):
        r = compare_documents(ROOT / "data/ch17", reverse=True)
        self.assertRegex(r.body, r"<ins>\s*not\s*</ins>")
        self.assertEqual(r.summary["original_file"], "revised.txt")

    def test_49_comparison_identical(self):
        p = self.fixture(17)
        shutil.copyfile(p / "original.txt", p / "revised.txt")
        self.assertEqual(compare_documents(p).summary["changed"], 0)
```

```
    def test_50_comparison_moved_paragraph(self):
        p = self.fixture(17)
        lines = (p / "original.txt").read_text().splitlines()
        (p / "revised.txt").write_text("\n".join([lines[1], lines[0], *lines[2:]]) + "\n")
        r = compare_documents(p)
        self.assertEqual((r.summary["changed"], r.summary["position_changes"]), (0, 2))

    def test_51_comparison_whitespace(self):
        p = self.fixture(17)
        text = (p / "original.txt").read_text().replace("shall not", "shall  not")
        (p / "revised.txt").write_text(text)
        self.assertEqual(compare_documents(p).summary["changed"], 1)
        self.assertEqual(compare_documents(p, whitespace=True).summary["changed"], 0)

    def test_52_comparison_repeated_word(self):
        p = self.fixture(17)
        (p / "original.txt").write_text("P01\tkeep keep copy\n")
        (p / "revised.txt").write_text("P01\tkeep copy\n")
        self.assertEqual(compare_documents(p).summary["changed"], 1)
        self.assertIn("<del>", compare_documents(p).body)

    def test_53_time_half_up_boundary(self):
        p = self.fixture(18)
        self.change(
            p / "time_entries.csv",
            lambda rr: rr.__setitem__(
                slice(None), [dict(rr[0], minutes="1", hourly_rate="240.30")]
            ),
        )
        self.assertEqual(time_entries(p).summary["total"], "4.01")

    def test_54_time_total_of_rounded_lines(self):
        p = self.fixture(18)
        self.change(
            p / "time_entries.csv",
            lambda rr: rr.__setitem__(
                slice(None),
                [
                    dict(rr[0], entry_id="X1", minutes="1", hourly_rate="240.30"),
                    dict(rr[0], entry_id="X2", minutes="1", hourly_rate="240.30"),
                ],
            ),
        )
        self.assertEqual(time_entries(p).summary["total"], "8.02")

    def test_55_time_duplicate(self):
        p = self.fixture(18)
        self.change(p / "time_entries.csv", lambda rr: rr.append(dict(rr[1])))
        r = time_entries(p)
        self.assertEqual(r.summary["total"], "395.00")
        self.assertEqual(r.summary["held"], 4)

    def test_56_time_nan_infinity_rejected(self):
        p = self.fixture(18)
        self.change(p / "time_entries.csv", lambda rr: rr[0].update(hourly_rate="NaN"))
        self.assertEqual(time_entries(p).summary["held"], 3)

    def test_57_closing_hold_does_not_disappear(self):
        p = self.fixture(19)
        self.change(p / "closing.csv", lambda rr: rr[2].update(review_date=""))
```

```
        r = closing(p)
        self.assertEqual(r.summary["hold"], 1)
        row = r.tables["closing_review.csv"][1][2]
        self.assertEqual((row["file_id"], row["classification"]), ("F03", "hold"))
        self.assertIn("invalid_review_date", row["issues"])

    def test_58_closing_today_and_history(self):
        p = self.fixture(19)
        self.change(p / "closing.csv", lambda rr: rr[0].update(review_date="2026-09-24"))
        r = closing(p)
        self.assertEqual(r.summary["due_for_review"], 1)
        self.assertEqual(r.tables["history.csv"][1][0]["history_id"], "H01")

    def test_59_reviews_next_day(self):
        self.assertEqual(reviews(ROOT / "data/ch22", as_of="2026-09-25").summary["complete"],
    2)

    def test_60_reviews_version_change(self):
        self.assertEqual(reviews(ROOT / "data/ch22", artifact="3").summary["complete"], 0)

    def test_61_reviews_duplicate_category(self):
        p = self.fixture(22)
        self.change(p / "reviews.csv", lambda rr: rr.append(dict(rr[0])))
        r = reviews(p)
        self.assertEqual(r.summary["complete"], 2)

    def test_62_reviews_missing_category_retained(self):
        p = self.fixture(22)
        self.change(p / "reviews.csv", lambda rr: rr.pop(0))
        r = reviews(p)
        self.assertEqual(len(r.tables["review_register.csv"][1]), 5)
        self.assertEqual(r.summary["complete"], 2)

    def test_63_worklist_baseline_values(self):
        rr = worklist(ROOT / "data/ch23").tables["worklist.csv"][1]
        self.assertEqual(
            [r["status"] for r in rr],
            [
                "response_recorded",
                "window_open",
                "window_passed",
                "needs_review",
                "window_open",
                "response_recorded",
            ],
        )
        self.assertEqual((rr[5]["any_response"], rr[5]["in_window"]), ("yes", "no"))

    def test_64_worklist_day14_to15(self):
        r = worklist(ROOT / "data/ch23", as_of="2026-09-25")
        self.assertEqual((r.summary["window_open"], r.summary["window_passed"]), (1, 2))

    def test_65_worklist_future_request(self):
        p = self.fixture(23)
        self.change(p / "requests.csv", lambda rr: rr[1].update(request_date="2026-09-25"))
        r = worklist(p)
        self.assertEqual(r.summary["not_yet_requested"], 1)
        self.assertEqual(r.tables["worklist.csv"][1][1]["in_window"], "unavailable")

    def test_66_worklist_future_response(self):
```

```
        p = self.fixture(23)
        self.change(p / "responses.csv", lambda rr: rr[0].update(response_date="2026-09-25"))
        r = worklist(p)
        self.assertEqual(r.tables["worklist.csv"][1][0]["status"], "window_open")

    def test_67_worklist_tied_existence_not_identity(self):
        p = self.fixture(23)
        self.change(p / "responses.csv", lambda rr: rr.append(dict(rr[0], response_id="CS02")
))
        r = worklist(p)
        rr = r.tables["worklist.csv"][1][0]
        self.assertEqual(
            (rr["any_response"], rr["in_window"], rr["earliest"]), ("yes", "yes", "ambiguous"
)
        )

    def test_68_worklist_bad_response_priority(self):
        p = self.fixture(23)
        self.change(p / "responses.csv", lambda rr: rr[0].update(response_date="2026-09-09"))
        r = worklist(p)
        self.assertEqual(r.tables["worklist.csv"][1][0]["status"], "needs_review")

    def test_69_worklist_selected_export(self):
        r = worklist(ROOT / "data/ch23", matter="M01")
        self.assertEqual([x["request_id"] for x in r.tables["worklist.csv"][1]], ["Q01", "Q02
"])
        self.assertNotIn("Q06", r.report())
        self.assertNotIn("M03", r.report())

    def test_70_large_conversion_and_capstone(self):
        out = self.root / "large"
        s = convert(
            ROOT / "data/matter_events_large.csv",
            ROOT / "data/matters_large.csv",
            ROOT / "data/assignments_large.csv",
            out,
        )
        self.assertEqual(
            [s[k] for k in ["source_events", "requests", "responses", "notes", "held"]],
            [12000, 3000, 3000, 6000, 0],
        )
        r = worklist(out, as_of="2026-12-31")
        self.assertEqual(
            (r.summary["requests"], r.summary["response_recorded"], r.summary["in_window_yes"
]),
            (3000, 3000, 3000),
        )

    def test_71_large_missing_assignment_stops(self):
        path = self.root / "assignments.csv"
        path.write_text("matter_id,owner\n")
        with self.assertRaises(ValueError):
            convert(
                ROOT / "data/matter_events_large.csv",
                ROOT / "data/matters_large.csv",
                path,
                self.root / "out",
            )
        self.assertFalse((self.root / "out").exists())
```

```
    def test_72_irregular_opening_fixture(self):
        from matter_tools import load_csv, check, EVENT_FIELDS, MATTER_FIELDS

        a, h = check(
            load_csv(ROOT / "data/matter_events_irregular.csv", EVENT_FIELDS),
            load_csv(ROOT / "data/matters_irregular.csv", MATTER_FIELDS),
        )
        self.assertEqual((len(a), len(h)), (8, 3))
        self.assertEqual([r["event_id"] for r in h], ["I008", "I009", "I010"])

    def test_73_saved_runs_are_repeatable_and_preserve_inputs(self):
        for ch in APPLICATIONS:
            with self.subTest(chapter=ch):
                p = ROOT / "data" / f"ch{ch:02}"
                before = {f.name: sha(f) for f in p.iterdir() if f.is_file()}
                result = APPLICATIONS[ch](p)
                a = self.root / f"{ch}a"
                b = self.root / f"{ch}b"
                save_result(result, p, a, {"chapter": ch})
                save_result(APPLICATIONS[ch](p), p, b, {"chapter": ch})
                self.assertEqual(
                    {f.name: f.read_bytes() for f in a.iterdir()},
                    {f.name: f.read_bytes() for f in b.iterdir()},
                )
                self.assertEqual(before, {f.name: sha(f) for f in p.iterdir() if f.is_file()
})
                with self.assertRaises(FileExistsError):
                    save_result(result, p, a, {"chapter": ch})

    def test_74_empty_matching_tables(self):
        rr, cc = match_records([], [], {}, date(2026, 9, 24))
        self.assertEqual((rr, cc), ([], []))

    def test_75_empty_header_only_input(self):
        p = self.fixture(4)
        path = p / "appointments.csv"
        path.write_text(",".join(APPOINTMENT_FIELDS) + "\n")
        r = consultation(p)
        self.assertEqual(r.summary["input_rows"], 0)
        self.assertIsNone(r.summary["percentage"])

    def test_76_bad_headers_and_csv_records_stop(self):
        p = self.fixture(4)
        path = p / "appointments.csv"
        path.write_text("wrong\nvalue\n")
        with self.assertRaises(ValueError):
            consultation(p)

    def test_77_html_escaping(self):
        p = self.fixture(5)
        self.change(
            p / "documents.csv", lambda rr: rr[0].update(title="<script>alert(1)</script>")
        )
        report = catalogue(p).report()
        self.assertIn("&lt;script&gt;", report)
        self.assertNotIn("<script>alert", report)

    def test_78_historical_date_required_format(self):
        with self.assertRaises(ValueError):
            worklist(ROOT / "data/ch23", as_of="September 24")
```

```
    def test_79_negative_window_rejected(self):
        with self.assertRaises(ValueError):
            matching(ROOT / "data/ch08", window=-1)


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

programs/test_experiments.py

```
"""One regression test for each chapter's executable controlled experiment."""

import tempfile
import unittest
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "tools"))
from run_experiments import run_experiment


class ExperimentTests(unittest.TestCase):
    pass


def make_test(chapter):
    def test(self):
        with tempfile.TemporaryDirectory() as folder:
            result = run_experiment(chapter, Path(folder) / "experiment")
            self.assertTrue(result["passed"])

    return test


for chapter in range(1, 24):
    setattr(ExperimentTests, f"test_chapter_{chapter:02}", make_test(chapter))
if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Appendix F

# Reproducible Experiments and Baseline Runs

tools/run_experiments.py

```
"""Reproduce the controlled experiment in any chapter in a NEW folder.

The source release is never changed. The experiment copies its input, records
before/after hashes, runs the reference, and checks hand-derived expectations.
"""

from pathlib import Path
import sys, csv, json, shutil, argparse, subprocess

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "programs"))
from legal_apps import APPLICATIONS
from legal_common import save_result, sha
from matter_tools import run
from make_demo_package import make
from zipfile import ZipFile

EXPECTED = {
    1: {"accepted_rows": 18, "held_rows": 0},
    2: {"accepted_rows": 12, "held_rows": 6},
    3: {"accepted_rows": 11999, "held_rows": 1, "selected_accepted": 4},
    4: {"numerator": 4, "denominator": 6},
    5: {"file_records": 9, "documents": 5, "versions": 7},
    6: {"accepted": 4, "held": 4},
    7: {"input_rows": 8, "output_rows": 8, "total_count": 8},
    8: {"matched": 1, "ambiguous": 1},
    9: {"included": 4},
    10: {"observations": 17},
    11: {"eligible_final": 100, "responded_final": 54},
    12: {"accepted": 8, "overlap_numbers": 0, "gap_numbers": 2},
    13: {"generated": 5, "held": 1},
    14: {"needs_review": 3},
    15: {"matches": 2},
    16: {"worksheet_issue": "missing_source_location"},
    17: {"changed": 3, "original_file": "revised.txt"},
    18: {"total": "579.01", "accepted": 5},
    19: {"due_for_review": 0, "upcoming": 2, "hold": 1, "needs_review": 2},
    20: {"mutation_detected": True},
    21: {"members": 12, "unrelated_excluded": True},
    22: {"complete": 4},
    23: {"not_yet_requested": 1, "window_open": 1, "response_recorded": 2},
}
```

```
def edit(path, callback):
    with path.open(encoding="utf-8", newline="") as f:
        reader = csv.DictReader(f)
        fields = reader.fieldnames
        rows = list(reader)
    callback(rows)
    with path.open("w", encoding="utf-8", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fields, lineterminator="\n")
        w.writeheader()
        w.writerows(rows)


def run_experiment(ch: int, output: Path) -> dict:
    if ch not in EXPECTED:
        raise ValueError("Chapter must be 1 through 23")
    if output.exists():
        raise FileExistsError("Use a new experiment destination")
    output.mkdir(parents=True)
    folder = output / "input"
    notes = ""
    if ch <= 3:
        folder.mkdir()
        names = [
            "matter_events_demo.csv",
            "matter_events_problem.csv",
            "matters_demo.csv",
            "matter_events_large.csv",
            "matters_large.csv",
        ]
        for name in names:
            shutil.copyfile(ROOT / "data" / name, folder / name)
    elif ch not in [20, 21]:
        shutil.copytree(ROOT / "data" / f"ch{ch:02}", folder)
    before = (
        {p.name: sha(p) for p in folder.iterdir() if p.is_file()} if folder.exists() else {}
    )
    params = {}
    if ch == 1:
        edit(
            folder / "matter_events_demo.csv",
            lambda rr: rr[2].update(event_date=rr[1]["event_date"]),
        )
        notes = "E03 now shares E02 date."
        observed = run(
            folder / "matter_events_demo.csv", folder / "matters_demo.csv", output / "result"
        )
    elif ch == 2:
        edit(
            folder / "matter_events_problem.csv",
            lambda rr: rr[4].update(event_date="2026-09-07"),
        )
        notes = "Restore E05 to clean-fixture date."
        # Read the matching clean value explicitly, never rely on an invented date.
        with (folder / "matter_events_demo.csv").open() as f:
            clean = list(csv.DictReader(f))
        edit(
            folder / "matter_events_problem.csv",
            lambda rr: rr[4].update(event_date=clean[4]["event_date"]),
        )
```

```
      observed = run(
          folder / "matter_events_problem.csv", folder / "matters_demo.csv", output / "
result"
      )
  elif ch == 3:
      edit(folder / "matter_events_large.csv", lambda rr: rr[6].update(client_id="C9999"))
      notes = "Wrong client on E00007, outside M0001."
      observed = run(
          folder / "matter_events_large.csv",
          folder / "matters_large.csv",
          output / "result",
          "M0001",
      )
  elif ch == 20:
      working = output / "faulty"
      shutil.copytree(
          ROOT / "programs",
          working / "programs",
          ignore=shutil.ignore_patterns("__pycache__"),
      )
      shutil.copytree(ROOT / "data", working / "data")
      f = working / "programs/legal_apps.py"
      text = f.read_text()
      assert "0 <= elapsed <= window" in text
      f.write_text(text.replace("0 <= elapsed <= window", "0 <= elapsed < window"))
      proc = subprocess.run(
          [
              sys.executable,
              "-m",
              "unittest",
              "test_legal_apps.LegalTests.test_15_matching_day14_and_day15",
              "-v",
          ],
          cwd=working / "programs",
          capture_output=True,
          text=True,
      )
      # Record the failure with the working folder shown by its relative name,
      # not by the machine path of whoever ran the helper.
      text = (proc.stdout + proc.stderr).replace(str(working.resolve()), "faulty")
      (output / "deliberate_failure.txt").write_text(text)
      observed = {
          "mutation_detected": proc.returncode == 1 and "FAIL: test_15" in proc.stderr
      }
      notes = "Inclusive day 14 was deliberately changed to exclusive."
  elif ch == 21:
      working = output / "source"
      working.mkdir()
      from make_demo_package import MEMBERS

      for name in MEMBERS:
          dest = working / name
          dest.parent.mkdir(parents=True, exist_ok=True)
          shutil.copyfile(ROOT / name, dest)
      (working / "UNRELATED_PRIVATE_NOTE.txt").write_text(
          "Invented canary; must not enter archive."
      )
      make(working, output / "Matter_Demo.zip")
      with ZipFile(output / "Matter_Demo.zip") as z:
          names = z.namelist()
```

```
        observed = {
            "members": len(names),
            "unrelated_excluded": "UNRELATED_PRIVATE_NOTE.txt" not in names,
        }
        notes = "Unrelated source-folder file is excluded by the allowlist."
    else:
        if ch == 4:
            edit(
                folder / "appointments.csv",
                lambda rr: rr[4].update(completion_date="2026-09-05"),
            )
            notes = "Supply A05 completion date."
        elif ch == 5:
            edit(
                folder / "documents.csv",
                lambda rr: rr.append(dict(rr[1], file_id="F09", source_id="S09")),
            )
            notes = "Add another file copy for the same version."
        elif ch == 6:
            edit(folder / "documents.csv", lambda rr: rr[2].update(source_id="S03"))
            notes = "Restore F03 source only."
        elif ch == 7:
            edit(folder / "appointments.csv", lambda rr: rr[4].update(status="missed"))
            notes = "Change A05 supplied status to missed."
        elif ch == 8:
            edit(
                folder / "responses.csv",
                lambda rr: rr.append(dict(rr[3], response_id="S08", source_id="RES08")),
            )
            notes = "Add a second earliest response for M02/R03."
        elif ch == 9:
            params = {"as_of": "2026-10-02"}
            notes = "Change reporting date only."
        elif ch == 10:
            edit(
                folder / "pairs.csv",
                lambda rr: rr.append(
                    dict(
                        rr[0],
                        pair_id="PAIR17",
                        request_id="REQ17",
                        request_date="2026-04-01",
                        response_date="2026-04-10",
                        elapsed_days="9",
                        source_id="PAIR-SOURCE-17",
                    )
                ),
            )
            notes = "Add an actual April date pair with nine elapsed days."
        elif ch == 11:
            edit(folder / "weeks.csv", lambda rr: rr[10].update(status="final"))
            notes = "Finalize W11 without changing its counts."
        elif ch == 12:
            edit(
                folder / "production.csv",
                lambda rr: rr[5].update(bates_start="ABC000017", page_count="2"),
            )
            notes = "Correct D06 to start at ABC000017; the gap at ABC000013-14 must remain."
        elif ch == 13:
            edit(
```

```
            folder / "letters.csv", lambda rr: rr[2].update(recipient="Example Recipient
3")
        )
        notes = "Repair L03 recipient, not L05 date."
    elif ch == 14:
        edit(folder / "dates.csv", lambda rr: rr[0].update(verification_date="2026-09-25"
))
        notes = "Verification after the as-of date needs review."
    elif ch == 15:
        params = {"query": "Alex Morgen"}
        notes = "Run the supplied one-edit search with a misspelling."
    elif ch == 16:
        edit(folder / "propositions.csv", lambda rr: rr[0].update(source_location=""))
        notes = "Remove P01 source location; authority counts do not change."
    elif ch == 17:
        params = {"reverse": True}
        notes = "Reverse original and revised directions without changing files."
    elif ch == 18:
        edit(
            folder / "time_entries.csv",
            lambda rr: rr.append(
                dict(
                    rr[0],
                    entry_id="T07",
                    minutes="1",
                    hourly_rate="240.30",
                    source_id="TIME07",
                )
            ),
        )
        notes = "Add the half-cent boundary line; round half up per line."
    elif ch == 19:
        edit(
            folder / "closing.csv",
            lambda rr: rr[0].update(review_date="2026-10-15", source_id="CLOSING01-REV2"),

        )
        edit(
            folder / "review_history.csv",
            lambda rr: rr.append(
                dict(
                    rr[0],
                    history_id="H02",
                    review_date="2026-09-24",
                    source_version="2",
                    outcome="Review again October 15; no disposition authorized",
                    source_id="HISTORY02",
                )
            ),
        )
        notes = "Save a revised supplied review date and append its history entry."
    elif ch == 22:
        edit(folder / "reviews.csv", lambda rr: rr[3].update(reviewer="Reviewer Example")
)
        notes = "Complete the synthetic data-handling reviewer field."
    elif ch == 23:
        edit(folder / "requests.csv", lambda rr: rr[1].update(request_date="2026-09-25"))
        notes = "Future request Q02 must not enter an open-window category."
    result = APPLICATIONS[ch](folder, **params)
    save_result(result, folder, output / "result", dict(chapter=ch, **params))
```

```
        observed = dict(result.summary)
        if ch == 16:
            observed["worksheet_issue"] = result.tables["propositions.csv"][1][0]["issues"]
    after = {p.name: sha(p) for p in folder.iterdir() if p.is_file()} if folder.exists() else
   {}
    checks = {k: observed.get(k) == v for k, v in EXPECTED[ch].items()}
    record = {
        "chapter": ch,
        "change": notes,
        "parameters": params,
        "before_hashes": before,
        "after_hashes": after,
        "expected": EXPECTED[ch],
        "observed": observed,
        "checks": checks,
        "passed": all(checks.values()),
    }
    (output / "experiment.json").write_text(json.dumps(record, indent=2, sort_keys=True) + "\
  n")
    if not record["passed"]:
        raise AssertionError(record)
    return record


if __name__ == "__main__":
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("chapter", type=int)
    p.add_argument("--out", type=Path, required=True)
    args = p.parse_args()
    print(json.dumps(run_experiment(args.chapter, args.out), indent=2))
```

tools/verify_and_run.py

```
"""Reproduce reference outputs in a NEW destination; never update baselines."""

from pathlib import Path
import sys, os, json, shutil, csv, tempfile, subprocess, argparse

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "programs"))
from legal_apps import APPLICATIONS, worklist, matching
from legal_common import save_result
from matter_tools import run
from convert_matter_events import convert


def generate(out: Path):
    if out.exists():
        raise FileExistsError("Reference destination must be new")
    out.mkdir(parents=True)
    run(ROOT / "data/matter_events_demo.csv", ROOT / "data/matters_demo.csv", out / "ch01")
    run(ROOT / "data/matter_events_problem.csv", ROOT / "data/matters_demo.csv", out / "ch02"
  )
    run(
        ROOT / "data/matter_events_large.csv",
        ROOT / "data/matters_large.csv",
        out / "ch03",
        "M0001",
    )
```

```
    run(
        ROOT / "data/matter_events_irregular.csv",
        ROOT / "data/matters_irregular.csv",
        out / "irregular",
    )
    # Later chapters: run each chapter folder exactly as a reader does, in a
    # temporary copy, so the saved run.json matches a reader's run.json.
    chapters = ROOT / "Chapters"
    if not chapters.is_dir():
        raise FileNotFoundError("Build Chapters/ first: python3 tools/make_chapter_folders.py
")
    meta = json.loads((ROOT / "CHAPTER_FILES.json").read_text(encoding="utf-8"))
    for ch in APPLICATIONS:
        entry = Path(meta[str(ch)]["program"]).name
        with tempfile.TemporaryDirectory() as temp:
            copy = Path(temp) / f"chapter{ch:02}"
            shutil.copytree(chapters / f"chapter{ch:02}", copy)
            proc = subprocess.run(
                [sys.executable, entry, "--out", str((out / f"ch{ch:02}").resolve())],
                cwd=copy,
                capture_output=True,
                text=True,
                env={**os.environ, "PYTHONDONTWRITEBYTECODE": "1"},
            )
            if proc.returncode:
                raise RuntimeError(f"chapter {ch}: {proc.stdout}{proc.stderr}")
    conv = out / "large_conversion"
    convert(
        ROOT / "data/matter_events_large.csv",
        ROOT / "data/matters_large.csv",
        ROOT / "data/assignments_large.csv",
        conv,
    )
    save_result(
        matching(conv, as_of="2026-12-31"),
        conv,
        out / "large_matching",
        {"chapter": 8, "as_of": "2026-12-31", "window": 14},
    )
    save_result(
        worklist(conv, as_of="2026-12-31"),
        conv,
        out / "large_worklist",
        {"chapter": 23, "as_of": "2026-12-31", "window": 14},
    )
    save_result(
        worklist(conv, as_of="2026-12-31", matter="M0001"),
        conv,
        out / "large_selected",
        {"chapter": 23, "as_of": "2026-12-31", "window": 14, "matter": "M0001"},
    )
    print("Reference runs saved:", out)


if __name__ == "__main__":
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--out", type=Path, required=True)
    generate(p.parse_args().out)
```

tools/make_chapter_folders.py

```
"""Build Chapters/chapter01 ... chapter23: one self-contained folder per chapter.

Each folder holds exactly the files a reader uploads to an assistant (or runs
locally) for that chapter, plus RUN.txt. The files are copies of the canonical
programs/ and data/ files; this script never edits them. Every folder is then
test-run in a temporary copy, and the printed result is recorded in RUN.txt.
"""

from __future__ import annotations
import argparse
import json
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
SHARED = ["run_chapter.py", "legal_apps.py", "legal_common.py"]
OPENING = {
    1: (
        "matter_view.py",
        ["matter_events_demo.csv", "matters_demo.csv"],
        "python3 matter_view.py --input matter_events_demo.csv --matters matters_demo.csv --
  out Run_01",
    ),
    2: (
        "matter_check.py",
        ["matter_events_problem.csv", "matters_demo.csv", "matter_events_demo.csv"],
        "python3 matter_check.py --input matter_events_problem.csv --matters matters_demo.csv
   --out Run_01",
    ),
    3: (
        "matter_run.py",
        [
            "matter_events_large.csv",
            "matters_large.csv",
            "matter_events_demo.csv",
            "matters_demo.csv",
            "matter_events_irregular.csv",
            "matters_irregular.csv",
        ],
        "python3 matter_run.py --input matter_events_large.csv --matters matters_large.csv --
  matter M0001 --out Run_01",
    ),
}
FULL_RELEASE = {
    20: "Chapter 20 tests the whole release. Work in the extracted release folder, not here.\
  n"
    'From that folder run:\n\n  cd programs\n  python3 -m unittest discover -p "test_*.py" -v
  \n\n'
    "Expected last lines:\n\n  Ran 157 tests\n  OK\n",
    21: "Chapter 21 packages files from the whole release. Work in the extracted release
  folder.\n"
    "From that folder run:\n\n  python3 programs/make_demo_package.py --root . --out
  my_matter_demo.zip\n\n"
    "Then open the ZIP and count its members: 12 (11 payload files and MANIFEST.json).\n",
}
```

```
def chapter_info(ch: int) -> dict:
    meta = json.loads((ROOT / "CHAPTER_FILES.json").read_text(encoding="utf-8"))
    return meta[str(ch)]


def build(out: Path) -> None:
    if out.exists():
        raise FileExistsError(f"{out} exists; choose a new destination")
    out.mkdir(parents=True)
    for ch in range(1, 24):
        folder = out / f"chapter{ch:02}"
        folder.mkdir()
        info = chapter_info(ch)
        if ch in FULL_RELEASE:
            (folder / "RUN.txt").write_text(
                f"CHAPTER {ch}: {info['title'].upper()}\n\n" + FULL_RELEASE[ch],
                encoding="utf-8",
            )
            continue
        if ch in OPENING:
            program, inputs, command = OPENING[ch]
            files = [ROOT / "programs" / program, ROOT / "programs" / "matter_tools.py"] + [
                ROOT / "data" / i for i in inputs
            ]
        else:
            program = Path(info["program"]).name
            command = f"python3 {program} --out Run_01"
            files = [ROOT / "programs" / program] + [ROOT / "programs" / s for s in SHARED]
            files += sorted(p for p in (ROOT / "data" / f"ch{ch:02}").iterdir() if p.is_file
  ())
            if ch == 23:
                files += [
                    ROOT / "programs" / "convert_matter_events.py",
                    ROOT / "programs" / "matter_tools.py",
                    ROOT / "data" / "matter_events_large.csv",
                    ROOT / "data" / "matters_large.csv",
                    ROOT / "data" / "assignments_large.csv",
                ]
        for f in files:
            shutil.copyfile(f, folder / f.name)
        printed = trial(folder, command)
        names = "\n".join("  " + f.name for f in files)
        text = (
            f"CHAPTER {ch}: {info['title'].upper()}\n"
            "Synthetic teaching data only. Not approved for real legal-office use.\n\n"
            f"FILES TO UPLOAD TOGETHER ({len(files)}):\n{names}\n\n"
            "Do not upload RUN.txt. Keep the filenames exactly as shown.\n\n"
            f"RUN ON YOUR OWN COMPUTER (Python 3.10 or later), from this folder:\n  {command
  }\n\n"
            "On Windows, substitute python or py if that is the installed interpreter command
  .\n"
            "Use a new output folder name (Run_02, Run_03, ...) for every later run.\n\n"
            f"EXPECTED PRINTED RESULT:\n{printed}\n"
            f"OUTPUT FILES: {', '.join(info['outputs'])}\n"
            f"SAVED REFERENCE RESULT: {info['quick_view']} in the release folder.\n"
        )
        (folder / "RUN.txt").write_text(text, encoding="utf-8")
    print("Chapter folders saved:", out)
```

```
def trial(folder: Path, command: str) -> str:
    """Run the folder's command in a temporary copy and return what it printed."""
    with tempfile.TemporaryDirectory() as temp:
        copy = Path(temp) / folder.name
        shutil.copytree(folder, copy)
        args = command.split()
        args[0] = sys.executable
        proc = subprocess.run(args, cwd=copy, capture_output=True, text=True)
        if proc.returncode:
            raise RuntimeError(f"{folder.name} failed:\n{proc.stdout}{proc.stderr}")
        return proc.stdout


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--out", type=Path, default=ROOT / "Chapters")
    build(parser.parse_args().out)
```

Appendix G

# Export Preparation and Additional Regression Cases

programs/prepare_event_export.py

```
"""Explicit mapping of a fictional office export to the opening event schema.

Dates are not guessed. Optional corrections are supplied decisions, recorded
in the mapping ledger. The output folder must be new; inputs are unchanged.
"""

import argparse
import csv
import json
import re
from pathlib import Path
from matter_tools import load_csv, check, valid_date, EVENT_FIELDS, MATTER_FIELDS
from legal_common import sha

EXPORT_FIELDS = [
    "event_number",
    "client_number",
    "matter_number",
    "date_text",
    "event_kind",
    "request_number",
    "source_label",
    "note",
]
CORRECTION_FIELDS = ["event_number", "verified_date", "source_id", "authorized_by"]


def write(path, fields, rows):
    with path.open("w", encoding="utf-8", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fields, lineterminator="\n")
        w.writeheader()
        w.writerows(rows)


def prepare(input_path, matters_path, out, corrections_path=None):
    raw = load_csv(input_path, EXPORT_FIELDS)
    matters = load_csv(matters_path, MATTER_FIELDS)
    corrections = {}
    if corrections_path:
        for r in load_csv(corrections_path, CORRECTION_FIELDS):
            if (
                r["event_number"] in corrections
                or not valid_date(r["verified_date"])
```

```
            or not r["source_id"].strip()
            or not r["authorized_by"].strip()
        ):
            raise ValueError(
                "Correction entries need a unique key, valid date, source and authorizer"
            )
        corrections[r["event_number"]] = r
    if set(corrections) - {r["event_number"] for r in raw}:
        raise ValueError("Correction names an absent source event")
mapped = []
mapping_issues = []
for r in raw:
    issues = []
    if not re.fullmatch(r"[0-9]+", r["event_number"]):
        issues.append("invalid_event_number")
    if r["request_number"] and not re.fullmatch(r"[0-9]+", r["request_number"]):
        issues.append("invalid_request_number")
    correction = corrections.get(r["event_number"])
    mapped.append(
        dict(
            event_id="E" + r["event_number"],
            client_id=r["client_number"],
            matter_id=r["matter_number"],
            event_date=correction["verified_date"] if correction else r["date_text"],
            event_type=r["event_kind"],
            request_id="R" + r["request_number"] if r["request_number"] else "",
            source_id=r["source_label"],
            summary=r["note"],
        )
    )
    mapping_issues.append(issues)
if any(mapping_issues):
    raise ValueError("Export identifier format is invalid; no mapping was written")
accepted, held = check(mapped, matters)
checks = {r["source_row"]: r.get("reasons", "") for r in accepted + held}
ledger = []
output = []
for i, (raw_row, row) in enumerate(zip(raw, mapped)):
    reasons = mapping_issues[i] + ([checks[i + 2]] if checks[i + 2] else [])
    correction = corrections.get(raw_row["event_number"], {})
    ledger.append(
        dict(
            source_row=i + 2,
            event_number=raw_row["event_number"],
            event_id=row["event_id"],
            original_date=raw_row["date_text"],
            mapped_date=row["event_date"],
            status="held" if reasons else "accepted",
            issues=";".join(reasons),
            correction_source=correction.get("source_id", ""),
            authorized_by=correction.get("authorized_by", ""),
        )
    )
    if not reasons:
        output.append(row)
if out.exists() or out.is_symlink():
    raise FileExistsError(f"Refusing to overwrite {out}")
out.mkdir(parents=True, exist_ok=False)
write(out / "matter_events.csv", EVENT_FIELDS, output)
write(out / "matters.csv", MATTER_FIELDS, matters)
```

```
    write(
        out / "mapping_ledger.csv",
        list(ledger[0])
        if ledger
        else [
            "source_row",
            "event_number",
            "event_id",
            "original_date",
            "mapped_date",
            "status",
            "issues",
            "correction_source",
            "authorized_by",
        ],
        ledger,
    )
    record = {
        "source_rows": len(raw),
        "accepted": len(output),
        "held": len(raw) - len(output),
        "input_sha256": sha(input_path),
        "matters_sha256": sha(matters_path),
        "corrections_sha256": sha(corrections_path) if corrections_path else None,
        "program_sha256": sha(Path(__file__)),
        "output_hashes": {p.name: sha(p) for p in sorted(out.iterdir()) if p.is_file()},
        "note": "Supplied correction metadata is recorded, not authenticated. No date format
  was inferred.",
    }
    (out / "preparation.json").write_text(json.dumps(record, indent=2, sort_keys=True) + "\n"
  )
    return record


def main():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--input", type=Path, required=True)
    p.add_argument("--matters", type=Path, required=True)
    p.add_argument("--out", type=Path, required=True)
    p.add_argument("--corrections", type=Path)
    a = p.parse_args()
    try:
        r = prepare(a.input, a.matters, a.out, a.corrections)
    except (OSError, ValueError, csv.Error) as e:
        p.exit(2, str(e) + "\n")
    print(json.dumps(r, indent=2, sort_keys=True))


if __name__ == "__main__":
    main()
```

programs/test_revision_cases.py

```
"""Cases found by reviewing claims and outputs beyond the baseline examples."""

import csv
import hashlib
import shutil
import subprocess
import sys
```

```
import tempfile
import unittest
import xml.etree.ElementTree as ET
from decimal import localcontext, Inexact
from pathlib import Path
from legal_apps import letters, turnaround, production_index, supplied_dates, worklist,
  time_entries
from legal_common import Result, save_result
from matter_tools import load_csv, EVENT_FIELDS

ROOT = Path(__file__).resolve().parents[1]


class RevisionCases(unittest.TestCase):
    def setUp(self):
        self.temp = tempfile.TemporaryDirectory()
        self.addCleanup(self.temp.cleanup)
        self.root = Path(self.temp.name)

    def fixture(self, ch):
        dest = self.root / f"ch{ch:02}"
        shutil.copytree(ROOT / "data" / f"ch{ch:02}", dest)
        return dest

    def change(self, path, modify):
        with path.open(newline="", encoding="utf-8") as f:
            rows = list(csv.DictReader(f))
        fields = list(rows[0])
        modify(rows)
        with path.open("w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=fields, lineterminator="\n")
            w.writeheader()
            w.writerows(rows)

    def test_time_teaching_limits_include_both_endpoints(self):
        p = self.fixture(18)
        self.change(
            p / "time_entries.csv",
            lambda rows: (
                rows[0].update(minutes="1000000", hourly_rate="1000000"),
                rows[1].update(minutes="0", hourly_rate="0"),
                rows[2].update(minutes="1000000", hourly_rate="0.0001"),
            ),
        )
        result = time_entries(p)
        ledger = {row["entry_id"]: row for row in result.tables["ledger.csv"][1]}
        self.assertEqual(ledger["T01"]["amount"], "16666666666.67")
        self.assertEqual(ledger["T02"]["amount"], "0.00")
        self.assertEqual(ledger["T03"]["amount"], "1.67")
        self.assertEqual(result.summary["accepted"], 4)
        self.assertEqual(result.summary["total"], "16666666868.34")

    def test_time_out_of_range_values_are_held_without_traceback(self):
        p = self.fixture(18)
        self.change(
            p / "time_entries.csv",
            lambda rows: (
                rows[0].update(minutes="1000001"),
                rows[1].update(hourly_rate="1E+30"),
                rows[2].update(minutes="9" * 5000),
                rows[4].update(hourly_rate="1000000.0001"),
            ),
        )
```

```
        result = time_entries(p)
        ledger = {row["entry_id"]: row for row in result.tables["ledger.csv"][1]}
        for identifier in ["T01", "T03"]:
            self.assertEqual(ledger[identifier]["issues"], "minutes_outside_teaching_range")
        for identifier in ["T02", "T05"]:
            self.assertEqual(ledger[identifier]["issues"], "rate_outside_teaching_range")
        self.assertTrue(all(row["amount"] == "" for row in ledger.values()))
        self.assertEqual(result.summary["held"], 6)
        self.assertEqual(result.summary["total"], "0.00")
        completed = subprocess.run(
            [sys.executable, str(ROOT / "programs/time_entry_check.py"),
             "--data", str(p), "--out", str(self.root / "bounded_cli")],
            capture_output=True, text=True,
        )
        self.assertEqual(completed.returncode, 0, completed.stderr)
        self.assertNotIn("Traceback", completed.stderr)

    def test_time_rate_scale_ignores_trailing_fractional_zeros(self):
        p = self.fixture(18)
        self.change(
            p / "time_entries.csv",
            lambda rows: (
                rows[0].update(hourly_rate="240.30000"),
                rows[1].update(hourly_rate="0.00001"),
                rows[2].update(hourly_rate="1E-1000000"),
                rows[4].update(hourly_rate="-0E+1000000"),
            ),
        )
        result = time_entries(p)
        ledger = {row["entry_id"]: row for row in result.tables["ledger.csv"][1]}
        self.assertEqual(ledger["T01"]["amount"], "120.15")
        self.assertEqual(ledger["T05"]["amount"], "0.00")
        for identifier in ["T02", "T03"]:
            self.assertEqual(ledger[identifier]["issues"], "
rate_precision_exceeds_four_places")
            self.assertEqual(ledger[identifier]["amount"], "")

    def test_time_arithmetic_uses_its_own_decimal_precision(self):
        p = self.fixture(18)
        self.change(
            p / "time_entries.csv",
            lambda rows: rows[0].update(minutes="1000000", hourly_rate="1000000"),
        )
        with localcontext() as context:
            context.prec = 3
            context.Emax = 3
            context.traps[Inexact] = True
            result = time_entries(p)
        self.assertEqual(result.summary["total"], "16666667121.67")
        self.assertEqual(result.summary["accepted"], 4)

    def test_recorded_matcher_differs_on_four_extra_inputs(self):
        sys.path.insert(0, str(ROOT / "tools"))
        try:
            from reproduce_learning_cases import recorded_build_probes
        finally:
            sys.path.pop(0)
        historical = ROOT / "teaching_cases/recorded_build/my_matcher_v3.py"
        before = hashlib.sha256(historical.read_bytes()).hexdigest()
        record = recorded_build_probes(self.root / "recorded_probes")
        self.assertEqual(hashlib.sha256(historical.read_bytes()).hexdigest(), before)
        self.assertEqual(record["historical_program_sha256"], before)
        expected = {
```

```
            "blank_owner": "missing_owner",
            "blank_source": "missing_source_id",
            "unknown_matter": "unknown_matter",
            "future_request": "",
        }
        self.assertEqual({case["case"] for case in record["cases"]}, set(expected))
        for case in record["cases"]:
            with self.subTest(case=case["case"]):
                self.assertTrue(case["expected_status_difference_confirmed"])
                self.assertIn("status", case["different_fields"])
                self.assertEqual(case["reference"]["any_response"], "unavailable")
                self.assertEqual(case["historical"]["issues"], (
                    "request_after_as_of" if case["case"] == "future_request" else ""
                ))
                self.assertIn(expected[case["case"]], case["reference"]["issues"])

    def test_letter_named_report_preserves_both_documents(self):
        p = self.fixture(13)
        self.change(p / "letters.csv", lambda rr: rr[0].update(letter_id="report"))
        r = letters(p)
        out = self.root / "out"
        save_result(r, p, out, {})
        self.assertEqual(r.summary["generated"], 4)
        self.assertIn("DRAFT FOR REVIEW", (out / "letter_report.html").read_text())
        self.assertIn("Draft administrative letter pack", (out / "report.html").read_text())
        for row in r.tables["manifest.csv"][1]:
            if row["status"] == "generated":
                self.assertIn("DRAFT FOR REVIEW", (out / row["output"]).read_text())

    def test_case_insensitive_letter_collisions_are_held(self):
        p = self.fixture(13)
        self.change(p / "letters.csv", lambda rr: rr[1].update(letter_id="l01"))
        r = letters(p)
        self.assertEqual(r.summary["generated"], 2)
        self.assertTrue(
            all(
                "case_insensitive_filename_collision" in x["issues"]
                for x in r.tables["manifest.csv"][1][:2]
            )
        )

    def test_writer_rejects_reserved_name_before_creating_output(self):
        p = self.fixture(13)
        out = self.root / "out"
        r = Result("bad", {}, extra={"report.html": "letter"})
        with self.assertRaises(ValueError):
            save_result(r, p, out, {})
        self.assertFalse(out.exists())

    def test_writer_rejects_table_extra_collision(self):
        p = self.fixture(13)
        out = self.root / "out"
        r = Result("bad", {}, extra={"a.csv": "text"})
        r.add("A.csv", [], ["id"])
        with self.assertRaises(ValueError):
            save_result(r, p, out, {})
        self.assertFalse(out.exists())

    def test_chart_april_text_changes_with_observations(self):
        p = self.fixture(10)
        self.change(
            p / "pairs.csv",
            lambda rr: rr.append(
```

```
                dict(
                    rr[0],
                    pair_id="APR",
                    request_id="APR",
                    request_date="2026-04-01",
                    response_date="2026-04-10",
                    elapsed_days="9",
                )
            ),
        )
        r = turnaround(p)
        self.assertEqual(r.tables["monthly.csv"][1][3]["count"], 1)
        svg = r.extra["turnaround.svg"]
        self.assertNotIn("No observed pairs: 2026-04", svg)
        self.assertIn("Every displayed month has at least one observed pair", svg)

    def test_chart_long_duration_marker_is_visible(self):
        p = self.fixture(10)
        self.change(
            p / "pairs.csv",
            lambda rr: rr.__setitem__(
                slice(None),
                [
                    dict(
                        rr[0],
                        request_date="2026-01-01",
                        response_date="2026-03-02",
                        elapsed_days="60",
                    )
                ],
            ),
        )
        r = turnaround(p)
        self.assertEqual(r.tables["monthly.csv"][1][0]["mean"], 60)
        svg = ET.fromstring(r.extra["turnaround.svg"])
        circles = list(svg.iter("{http://www.w3.org/2000/svg}circle"))
        self.assertEqual(len(circles), 1)
        self.assertGreaterEqual(float(circles[0].attrib["cy"]), 40)
        self.assertLessEqual(float(circles[0].attrib["cy"]), 320)

    def test_chart_zero_is_observed_and_finite(self):
        p = self.fixture(10)
        self.change(
            p / "pairs.csv",
            lambda rr: rr.__setitem__(
                slice(None),
                [dict(rr[0], response_date=rr[0]["request_date"], elapsed_days="0")],
            ),
        )
        r = turnaround(p)
        svg = ET.fromstring(r.extra["turnaround.svg"])
        self.assertEqual(len(list(svg.iter("{http://www.w3.org/2000/svg}circle"))), 1)
        self.assertIn('cy="320.0"', r.extra["turnaround.svg"])

    def test_chart_empty_has_no_markers(self):
        p = self.fixture(10)
        self.change(p / "pairs.csv", lambda rr: rr.clear())
        r = turnaround(p)
        self.assertEqual(r.summary["observations"], 0)
        self.assertNotIn("<circle", r.extra["turnaround.svg"])
        self.assertIn("Gaps are not zero days", r.extra["turnaround.svg"])

    def test_nested_range_uses_greatest_end(self):
```

```
        p = self.fixture(12)
        self.change(
            p / "production.csv",
            lambda rr: rr[0].update(bates_end="ABC000100", page_count="100"),
        )
        r = production_index(p)
        v = r.tables["volumes.csv"][1][0]
        self.assertEqual(v["last_label"], "ABC000100")
        self.assertEqual(v["range_note"], "supplied_ranges_including_held")

    def test_mixed_prefix_volume_has_no_single_endpoint(self):
        p = self.fixture(12)
        self.change(
            p / "production.csv",
            lambda rr: rr[0].update(bates_start="XYZ000001", bates_end="XYZ000003"),
        )
        v = production_index(p).tables["volumes.csv"][1][0]
        self.assertEqual((v["first_label"], v["last_label"]), ("", ""))
        self.assertIn("unavailable", v["range_note"])

    def test_invalid_range_volume_has_no_endpoint(self):
        p = self.fixture(12)
        self.change(p / "production.csv", lambda rr: rr[0].update(bates_end="bad"))
        v = production_index(p).tables["volumes.csv"][1][0]
        self.assertEqual(v["last_label"], "")
        self.assertIn("unavailable", v["range_note"])

    def test_unclosed_quote_is_file_error(self):
        p = self.root / "broken.csv"
        lines = (ROOT / "data/matter_events_demo.csv").read_text().splitlines()
        cells = lines[-1].split(",")
        cells[-1] = '"' + cells[-1]
        lines[-1] = ",".join(cells)
        p.write_text("\n".join(lines) + "\n")
        with self.assertRaisesRegex(ValueError, "Malformed CSV"):
            load_csv(p, EVENT_FIELDS)

    def test_self_supersession_needs_review(self):
        p = self.fixture(14)
        self.change(
            p / "dates.csv",
            lambda rr: rr[0].update(record_status="superseded", supersedes="D01"),
        )
        r = supplied_dates(p).tables["date_register.csv"][1][0]
        self.assertEqual(r["classification"], "needs_review")
        self.assertIn("self_supersession", r["issues"])

    def test_supersession_cycle_is_exposed(self):
        p = self.fixture(14)

        def edit(rr):
            rr[0].update(record_status="superseded", supersedes="D07")
            rr.append(dict(rr[0], date_id="D07", supersedes="D01"))

        self.change(p / "dates.csv", edit)
        rows = supplied_dates(p).tables["date_register.csv"][1]
        self.assertTrue(
            all(
                "cyclic_supersession" in r["issues"]
                for r in rows
                if r["date_id"] in {"D01", "D07"}
            )
        )
```

```
    def test_superseded_without_successor_is_incomplete(self):
        p = self.fixture(14)
        self.change(p / "dates.csv", lambda rr: rr[0].update(record_status="superseded"))
        r = supplied_dates(p).tables["date_register.csv"][1][0]
        self.assertIn("missing_or_ambiguous_successor", r["issues"])

    def test_invalid_candidate_does_not_destroy_valid_request_evidence(self):
        p = self.fixture(23)
        self.change(p / "responses.csv", lambda rr: rr[0].update(response_date="2026-09-09"))
        q = worklist(p).tables["worklist.csv"][1][0]
        self.assertEqual(
            [q[k] for k in ["status", "any_response", "in_window", "earliest"]],
            ["needs_review", "no", "no", "none"],
        )

    def test_good_candidate_survives_bad_secondary_candidate(self):
        p = self.fixture(23)
        self.change(
            p / "responses.csv",
            lambda rr: rr.append(dict(rr[0], response_id="BAD", response_date="2026-09-09")),
        )
        q = worklist(p).tables["worklist.csv"][1][0]
        self.assertEqual(
            [
                q[k]
                for k in [
                    "status",
                    "any_response",
                    "in_window",
                    "earliest",
                    "selected_response",
                ]
            ],
            ["needs_review", "yes", "yes", "unique", "CS01"],
        )

    def test_mixed_worklist_named_expectations(self):
        r = worklist(ROOT / "data/ch23_mixed")
        rows = {x["request_id"]: x for x in r.tables["worklist.csv"][1]}
        self.assertEqual(len(rows), 9)
        self.assertEqual(
            [rows[f"Q{i:02}"]["status"] for i in range(1, 10)],
            [
                "response_recorded",
                "window_open",
                "window_passed",
                "needs_review",
                "window_open",
                "response_recorded",
                "not_yet_requested",
                "response_recorded",
                "needs_review",
            ],
        )
        self.assertEqual(rows["Q08"]["earliest"], "ambiguous")
        self.assertEqual(rows["Q09"]["selected_response"], "CS09GOOD")
        self.assertEqual(r.summary["in_window_yes"], 3)

    def test_owner_reassignment_preserves_evidence(self):
        a = worklist(ROOT / "data/ch23_mixed")
        b = worklist(ROOT / "data/ch23_reassigned")
        ra = a.tables["worklist.csv"][1]
```

```
        rb = b.tables["worklist.csv"][1]
        self.assertEqual(ra[1]["owner"], "B")
        self.assertEqual(rb[1]["owner"], "A")
        self.assertEqual(
            [{k: v for k, v in r.items() if k != "owner"} for r in ra],
            [{k: v for k, v in r.items() if k != "owner"} for r in rb],
        )

    def test_export_mapping_preserves_text_and_holds_ambiguous_date(self):
        from prepare_event_export import prepare

        d = ROOT / "data/export_example"
        out = self.root / "prepared"
        r = prepare(d / "office_export.csv", d / "matters.csv", out)
        self.assertEqual((r["source_rows"], r["accepted"], r["held"]), (4, 3, 1))
        rows = load_csv(out / "matter_events.csv", EVENT_FIELDS)
        self.assertEqual([x["event_id"] for x in rows], ["E001", "E002", "E004"])
        self.assertEqual(rows[0]["summary"], "Request records, first set")
        self.assertEqual(rows[1]["summary"], "Response received\nTwo attachments recorded")
        self.assertEqual(rows[0]["request_id"], "R001")

    def test_export_correction_is_recorded(self):
        from prepare_event_export import prepare

        d = ROOT / "data/export_example"
        out = self.root / "prepared"
        r = prepare(d / "office_export.csv", d / "matters.csv", out, d / "verified_dates.csv"
  )
        self.assertEqual((r["accepted"], r["held"]), (4, 0))
        with (out / "mapping_ledger.csv").open(newline="") as f:
            rows = list(csv.DictReader(f))
        self.assertEqual(
            [
                rows[2][k]
                for k in ["original_date", "mapped_date", "correction_source", "authorized_by
  "]
            ],
            ["09/05/2026", "2026-09-05", "CORR003", "Synthetic Reviewer"],
        )


if __name__ == "__main__":
    unittest.main()
```